Pontificia Universidad Javeriana

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Laboratorio 1

Implementación de un Monolito con patrón MVC y DAO

personapi-dotnet

.NET 10 (LTS) · ASP.NET Core MVC · Entity Framework Core 10
SQL Server 2025 · REST · Swagger / OpenAPI 3 · Docker

Integrantes

Sara Albarracín
Juan Diego Rojas
Miguel Sarmiento

Profesor

Andrés Armando Sánchez Martín

Arquitectura de Software

Bogotá D.C., Octubre de 2026

Índice

1. Marco conceptual **4**

1.1. Arquitectura de software 4
1.2. Arquitectura monolítica 4
1.3. Patrón Modelo–Vista–Controlador (MVC) 4
1.4. Patrón Data Access Object (DAO) y Repository 5
1.5. Plataforma .NET 5
1.6. ASP.NET Core 6
1.7. Entity Framework Core (ORM) 6
1.8. Microsoft SQL Server 6
1.9. REST 7
1.10. Swagger y OpenAPI 3 7
1.11. Docker 7
1.12. Git, repositorios y etiquetas 7

2. Diseño **8**

2.1. Requerimientos 8
2.2. Modelo de datos 8
2.3. Arquitectura de la solución 10
2.4. Modelo C4 11
2.5. Estructura del proyecto 13
2.6. Diseño de la API REST 14
2.7. Diseño de las vistas MVC 15
2.8. Diseño del despliegue 16

3. Procedimiento **16**

3.1. Paso 1. Crear el repositorio Git 17
3.2. Paso 2 y 3. Instalar SQL Server 2025 Express y SSMS 17
3.3. Paso 4 y 5. Crear la base de datos y las tablas (DDL) 18
3.4. Paso 6. Instalar Visual Studio Community 20
3.5. Paso 7. Clonar el repositorio 20

3.6. Paso 8. Desarrollo en Visual Studio 20
3.6.1. 8.1 – 8.4 Creación del proyecto 20
3.6.2. 8.5 – 8.6 Conexión desde el Explorador de objetos de SQL Server . . 21
3.6.3. 8.7 – 8.8 Instalación de paquetes NuGet 21
3.6.4. 8.9 – 8.10 Generación de entidades (*scaffolding*) 21
3.6.5. 8.11 Interfaces (contratos DAO) 21
3.6.6. 8.12 Repositorios (implementación DAO) 22
3.6.7. 8.13 Controladores 22
3.6.8. 8.14 Despliegue 24
3.7. Manual técnico de despliegue 25
3.7.1. Requisitos y versiones 25
3.7.2. Paso M1. Instalar Git 26
3.7.3. Paso M2. Instalar el SDK de .NET 10 30
3.7.4. Paso M3. Instalar SQL Server 2025 Express (camino A) 32
3.7.5. Paso M4. Instalar SSMS 22 y `sqlcmd` (camino A) 34
3.7.6. Paso M5. Instalar Visual Studio 2026 Community (opcional) 38
3.7.7. Paso M6. Instalar Docker Desktop (camino B) 41
3.7.8. Paso M7. Clonar el repositorio 42
3.7.9. Camino A · Paso M8. Crear `persona_db` 44
3.7.10. Camino A · Paso M9. Configurar la cadena de conexión 46
3.7.11. Camino A · Paso M10. Restaurar, compilar y probar 47
3.7.12. Camino A · Paso M11. Ejecutar la aplicación 48
3.7.13. Camino B · Paso M12. Crear el archivo `.env` 50
3.7.14. Camino B · Paso M13. Construir y levantar los contenedores 51
3.7.15. Camino B · Paso M14. Comprobar los contenedores 54
3.7.16. Camino B · Paso M15. Usar la aplicación 57
3.7.17. Camino B · Paso M16. Detener y limpiar 62
3.7.18. Verificación posterior al despliegue 63
3.7.19. Fallos frecuentes 64
3.7.20. Registro de verificación 66
3.8. Pruebas de funcionamiento 67
3.9. Paso 9 y 10. Publicación y etiqueta 70

4. Conclusiones y lecciones aprendidas **70**

4.1. Conclusiones 70

4.2. Lecciones aprendidas 70

4.3. Preguntas para el grupo 71

Referencias **71**

1. Marco conceptual

Este laboratorio construye `personapi-dotnet`, una aplicación monolítica que expone operaciones CRUD (*Create, Read, Update, Delete*) sobre un modelo de datos de personas, profesiones, estudios y teléfonos, tanto mediante vistas web (HTML generado en el servidor) como mediante una API REST documentada con Swagger. A continuación se presentan los conceptos que sustentan la solución.

1.1 Arquitectura de software

La arquitectura de software es la estructura (o conjunto de estructuras) de un sistema, compuesta por sus elementos, las propiedades visibles de estos y las relaciones entre ellos [1]. Las decisiones arquitectónicas determinan atributos de calidad como la mantenibilidad, la escalabilidad, la desplegabilidad y la facilidad de prueba, y son las más costosas de cambiar una vez el sistema está construido.

1.2 Arquitectura monolítica

Un **monolito** es una aplicación en la que toda la funcionalidad (presentación, lógica de negocio y acceso a datos) se compila y se despliega como *una sola unidad* ejecutable [2]. Sus principales características son:

- **Un único artefacto de despliegue**: en este caso, un ensamblado .NET publicado dentro de una imagen Docker.
- **Comunicación interna en proceso**: los componentes se invocan mediante llamadas a métodos, no por red, lo que reduce la latencia y la complejidad.
- **Base de datos compartida**: todos los módulos operan sobre el mismo esquema.

Un monolito no implica desorden: puede (y debe) estar *estructurado en capas*, de modo que cada capa tenga una responsabilidad clara y dependa únicamente de la capa inferior. Esta organización, conocida como **arquitectura en N capas** (*N-Tier / N-Layer*), es la recomendada por Microsoft para aplicaciones web no distribuidas, donde el *middle tier* se divide en presentación, lógica de negocio y acceso a datos [3].

1.3 Patrón Modelo–Vista–Controlador (MVC)

MVC es un patrón arquitectónico, propuesto originalmente por Trygve Reenskaug en Smalltalk-80 [4], que separa una aplicación interactiva en tres tipos de componentes:

Modelo Representa los datos y las reglas del dominio. Recupera y almacena el estado en un medio persistente (la base de datos).

Vista Presenta la información del modelo al usuario. En ASP.NET Core las vistas se escriben con el motor *Razor* (archivos `.cshtml`), que mezcla HTML con código C# precedido de `@`.

Controlador Recibe la petición del usuario, interactúa con el modelo y selecciona la vista (o la respuesta JSON) que se devolverá.

ASP.NET Core MVC implementa el patrón *Front Controller*: todas las peticiones HTTP entran por un único punto y el motor de enrutamiento las asigna a una clase controladora y a un método de acción según la URL (por convención `/{controller}/{action}/{id?}` o mediante *attribute routing* con `[Route]`). Las operaciones de escritura siguen un esquema de dos pasos: una acción `HttpGet` presenta el formulario y una acción `HttpPost` recibe los datos, que el *model binder* convierte automáticamente en parámetros u objetos fuertemente tipados [5].

Entre sus ventajas destacan la separación de responsabilidades (*separation of concerns*), el soporte nativo de inyección de dependencias e inversión de control, la facilidad para aplicar pruebas automatizadas y el control total sobre el HTML generado.

1.4 Patrón Data Access Object (DAO) y Repository

El patrón **DAO** abstrae y encapsula todo el acceso a la fuente de datos detrás de una interfaz [6]. El resto de la aplicación conoce únicamente la interfaz (por ejemplo `IPersonaRepository`) y no sabe si los datos provienen de SQL Server, de un archivo o de un servicio externo. Sus participantes son:

- **Cliente (BusinessObject)**: el servicio o controlador que necesita datos.
- **Interfaz DAO**: contrato con las operaciones disponibles (`GetAll`, `GetById`, `Add`, `Update`, `Delete`).
- **Implementación DAO**: clase concreta que usa la tecnología de persistencia (aquí, Entity Framework Core).
- **Objeto de transferencia / entidad**: el objeto que viaja entre capas.

En el ecosistema .NET este patrón suele denominarse **Repository** [7]; en este documento ambos términos se usan de forma equivalente, pues la intención es la misma: desacoplar la lógica de la aplicación de los detalles de persistencia. Al combinarlo con inyección de dependencias, la implementación concreta se registra una sola vez en `Program.cs` y puede sustituirse (por ejemplo, por un *mock* en pruebas) sin modificar los controladores.

1.5 Plataforma .NET

.NET es una plataforma de desarrollo gratuita, de código abierto y multiplataforma (Windows, Linux y macOS) que sucede al .NET Framework, exclusivo de Windows. Su núcleo es el

Common Language Runtime (CLR), implementación del estándar *Common Language Infrastructure* (CLI), que ejecuta el código intermedio (CIL) generado por los compiladores y provee servicios de seguridad, recolección de basura y manejo de excepciones; el código que se ejecuta dentro del CLR se denomina *managed code*.

Microsoft publica una versión mayor cada noviembre; las versiones pares son de soporte a largo plazo (LTS, tres años) y las impares de soporte estándar (STS) [8]. Siguiendo la indicación de usar las *últimas versiones estables*, se seleccionó **.NET 10 (LTS)**, publicado en noviembre de 2025 y con soporte hasta noviembre de 2028, en lugar de .NET 7, cuyo soporte terminó en mayo de 2024.

1.6 ASP.NET Core

ASP.NET Core es el *framework* web de .NET. Una misma aplicación puede contener controladores MVC que devuelven vistas Razor y controladores de API (`[ApiController]`) que devuelven JSON, lo que permite cubrir en un solo monolito tanto la interfaz web como la API REST exigidas en el laboratorio. Incluye un contenedor de inyección de dependencias, un sistema de configuración por capas (`appsettings.json`, variables de entorno) y el servidor web multiplataforma *Kestrel*.

1.7 Entity Framework Core (ORM)

Un **ORM** (*Object-Relational Mapper*) mapea las tablas de una base de datos relacional a clases del lenguaje y evita escribir código repetitivo de acceso a datos. Entity Framework Core es el ORM oficial de .NET [9]. Sus elementos principales son:

- **DbContext**: representa una sesión con la base de datos, implementa los patrones *Unit of Work* y *Identity Map* y rastrea los cambios.
- **DbSet<T>**: representa una tabla y permite consultas con LINQ, que EF traduce a SQL y cuyos resultados mapea a entidades.
- **Proveedor**: `Microsoft.EntityFrameworkCore.SqlServer` traduce las operaciones al dialecto T-SQL.

EF Core admite dos enfoques: *Code First* (las clases generan el esquema) y *Database First*. En este laboratorio se usa **Database First** mediante *scaffolding* (ingeniería inversa): el comando `Scaffold-DbContext` lee el esquema existente de `persona_db` y genera las clases entidad y el contexto.

1.8 Microsoft SQL Server

SQL Server es el sistema gestor de bases de datos relacionales de Microsoft. Se utilizó **SQL Server 2025** (versión 17.x), la versión estable más reciente, en su edición *Express* para la instalación local y mediante la imagen oficial `mcr.microsoft.com/mssql/server:2025-latest`

para el entorno Docker. Su administración se realiza con **SQL Server Management Studio (SSMS)**.

1.9 REST

REST (*Representational State Transfer*) es un estilo arquitectónico definido por Roy Fielding [11] para sistemas hipermedia distribuidos. Sus restricciones son: cliente-servidor, comunicación sin estado, respuestas cacheables, interfaz uniforme, sistema en capas y, opcionalmente, código bajo demanda. En una API REST sobre HTTP:

- Cada recurso se identifica con una URI (`/api/personas/123`).
- Los verbos HTTP expresan la operación: `GET` (leer), `POST` (crear), `PUT` (reemplazar/actualizar) y `DELETE` (eliminar).
- Los códigos de estado comunican el resultado: `200 OK`, `201 Created`, `204 No Content`, `400 Bad Request`, `404 Not Found`, `409 Conflict`.

1.10 Swagger y OpenAPI 3

OpenAPI es una especificación estándar, independiente del lenguaje, para describir APIs HTTP [12]; su versión 3 es la evolución de la antigua especificación Swagger 2.0. **Swagger UI** es la herramienta que, a partir de ese documento, genera una página web interactiva para explorar y probar los *endpoints*. En .NET se integra con el paquete `Swashbuckle.AspNetCore`, que genera el documento OpenAPI 3 en `/swagger/v1/swagger.json` y publica la interfaz en `/swagger`.

1.11 Docker

Docker empaqueta una aplicación junto con su entorno de ejecución en una **imagen** inmutable, que se ejecuta como **contenedor** aislado [14]. *Docker Compose* permite declarar en un archivo YAML varios servicios relacionados (aquí, la aplicación y SQL Server) y levantarlos con un solo comando. Esto elimina el problema de «funciona en mi máquina» y hace reproducible el despliegue del laboratorio en cualquier sistema operativo.

1.12 Git, repositorios y etiquetas

Git es un sistema de control de versiones distribuido. Un **tag** es una referencia inmutable a un *commit* concreto que se utiliza para marcar versiones entregables (por ejemplo `v1.0.0`); es el mecanismo de entrega solicitado para este laboratorio.

2. Diseño

2.1 Requerimientos

Funcionales

RF1. Crear, consultar (todos y por llave), actualizar y eliminar *personas*.

RF2. Crear, consultar, actualizar y eliminar *profesiones*.

RF3. Crear, consultar, actualizar y eliminar *teléfonos*, asociados a una persona.

RF4. Crear, consultar, actualizar y eliminar *estudios*, que relacionan una persona con una profesión.

RF5. Ofrecer las operaciones anteriores mediante vistas web (MVC) y mediante *endpoints* REST.

RF6. Documentar y permitir probar la API desde Swagger UI.

No funcionales

RNF1. Usar las últimas versiones estables: .NET 10 LTS y SQL Server 2025.

RNF2. Estructurar el monolito en capas con responsabilidades separadas.

RNF3. Aplicar los patrones MVC y DAO.

RNF4. Desplegar la solución con Docker y Docker Compose.

RNF5. Validar los datos de entrada y devolver códigos HTTP adecuados.

2.2 Modelo de datos

El modelo, proporcionado originalmente en MySQL, consta de cuatro tablas (Figura 1). La tabla `estudios` es una entidad asociativa que resuelve la relación muchos-a-muchos entre `persona` y `profesion`, con llave primaria compuesta (`id_prof`, `cc_per`). La tabla `telefono` mantiene una relación uno-a-muchos con `persona` a través de la columna `duenio`.

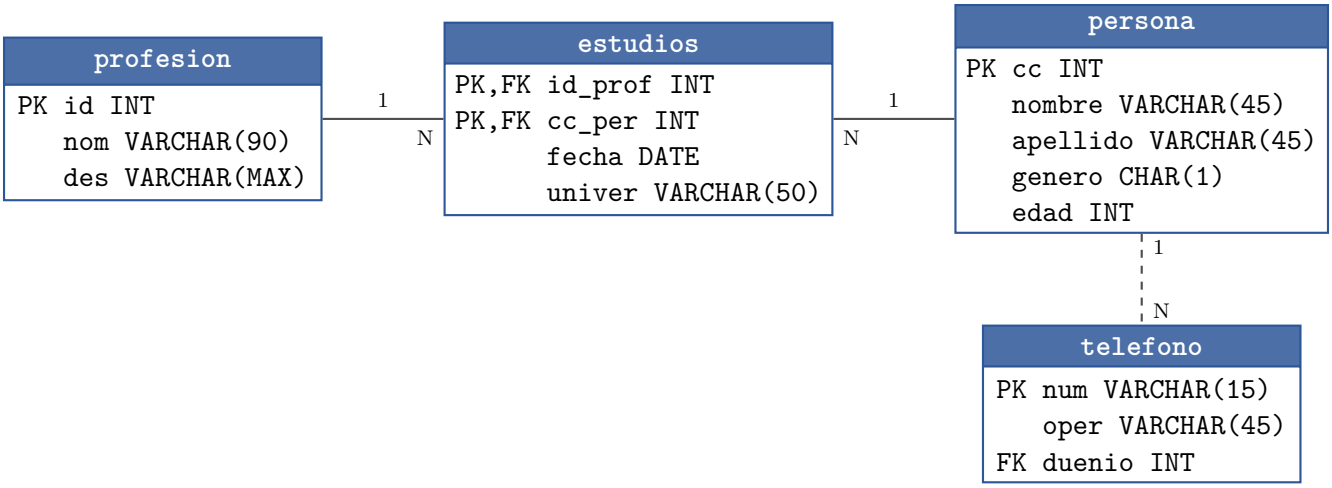


Figura 1: Modelo entidad-relación de `persona_db` (adaptado a SQL Server).

Como el script original está escrito para MySQL, fue necesario traducir los tipos y restricciones a T-SQL (Tabla 1).

Tabla 1: Adaptación de tipos de MySQL a SQL Server.

MySQL	**SQL Server**	**Justificación**
`INT(15)`, `INT(6)`, `INT(3)`	`INT`	En SQL Server `INT` no admite ancho de visualización; el rango ($\pm 2\,147\,483\,647$) cubre las cédulas colombianas.
`ENUM('M','F')`	`CHAR(1)` + `CHECK`	SQL Server no tiene `ENUM`; la restricción `CHECK (genero IN ('M','F'))` garantiza el mismo dominio.
`TEXT`	`VARCHAR(MAX)`	`TEXT` está obsoleto en SQL Server; `VARCHAR(MAX)` es su reemplazo.
`ENGINE=InnoDB`, `CHARSET latin1`	(se omite)	Opciones propias de MySQL; SQL Server usa la intercalación de la base de datos.

Las llaves foráneas conservan la semántica del modelo original (`NO ACTION`): no se puede eliminar una persona o profesión que tenga estudios o teléfonos asociados. La aplicación detecta este caso y responde con `409 Conflict` en la API y con un mensaje de error en la vista, en lugar de dejar propagar la excepción.

2.3 Arquitectura de la solución

La aplicación es un monolito organizado en cuatro capas lógicas dentro de un único proyecto ASP.NET Core (Figura 2). Cada capa solo depende de la capa inmediatamente inferior a través de interfaces, y las dependencias se resuelven mediante el contenedor de inyección de dependencias de ASP.NET Core.

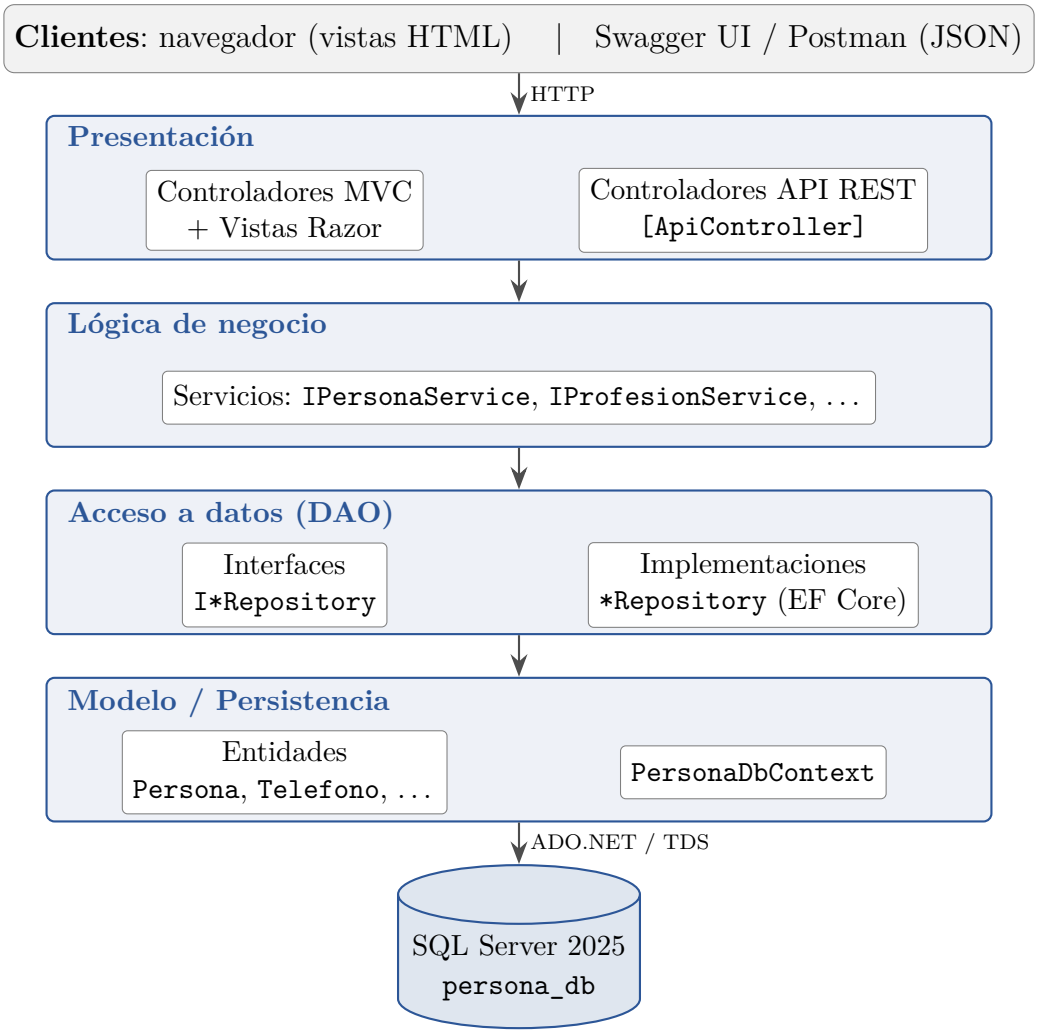


Figura 2: Arquitectura en capas del monolito `personapi-dotnet`.

Presentación Contiene los controladores MVC (que devuelven vistas Razor) y los controladores de API (que devuelven JSON). Ambos tipos de controlador reutilizan los mismos servicios, por lo que no se duplica lógica.

Lógica de negocio Servicios que aplican las reglas del dominio: verificar que la llave no exista antes de crear, que la persona dueña de un teléfono exista, que no se elimine un registro con dependencias, etc.

Acceso a datos (DAO) Interfaces que definen el contrato de persistencia e implementaciones que lo cumplen usando EF Core. Es la única capa que conoce el `DbContext`.

Modelo / Persistencia Entidades y contexto generados por *scaffolding* desde la base de datos.

2.4 Modelo C4

El modelo C4 describe la misma arquitectura en niveles de detalle creciente. Los cuatro diagramas se derivan del SAD y del SDD de la wiki y se comprueban contra el código del repositorio; sus fuentes en texto se versionan en `docs/wiki/02-arquitectura/diagramas/` y las imágenes exportadas se guardan junto a este documento, en `diagramas/`. Los tres primeros niveles salen del modelo en Structurizr DSL; el cuarto, que es un diagrama de clases, se escribe en PlantUML, porque Structurizr no dibuja el nivel de código.

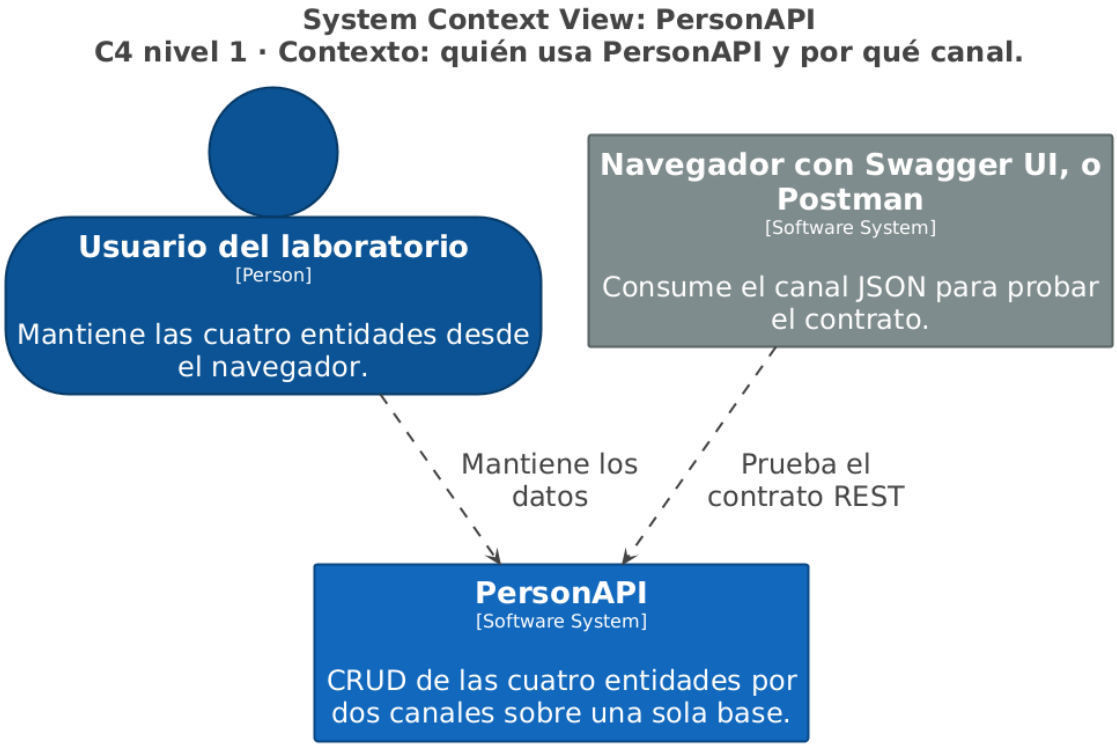


Figura 3: C4 nivel 1 (contexto): el usuario, el monolito `personapi-dotnet` y SQL Server `persona_db` como único almacén.

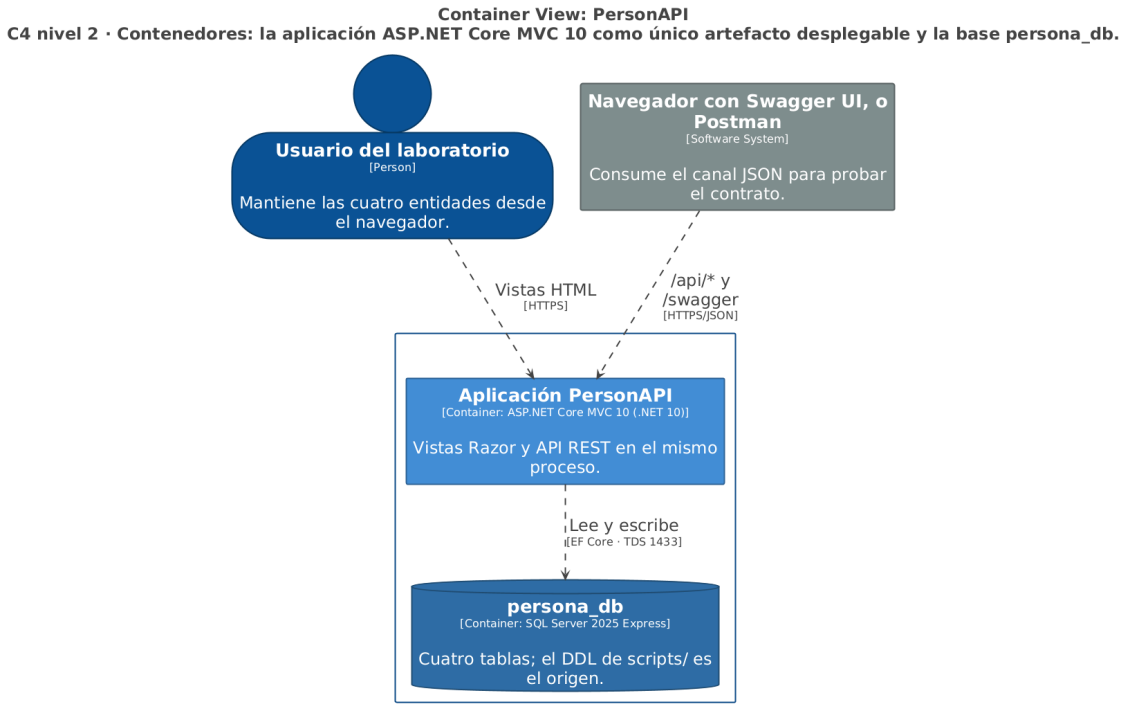


Figura 4: C4 nivel 2 (contenedores): la aplicación ASP.NET Core MVC 10 como único artefacto desplegable, con sus dos canales (vistas Razor y API REST), y la base `persona_db`.

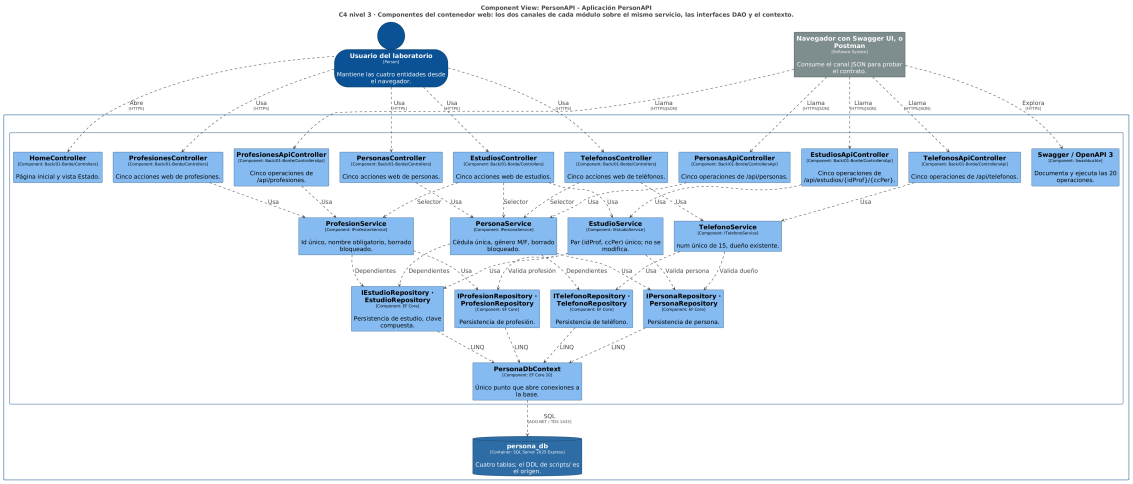


Figura 5: C4 nivel 3 (componentes) del contenedor web: controladores MVC y API, servicios, interfaces DAO e implementaciones EF Core.

El cuarto nivel es, por definición, un diagrama de clases del interior de un componente: el **diagrama de clases de la solución y el nivel 4 del modelo C4 son el mismo documento**, no dos vistas distintas, y por eso aparece una sola vez. La Figura 6 aplica el patrón DAO a la entidad `Persona`; las otras tres entidades siguen exactamente la misma estructura.

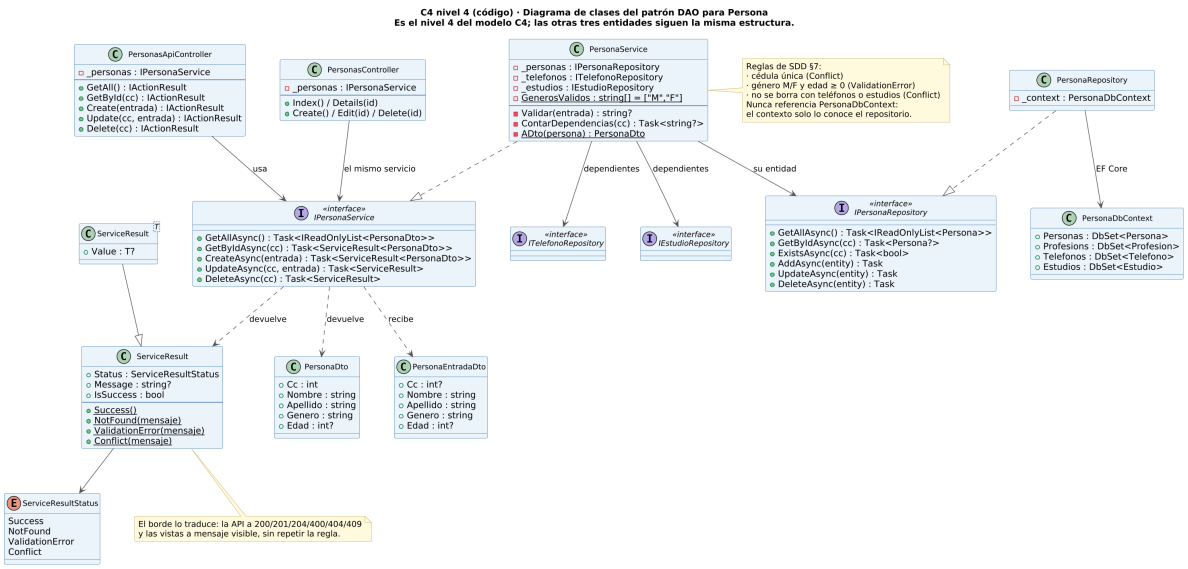


Figura 6: C4 nivel 4 (código): diagrama de clases del patrón DAO para `Persona`.

La cadena que allí se ve es la que exige la arquitectura, y se lee de arriba abajo:

1. el controlador —de vistas o de API— depende de la interfaz `IPersonaService` y nunca de su implementación;
2. `PersonaService` implementa esa interfaz y concentra las reglas del dominio;
3. ese servicio depende de **tres** interfaces de repositorio: la de su propia entidad, `IPersonaRepository`, y las de teléfono y estudio, que necesita para comprobar dependientes antes de permitir un borrado;
4. cada `*Repository` implementa su interfaz y es el único punto que conoce `PersonaDbContext`, de modo que EF Core no se filtra hacia arriba;
5. el resultado de cada operación viaja en `ServiceResult`, que el borde traduce a código HTTP en la API y a mensaje visible en las vistas, sin repetir la regla.

2.5 Estructura del proyecto

```
personapi-dotnet/
|-- personapi-dotnet.sln
|-- README.md
|-- Dockerfile
|-- docker-compose.yml
|-- .dockerignore
|-- scripts/
|   |-- 01_ddl.sql                  # Creacion de BD y tablas
|   `-- 02_dml.sql                  # Datos de prueba
`-- personapi-dotnet/
    |-- Program.cs                  # Configuracion, DI, Swagger
```

```
|-- appsettings.json            # Cadena de conexion
|-- README.md                   # Mapa de capas
|-- Front/                      # Lo que ve el navegador (Razor
   server-side)
|   |-- Views/
|   |   |-- Shared/_Layout.cshtml
|   |   |-- Personas/  (Index, Details, Create, Edit, Delete)
|   |   |-- Profesiones/ ...
|   |   |-- Telefonos/ ...
|   |   `-- Estudios/ ...
|   |-- wwwroot/                #   CSS, JS, imagenes
|   `-- ViewModels/             #   Contrato de Razor
`-- Back/                       # Logica: cuatro capas
    |-- 01-Borde/               # CAPA 1: unico sitio que habla HTTP
    |   |-- Controllers/        #   MVC
    |   |   |-- HomeController.cs
    |   |   |-- PersonasController.cs
    |   |   |-- ProfesionesController.cs
    |   |   |-- TelefonosController.cs
    |   |   `-- EstudiosController.cs
    |   |-- ControllersApi/     #   REST /api/*
    |   |   |-- PersonasApiController.cs
    |   |   |-- ProfesionesApiController.cs
    |   |   |-- TelefonosApiController.cs
    |   |   `-- EstudiosApiController.cs
    |   `-- Dtos/               #   Contrato de REST
    |-- 02-Aplicacion/          # CAPA 2: reglas de negocio
    |   `-- Services/
    |       |-- Interfaces/  (IPersonaService.cs, ...)
    |       `-- Implementations/  (PersonaService.cs, ...)
    |-- 03-AccesoDatos/         # CAPA 3: DAO, unico sitio con EF
       Core
    |   `-- Repository/
    |       |-- Interfaces/  (IPersonaRepository.cs, ...)
    |       `-- Implementations/  (PersonaRepository.cs, ...)
    `-- 04-Persistencia/        # CAPA 4: Database First
        `-- Entities/           #   Generado por Scaffold-DbContext
            |-- PersonaDbContext.cs
            |-- Persona.cs
            |-- Profesion.cs
            |-- Estudio.cs
            `-- Telefono.cs
```

Listing 1: Estructura de carpetas del repositorio.

2.6 Diseño de la API REST

Todos los recursos siguen la convención `/api/{recurso}`. La entidad `estudios`, por tener llave compuesta, recibe ambos componentes en la ruta.

Tabla 2: Endpoints REST expuestos por la aplicación.

Método	Ruta	Descripción	Éxito
GET	/api/personas	Lista todas las personas	200
GET	/api/personas/{cc}	Obtiene una persona	200 / 404
POST	/api/personas	Crea una persona	201 / 409
PUT	/api/personas/{cc}	Actualiza una persona	204 / 404
DELETE	/api/personas/{cc}	Elimina una persona	204 / 409
GET	/api/profesiones	Lista todas las profesiones	200
GET	/api/profesiones/{id}	Obtiene una profesión	200 / 404
POST	/api/profesiones	Crea una profesión	201 / 409
PUT	/api/profesiones/{id}	Actualiza una profesión	204 / 404
DELETE	/api/profesiones/{id}	Elimina una profesión	204 / 409
GET	/api/telefonos	Lista todos los teléfonos	200
GET	/api/telefonos/{num}	Obtiene un teléfono	200 / 404
POST	/api/telefonos	Crea un teléfono	201 / 400
PUT	/api/telefonos/{num}	Actualiza un teléfono	204 / 404
DELETE	/api/telefonos/{num}	Elimina un teléfono	204 / 404
GET	/api/estudios	Lista todos los estudios	200
GET	/api/estudios/{idProf}/{ccPer}	Obtiene un estudio	200 / 404
POST	/api/estudios	Crea un estudio	201 / 400
PUT	/api/estudios/{idProf}/{ccPer}	Actualiza un estudio	204 / 404
DELETE	/api/estudios/{idProf}/{ccPer}	Elimina un estudio	204 / 404

2.7 Diseño de las vistas MVC

Para cada entidad existe un controlador MVC con las cinco acciones estándar de *scaffolding*, cada una asociada a una vista Razor:

Tabla 3: Rutas de las vistas web (ejemplo con `Personas`).

Ruta	Verbo	Vista / acción
/Personas	GET	`Index.cshtml`: tabla con todos los registros
/Personas/Details/{id}	GET	`Details.cshtml`: detalle de un registro
/Personas/Create	GET, POST	`Create.cshtml`: formulario de creación
/Personas/Edit/{id}	GET, POST	`Edit.cshtml`: formulario de edición
/Personas/Delete/{id}	GET, POST	`Delete.cshtml`: confirmación de borrado

Las listas desplegables para seleccionar la persona dueña de un teléfono o la profesión de un estudio se construyen con `SelectList`, de modo que el usuario nunca escribe manualmente

una llave foránea.

2.8 Diseño del despliegue

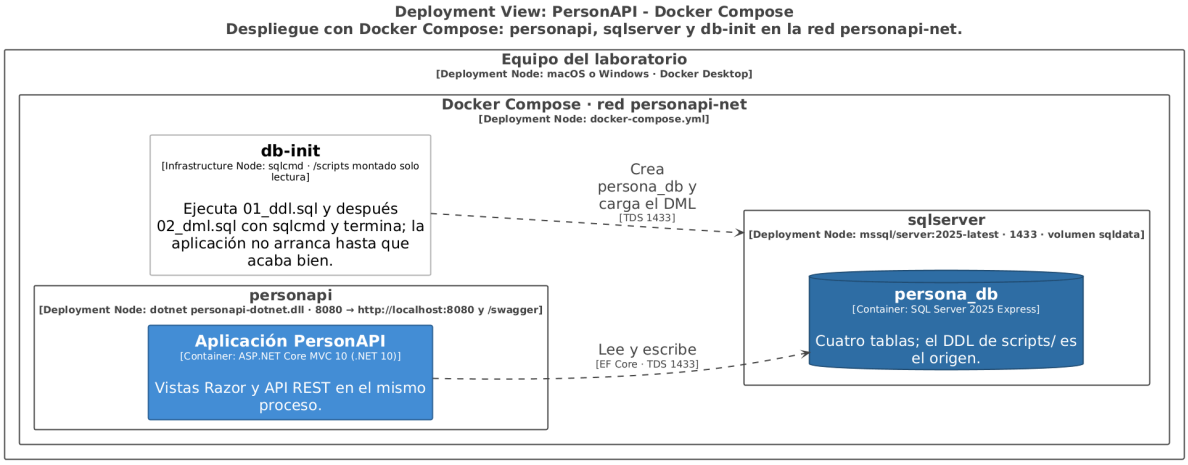


Figura 7: Vista de despliegue con Docker Compose: los tres servicios de `docker-compose.yml` en la red `personapi-net`, con su imagen, el puerto que publican, el volumen de datos y las URL que quedan disponibles. `db-init` ejecuta el DDL y el DML y termina; la aplicación no arranca hasta que ese servicio acaba bien.

3. Procedimiento

El procedimiento sigue la guía del laboratorio. Donde la guía menciona versiones anteriores (SQL Server 2019, SSMS 18, Visual Studio 2022, .NET 6/7) se usaron las versiones estables más recientes, según lo solicitado (Tabla 4).

Tabla 4: Versiones de herramientas utilizadas.

Herramienta	**Guía**	**Utilizada**
SDK de .NET	.NET 6 / 7	.NET 10 (LTS)
Motor de base de datos	SQL Server 2019 Express	SQL Server 2025 Express / imagen Docker 2025
Cliente de base de datos	SSMS 18	SSMS 22
IDE	Visual Studio 2022	Visual Studio 2026 Community (requerido para .NET 10)
ORM	EF Core	EF Core 10
Documentación de API	Swagger 3	Swashbuckle.AspNetCore (OpenAPI 3)
Contenedores	–	Docker Desktop + Docker Compose v2

3.1 Paso 1. Crear el repositorio Git

Se creó en GitHub el repositorio público `AS-personapi-dotnet`, inicializado con un `README.md` y un `.gitignore` para Visual Studio. La solución que contiene se llama `personapi-dotnet`.

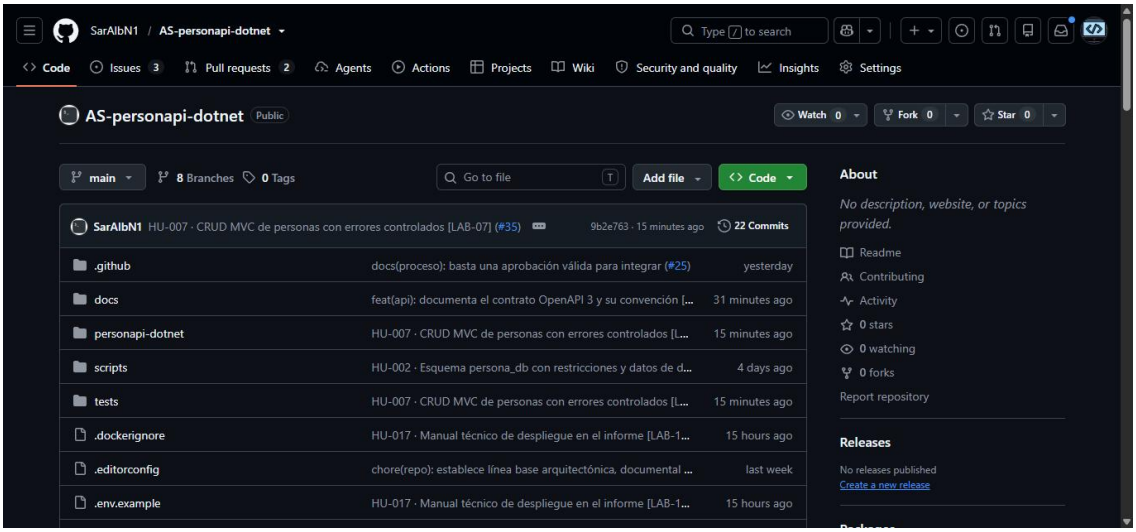


Figura 8: Paso 1: repositorio `AS-personapi-dotnet` en GitHub, con la rama `main` y las carpetas `docs/`, `personapi-dotnet/`, `scripts/` y `tests/`.

3.2 Paso 2 y 3. Instalar SQL Server 2025 Express y SSMS

1. Descargar el instalador de SQL Server 2025 Express desde el sitio de Microsoft y elegir el tipo de instalación **Básica**. La instancia queda registrada como `localhost\SQLEXPRESS`.
2. Habilitar la autenticación mixta (Windows y SQL Server) y asignar contraseña al usuario `sa`, necesario para conectarse desde Docker.
3. Instalar SQL Server Management Studio (SSMS) y conectarse a la instancia.

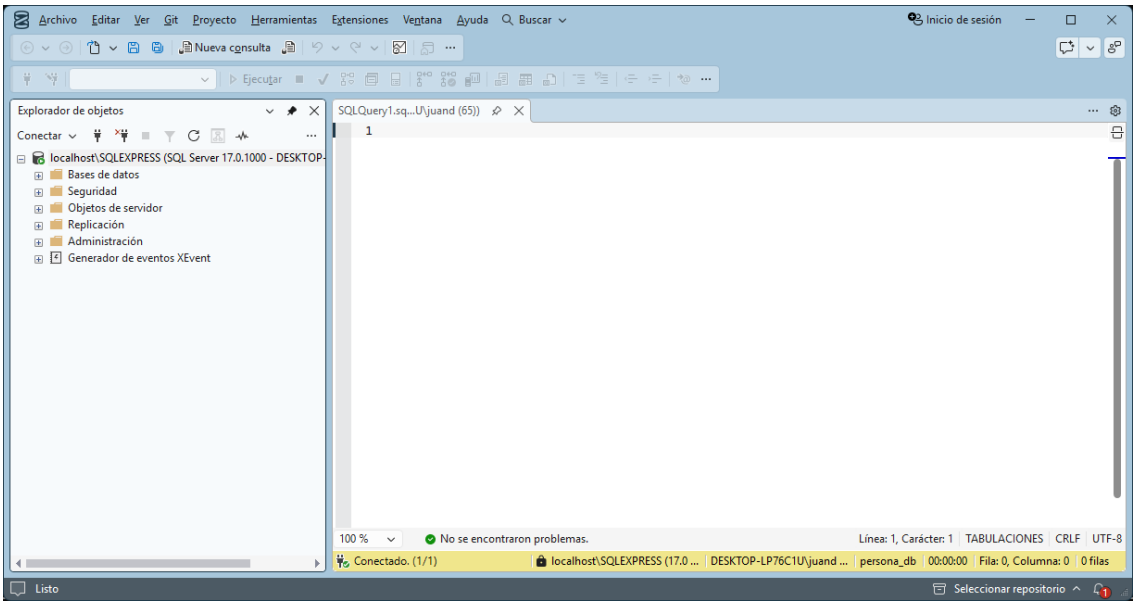


Figura 9: Pasos 2 y 3: SSMS 22 conectado a `localhost\SQLEXPRESS` (SQL Server 17.0) con autenticación de Windows; la barra de estado indica «Conectado» y la base `persona_db`.

3.3 Paso 4 y 5. Crear la base de datos y las tablas (DDL)

Se creó la base de datos `persona_db`, se asignó como propietario al usuario `sa` y se crearon las tablas según el modelo, con las adaptaciones de la Tabla 1.

```
IF DB_ID('persona_db') IS NULL
    CREATE DATABASE persona_db;
GO
ALTER AUTHORIZATION ON DATABASE::persona_db TO sa;
GO
USE persona_db;
GO

CREATE TABLE persona (
    cc         INT           NOT NULL,
    nombre     VARCHAR(45)   NOT NULL,
    apellido   VARCHAR(45)   NOT NULL,
    genero     CHAR(1)       NOT NULL,
    edad       INT           NULL,
    CONSTRAINT PK_persona PRIMARY KEY (cc),
    CONSTRAINT CK_persona_genero CHECK (genero IN ('M', 'F'))
);

CREATE TABLE profesion (
    id    INT            NOT NULL,
    nom   VARCHAR(90)    NOT NULL,
    des   VARCHAR(MAX)   NULL,
    CONSTRAINT PK_profesion PRIMARY KEY (id)
);

```

```
CREATE TABLE estudios (
    id_prof   INT          NOT NULL,
    cc_per    INT          NOT NULL,
    fecha     DATE         NULL,
    univer    VARCHAR(50)  NULL,
    CONSTRAINT PK_estudios PRIMARY KEY (id_prof, cc_per),
    CONSTRAINT estudio_persona_fk   FOREIGN KEY (cc_per)  REFERENCES
       persona (cc),
    CONSTRAINT estudio_profesion_fk FOREIGN KEY (id_prof) REFERENCES
       profesion (id)
);
CREATE INDEX estudio_persona_fk ON estudios (cc_per);

CREATE TABLE telefono (
    num      VARCHAR(15)  NOT NULL,
    oper     VARCHAR(45)  NOT NULL,
    duenio   INT          NOT NULL,
    CONSTRAINT PK_telefono PRIMARY KEY (num),
    CONSTRAINT telefono_persona_fk FOREIGN KEY (duenio) REFERENCES
       persona (cc)
);
CREATE INDEX telefono_persona_fk ON telefono (duenio);
GO
```

Listing 2: Script DDL `scripts/01_ddl.sql`.

```
USE persona_db;
GO
INSERT INTO persona (cc, nombre, apellido, genero, edad) VALUES
 (1010101010, 'Ana',    'Gómez',    'F', 28),
 (1020202020, 'Carlos', 'Pérez',    'M', 35),
 (1030303030, 'Laura',  'Martínez', 'F', 22);

INSERT INTO profesion (id, nom, des) VALUES
 (1, 'Ingeniería de Sistemas', 'Diseño y desarrollo de sistemas de
    información'),
 (2, 'Medicina',               'Diagnóstico y tratamiento de
    enfermedades'),
 (3, 'Derecho',                'Asesoría y representación legal');

INSERT INTO estudios (id_prof, cc_per, fecha, univer) VALUES
 (1, 1010101010, '2020-12-15', 'Pontificia Universidad Javeriana'),
 (2, 1020202020, '2015-06-20', 'Universidad Nacional de Colombia'),
 (1, 1030303030, '2024-07-01', 'Universidad de los Andes');

INSERT INTO telefono (num, oper, duenio) VALUES
 ('3001234567', 'Claro',    1010101010),
 ('3109876543', 'Movistar', 1020202020),
 ('3205551234', 'Tigo',     1030303030);
GO
```

Listing 3: Script DML `scripts/02_dml.sql` (extracto).

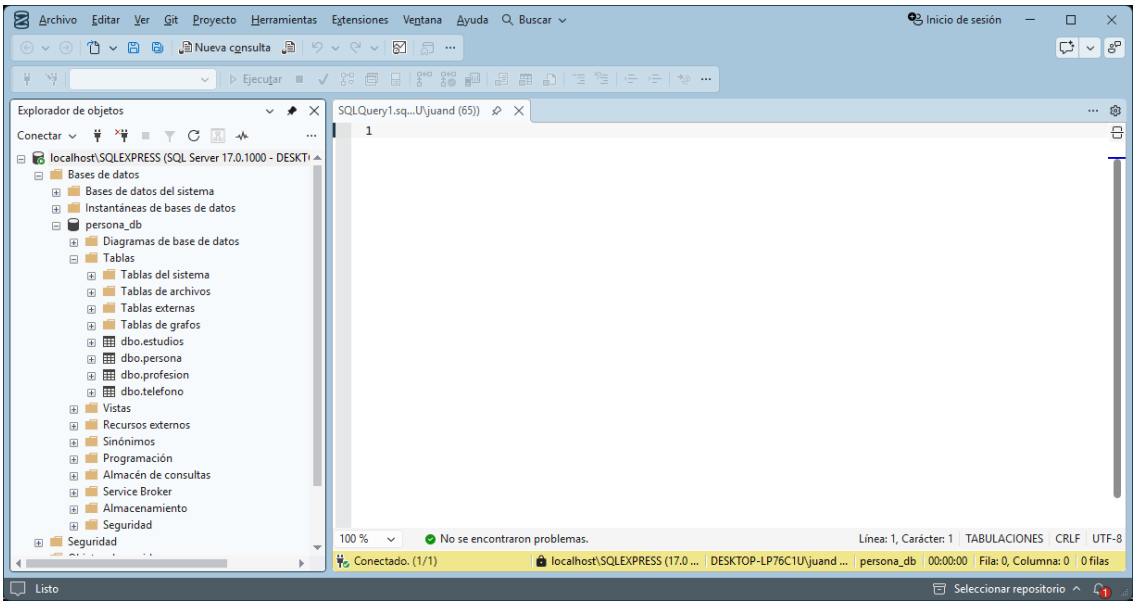


Figura 10: Paso 5: Explorador de objetos de SSMS con las cuatro tablas de `persona_db`: `dbo.estudios`, `dbo.persona`, `dbo.profesion` y `dbo.telefono`.

3.4 Paso 6. Instalar Visual Studio Community

Se instaló Visual Studio 2026 Community con las cargas de trabajo:

1. Desarrollo de ASP.NET y web.
2. Almacenamiento y procesamiento de datos.
3. Plantillas de proyecto y elementos de .NET Framework.
4. Características avanzadas de ASP.NET.

3.5 Paso 7. Clonar el repositorio

```
git clone https://github.com/USUARIO/personapi-dotnet.git
cd personapi-dotnet
```

3.6 Paso 8. Desarrollo en Visual Studio

3.6.1 8.1 – 8.4 Creación del proyecto

Se creó un proyecto con la plantilla **Aplicación web de ASP.NET Core (Modelo-Vista-Controlador)**, con nombre `personapi-dotnet`, *framework* **.NET 10.0**, sin autenticación y sin configuración HTTPS. Equivalente por línea de comandos:

```
dotnet new sln -n personapi-dotnet
dotnet new mvc -n personapi-dotnet -f net10.0 --auth None --no-https
dotnet sln add personapi-dotnet/personapi-dotnet.csproj
```

3.6.2 8.5 – 8.6 Conexión desde el Explorador de objetos de SQL Server

Desde **Ver → Explorador de objetos de SQL Server** se agregó la conexión a `localhost\SQLEXPRESS` y se verificó con *Probar conexión*.

3.6.3 8.7 – 8.8 Instalación de paquetes NuGet

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Swashbuckle.AspNetCore
```

3.6.4 8.9 – 8.10 Generación de entidades (*scaffolding*)

Se creó la carpeta `Back/04-Persistencia/Entities` y en la Consola del Administrador de paquetes se ejecutó:

```
Scaffold-DbContext "Server=localhost\SQLEXPRESS;Database=persona_db;
    Trusted_Connection=True;TrustServerCertificate=true" Microsoft.
    EntityFrameworkCore.SqlServer -OutputDir Back/04-Persistencia/
    Entities
```

El comando generó `PersonaDbContext` y las clases `Persona`, `Profesion`, `Estudio` y `Telefono`, con sus propiedades de navegación. Luego se movió la cadena de conexión del contexto a `appsettings.json`:

```
{
  "ConnectionStrings": {
    "PersonaDb": "Server=localhost\\SQLEXPRESS;Database=persona_db;
       Trusted_Connection=True;TrustServerCertificate=true"
  }
}
```

Listing 4: Fragmento de `appsettings.json`.

3.6.5 8.11 Interfaces (contratos DAO)

```
public interface IPersonaRepository
{
    Task<IEnumerable<Persona>> GetAllAsync();
    Task<Persona?> GetByIdAsync(int cc);
```

```
    Task<bool> ExistsAsync(int cc);
    Task AddAsync(Persona persona);
    Task UpdateAsync(Persona persona);
    Task DeleteAsync(Persona persona);
}
```

Listing 5: Back/03-AccesoDatos/Repository/Interfaces/IPersonaRepository.cs.

3.6.6 8.12 Repositorios (implementación DAO)

```
public class PersonaRepository : IPersonaRepository
{
    private readonly PersonaDbContext _context;

    public PersonaRepository(PersonaDbContext context) => _context =
        context;

    public async Task<IEnumerable<Persona>> GetAllAsync() =>
        await _context.Personas.AsNoTracking().ToListAsync();

    public async Task<Persona?> GetByIdAsync(int cc) =>
        await _context.Personas.FindAsync(cc);

    public Task<bool> ExistsAsync(int cc) =>
        _context.Personas.AnyAsync(p => p.Cc == cc);

    public async Task AddAsync(Persona persona)
    {
        _context.Personas.Add(persona);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Persona persona)
    {
        _context.Personas.Update(persona);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(Persona persona)
    {
        _context.Personas.Remove(persona);
        await _context.SaveChangesAsync();
    }
}
```

Listing 6: Back/03-AccesoDatos/Repository/Implementations/PersonaRepository.cs.

3.6.7 8.13 Controladores

Los controladores MVC se generaron con *Agregar → Nuevo elemento con scaffolding → Controlador de MVC con vistas que usan Entity Framework* y se modificaron para depender

del servicio en lugar del `DbContext`. Los controladores de API se escribieron manualmente:

```
[ApiController]
[Route("api/personas")]
public class PersonasApiController : ControllerBase
{
    private readonly IPersonaService _service;

    public PersonasApiController(IPersonaService service) => _service =
        service;

    [HttpGet]
    public async Task<IActionResult> GetAll() => Ok(await _service.
        GetAllAsync());

    [HttpGet("{cc:int}")]
    public async Task<IActionResult> Get(int cc)
    {
        var persona = await _service.GetByIdAsync(cc);
        return persona is null ? NotFound() : Ok(persona);
    }

    [HttpPost]
    public async Task<IActionResult> Post(Persona persona)
    {
        if (!await _service.CreateAsync(persona))
            return Conflict($"Ya existe una persona con cc {persona.Cc}"
                );
        return CreatedAtAction(nameof(Get), new { cc = persona.Cc },
            persona);
    }

    [HttpPut("{cc:int}")]
    public async Task<IActionResult> Put(int cc, Persona persona)
    {
        if (cc != persona.Cc) return BadRequest();
        return await _service.UpdateAsync(cc, persona) ? NoContent() :
            NotFound();
    }

    [HttpDelete("{cc:int}")]
    public async Task<IActionResult> Delete(int cc)
    {
        var result = await _service.DeleteAsync(cc);
        return result switch
        {
            DeleteResult.NotFound      => NotFound(),
            DeleteResult.HasDependents => Conflict("La persona tiene
                estudios o teléfonos asociados"),
            _                          => NoContent()
        };
    }
}
```

Listing 7: `Back/01-Borde/ControllersApi/PersonasApiController.cs`.

El registro de dependencias y la configuración de Swagger se realizan en `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews()
    .AddJsonOptions(o => o.JsonSerializerOptions.ReferenceHandler =
        System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles);

builder.Services.AddDbContext<PersonaDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("
       PersonaDb")));

// Capa de acceso a datos (DAO)
builder.Services.AddScoped<IPersonaRepository, PersonaRepository>();
builder.Services.AddScoped<IProfesionRepository, ProfesionRepository>();
builder.Services.AddScoped<ITelefonoRepository, TelefonoRepository>();
builder.Services.AddScoped<IEstudioRepository, EstudioRepository>();

// Capa de lógica de negocio
builder.Services.AddScoped<IPersonaService, PersonaService>();
builder.Services.AddScoped<IProfesionService, ProfesionService>();
builder.Services.AddScoped<ITelefonoService, TelefonoService>();
builder.Services.AddScoped<IEstudioService, EstudioService>();

// Swagger / OpenAPI 3
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
    c.SwaggerDoc("v1", new() { Title = "personapi-dotnet", Version = "v1
       " }));

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Listing 8: Fragmento de `Program.cs`.

3.6.8 8.14 Despliegue

El proyecto se despliega de dos formas: como proceso local de ASP.NET Core contra SQL Server 2025 Express (perfil `http`, http://localhost:5093) o con Docker Compose (http://localhost:8080). Los dos caminos, con todos sus pasos desde la instalación de las herramientas, se describen en el manual técnico de la sección 3.7.

3.7 Manual técnico de despliegue

Este manual permite que cualquier persona, partiendo de un equipo con Windows 10 u 11 de 64 bits sin las herramientas instaladas, despliegue `personapi-dotnet` y compruebe que funciona, sin consultar a los autores. Cada paso indica qué se hace, el comando exacto, el resultado esperado y la captura que lo evidencia.

Hay dos caminos, y ambos parten de los pasos M1 a M7:

Camino A · ejecución local La aplicación corre con `dotnet run` contra la instancia local `localhost\SQLEXPRESS`, con autenticación integrada de Windows. Es el camino con el que se verifican las historias del proyecto (pasos M8 a M11).

Camino B · Docker Compose Un solo comando levanta SQL Server 2025, carga los scripts y publica la aplicación en contenedores. No requiere SQL Server instalado en el equipo (pasos M12 a M16).

El `README.md` del repositorio sigue siendo la guía operativa breve; este manual la desarrolla paso a paso y no la sustituye. **Ninguna captura ni comando de este manual muestra contraseñas**: la del usuario `sa` del camino B vive solo en el archivo `.env`, que Git ignora.

3.7.1 Requisitos y versiones

Tabla 5: Herramientas del despliegue y versiones con las que se verificó el manual.

Herramienta	Camino	Versión verificada
Git	A y B	2.53 (`git -version`)
SDK de .NET	A	10.0.402 (`dotnet -version`); cualquier SDK 10.0.x
SQL Server 2025 Express	A	17.0.1000.7, Express Edition (64-bit)
SSMS	A	SQL Server Management Studio 22
`sqlcmd`	A	`sqlcmd` de Microsoft (versión Go)
Visual Studio 2026 Community	A (opcional)	Solo para editar y depurar; no es necesario para desplegar
Docker Desktop	B	Docker Engine 29.8.1 y Docker Compose v5.5.1, motor Linux sobre WSL 2
Imágenes Docker	B	`mcr.microsoft.com/dotnet/sdk:10.0`, `mcr.microsoft.com/dotnet/aspnet:10.0` y `mcr.microsoft.com/mssql/server:2025-latest` (SQL Server 17.0.5035.5, Express)

Recursos mínimos para el camino B: 4 GB de RAM libres para Docker y unos 5 GB de disco para las imágenes. La primera construcción descarga las imágenes y tarda varios minutos; las siguientes, segundos.

3.7.2 Paso M1. Instalar Git

1. Descargar el instalador de https://git-scm.com/downloads y ejecutarlo con las opciones por defecto.
2. Abrir una terminal nueva (PowerShell) y comprobar la instalación:

```
git --version
# git version 2.53.0.windows.2
```

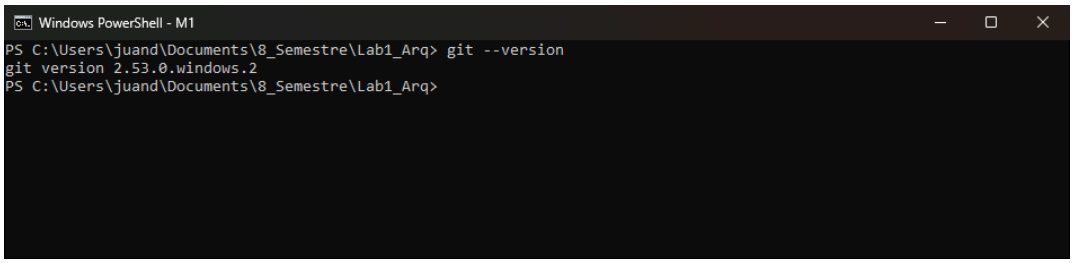


Figura 11: Paso M1: Git instalado.

Instalación por terminal (`winget`). El mismo paso se puede hacer desde PowerShell con el gestor de paquetes de Windows, que descarga el instalador oficial, comprueba su hash y lo ejecuta. Si la herramienta ya está instalada, `winget` lo detecta y cancela la reinstalación, como ocurrió en el equipo de verificación:

```
winget search --id Git.Git --source winget
winget install --id Git.Git -e --source winget --accept-package-
    agreements --accept-source-agreements
```

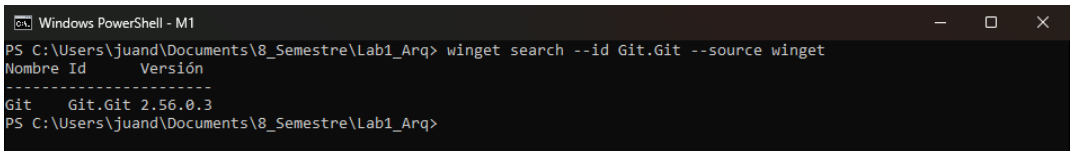


Figura 12: Paso M1: `winget search` encuentra el paquete `Git.Git`.

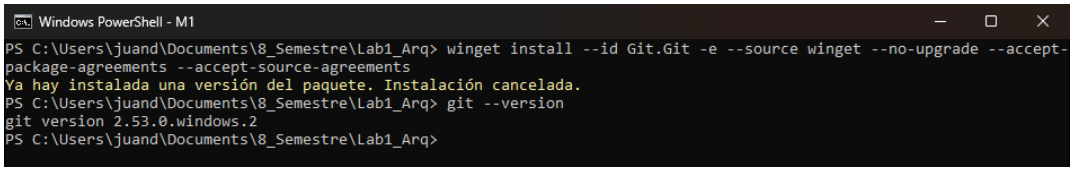


Figura 13: Paso M1: `winget install` de Git; Git ya estaba instalado y `winget` canceló la reinstalación.

Asistente gráfico de Git. Quien prefiera el instalador descargado de la web pasa por las páginas siguientes. En todas se deja la opción que viene marcada y se pulsa *Next*; las que importan para este proyecto son la del PATH, que debe quedar en *Git from the command line and also from 3rd-party software* para que PowerShell y Visual Studio encuentren `git`, y la de finales de línea, *Checkout Windows-style, commit Unix-style line endings*. Las capturas son del instalador 2.56.0.3 que descarga `winget` en la fecha de verificación; las páginas y sus valores por defecto son los mismos que en la 2.53. Notas sobre el equipo de verificación:

- Como Git ya estaba instalado, el asistente reutiliza la carpeta de la instalación existente y no muestra la página de carpeta de destino; en un equipo limpio aparece antes de la de componentes.
- Con Git instalado, la casilla *Only show new options* viene marcada y salta directo a la instalación; para recorrer todas las páginas hay que desmarcarla en la primera.
- El recorrido se canceló en la última página, antes de pulsar *Install*, así que la instalación existente no cambió.

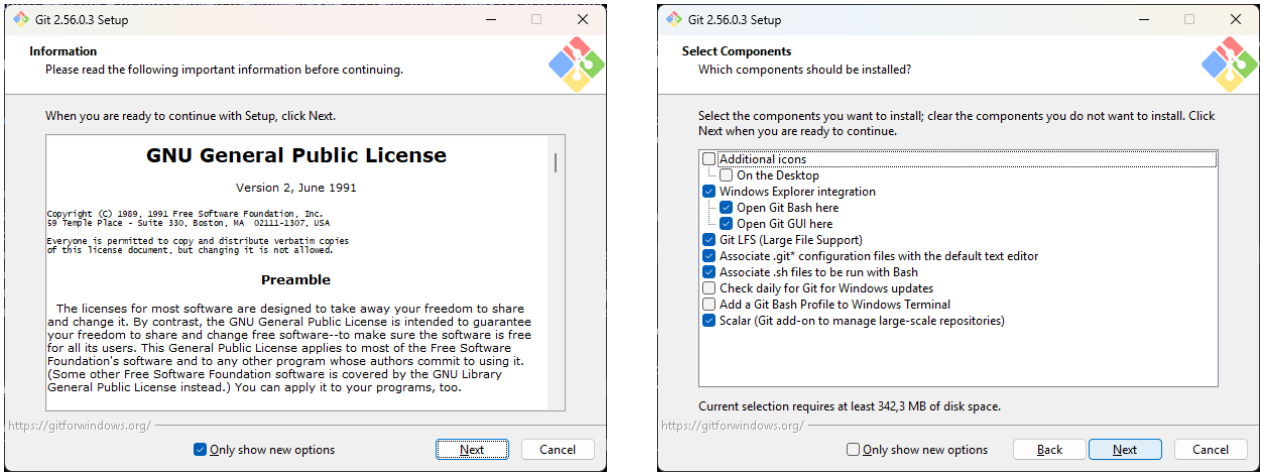


Figura 14: Paso M1, asistente de Git: licencia y componentes.

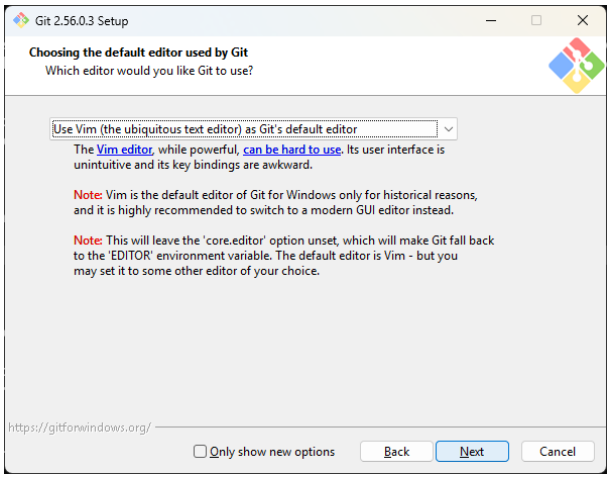


Figura 15: Paso M1, asistente de Git: editor por defecto y nombre de la rama inicial.

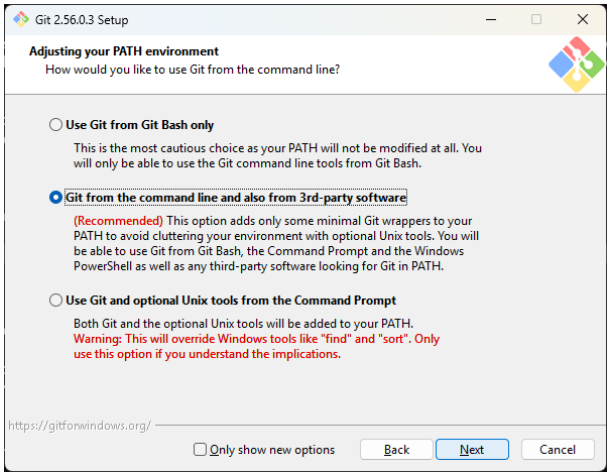


Figura 16: Paso M1, asistente de Git: PATH (opción recomendada) y cliente SSH.

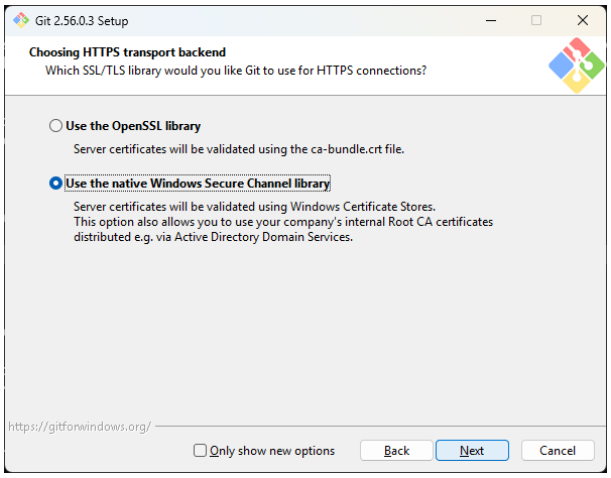
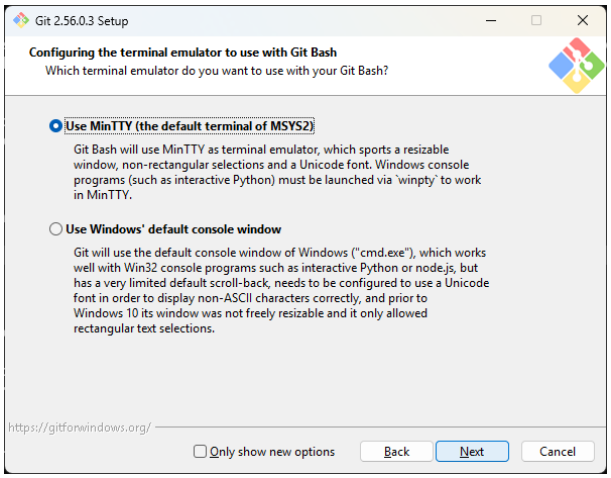


Figura 17: Paso M1, asistente de Git: biblioteca HTTPS y conversión de finales de línea.



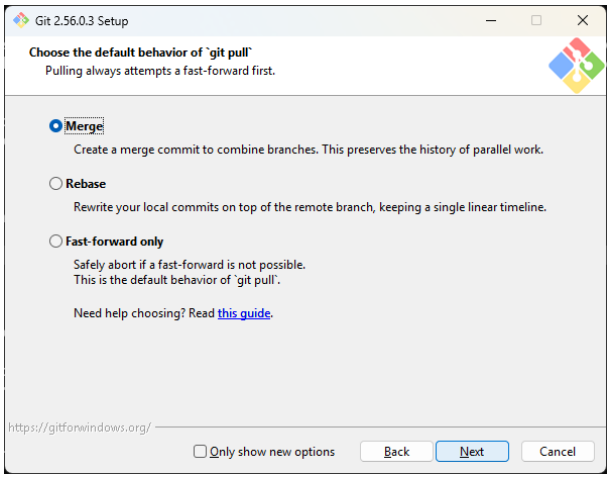
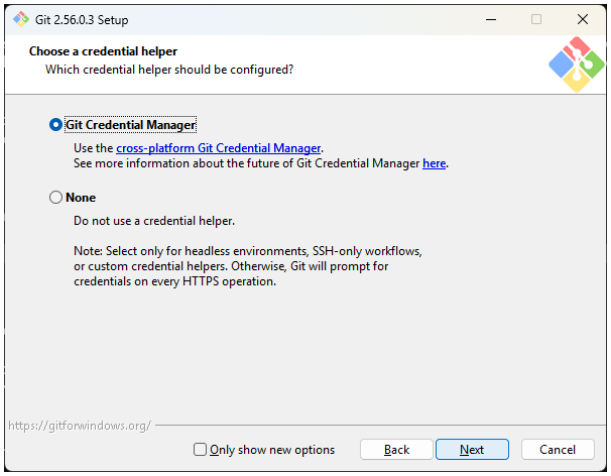


Figura 18: Paso M1, asistente de Git: terminal de Git Bash y comportamiento de `git pull`.



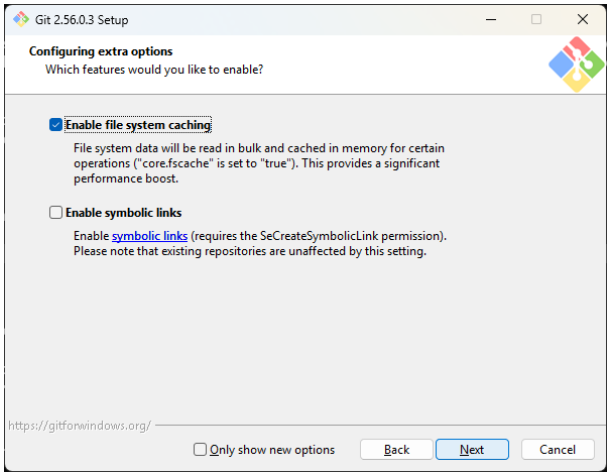


Figura 19: Paso M1, asistente de Git: gestor de credenciales y opciones extra.

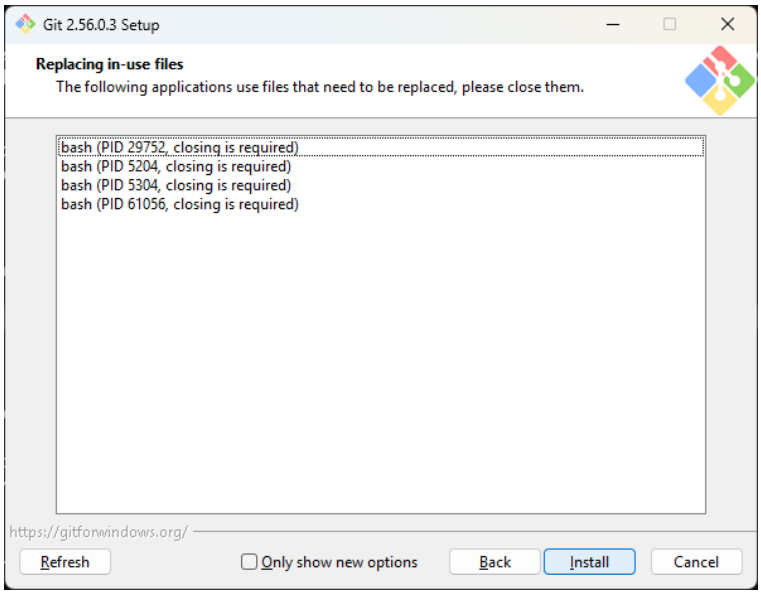


Figura 20: Paso M1, asistente de Git: al reinstalar o actualizar, el asistente pide cerrar las terminales de Git Bash abiertas antes de *Install*. En un equipo limpio esta página no aparece.

3.7.3 Paso M2. Instalar el SDK de .NET 10

1. Descargar el **SDK** (no solo el *runtime*) de .NET 10 para Windows x64 desde `https://dotnet.microsoft.com/download/dotnet/10.0` e instalarlo.
2. En una terminal nueva:

```
dotnet --version
# 10.0.402   (cualquier 10.0.x es valido)
dotnet --list-sdks
```

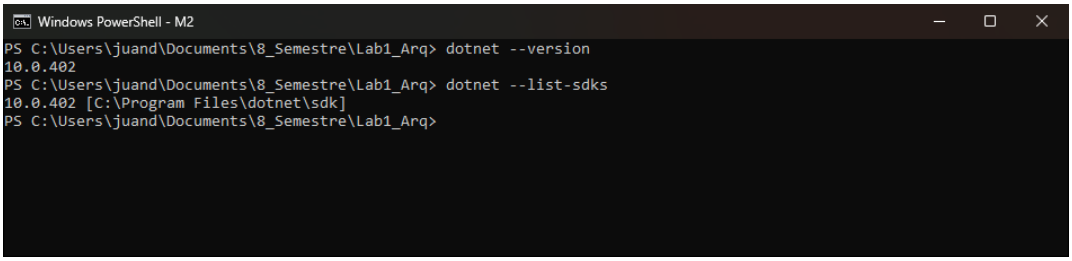


Figura 21: Paso M2: SDK de .NET 10 instalado.

Instalación por terminal (`winget`). El mismo paso se puede hacer desde PowerShell con el gestor de paquetes de Windows, que descarga el instalador oficial, comprueba su hash y lo ejecuta. Si la herramienta ya está instalada, `winget` lo detecta y cancela la reinstalación, como ocurrió en el equipo de verificación:

```
winget search --id Microsoft.DotNet.SDK --source winget
winget install --id Microsoft.DotNet.SDK.10 -e --source winget --accept-
    package-agreements --accept-source-agreements
```

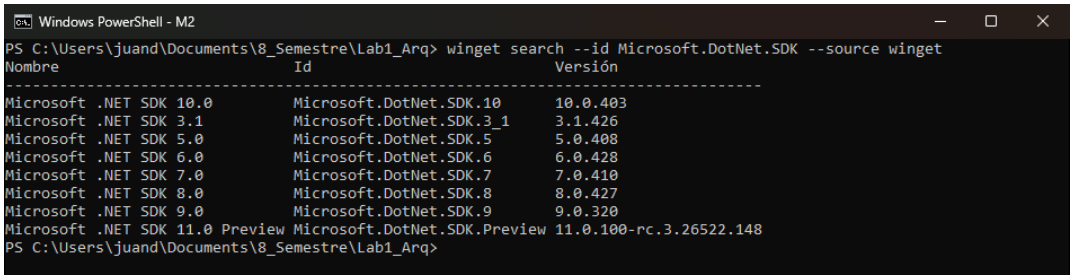


Figura 22: Paso M2: paquetes del SDK de .NET en `winget`; se usa `Microsoft.DotNet.SDK.10`.

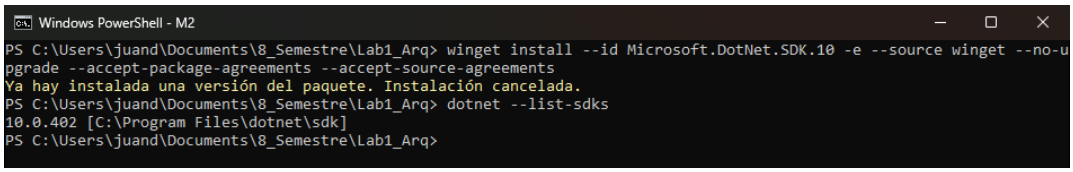


Figura 23: Paso M2: `winget install` del SDK de .NET 10 y `dotnet -list-sdks`.

Asistente gráfico. El instalador del SDK (el mismo que descarga `winget`, versión 10.0.403 en la fecha de verificación) tiene una sola página: la descripción del SDK y los términos que se aceptan al pulsar *Instalar*. La instalación pide permiso de administrador y no tiene opciones. En el equipo de verificación se cerró con *Cerrar* sin instalar, y `dotnet -list-sdks` siguió mostrando solo la 10.0.402.

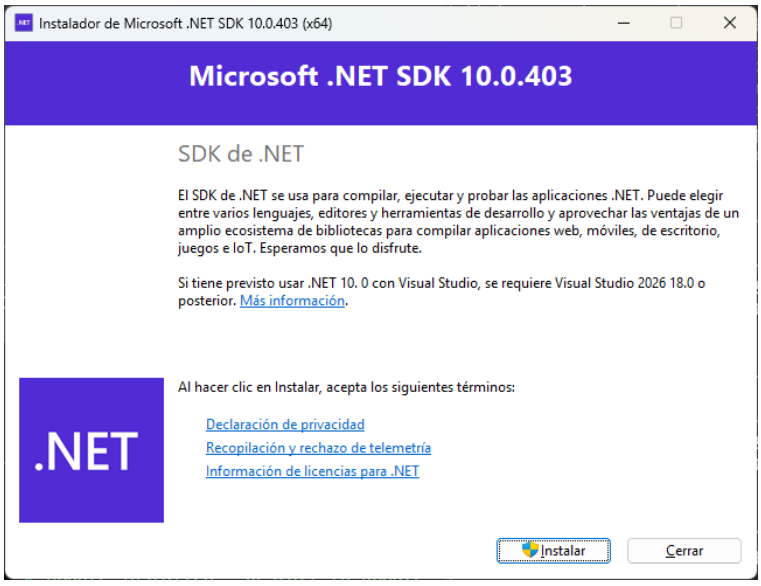


Figura 24: Paso M2: asistente del SDK de .NET 10; se cerró sin instalar.

3.7.4 Paso M3. Instalar SQL Server 2025 Express (camino A)

1. Descargar SQL Server 2025 Express desde `https://www.microsoft.com/sql-server/sql-server-downloads`.
2. Ejecutar el instalador y elegir el tipo de instalación **Básica**. Al terminar, el instalador muestra el nombre de la instancia, `SQLEXPRESS`, y la cadena de conexión `localhost\SQLEXPRESS`.
3. No hace falta crear usuarios ni contraseñas: el camino A usa la autenticación integrada de Windows con el usuario que inició sesión, que el instalador deja como administrador de la instancia.
4. Abrir una terminal nueva y comprobar que el servicio de la instancia está en ejecución y que responde con la versión y la edición esperadas (`sqlcmd` se instala en el paso M4):

```
Get-Service 'MSSQL$SQLEXPRESS' | Format-Table Status, Name, DisplayName
    -AutoSize
# Running  MSSQL$SQLEXPRESS  SQL Server (SQLEXPRESS)
sqlcmd -S 'localhost\SQLEXPRESS' -E -C -W -Q "SELECT SERVERPROPERTY('
    ProductVersion') AS version, SERVERPROPERTY('Edition') AS edicion"
# 17.0.1000.7 Express Edition (64-bit)
```



Figura 25: Paso M3: servicio SQL Server (SQLEXPRESS) en ejecución y versión 17.0.1000.7 Express Edition.

Instalación por terminal (`winget`). El paquete de la edición Express es `Microsoft.SQLServer.2025.Express`. Su instalador pide permisos de administrador (Windows muestra el aviso de Control de cuentas de usuario) y abre el mismo asistente descrito arriba, donde se elige **Básica**:

```
winget search "SQL Server 2025" --source winget
winget install --id Microsoft.SQLServer.2025.Express -e --source winget
    --accept-package-agreements --accept-source-agreements
```

En el equipo de verificación no se ejecutó `winget install` para SQL Server: la instancia `SQLEXPRESS` ya existía y `winget` no la reconoce con ese identificador, así que el comando habría iniciado una segunda instalación.

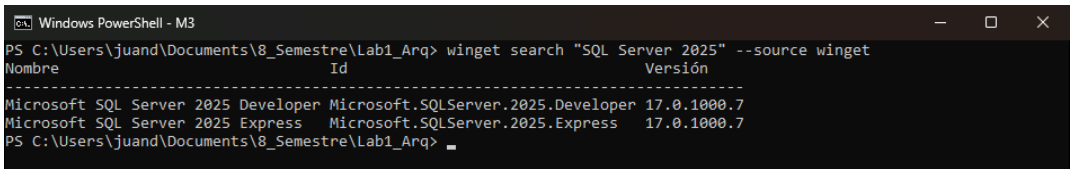


Figura 26: Paso M3: paquetes de SQL Server 2025 en `winget`; el camino A usa la edición Express.

Asistente gráfico. El instalador de SQL Server 2025 Express pide permiso de administrador al abrirse y ofrece tres tipos de instalación. Para este proyecto basta **Basic**, que instala el motor con la configuración por defecto y crea la instancia `SQLEXPRESS`. Si el idioma de Windows y el formato regional no coinciden, como en el equipo de verificación (es-MX y es-ES), avisa que continuará en inglés: se responde *Yes*. Después vienen la licencia (*Accept*) y la carpeta de instalación, con el espacio necesario y el tamaño de la descarga; *Install* descarga unos 730 MB e instala. En el equipo de verificación el recorrido se cerró en esa última página, antes de *Install*, porque la instancia ya existía.

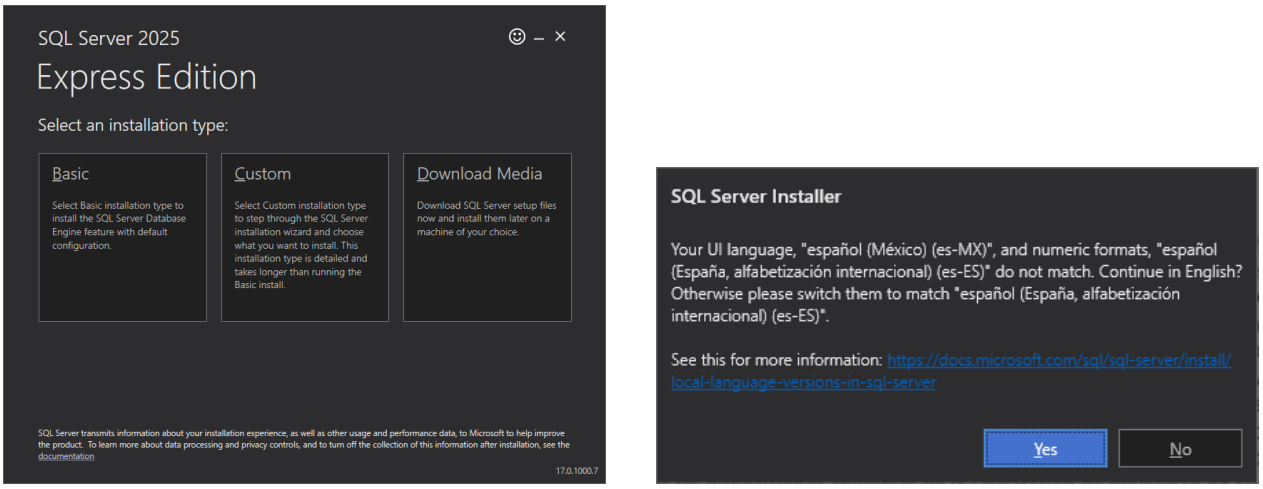


Figura 27: Paso M3, asistente de SQL Server 2025 Express: tipo de instalación y aviso de idioma.

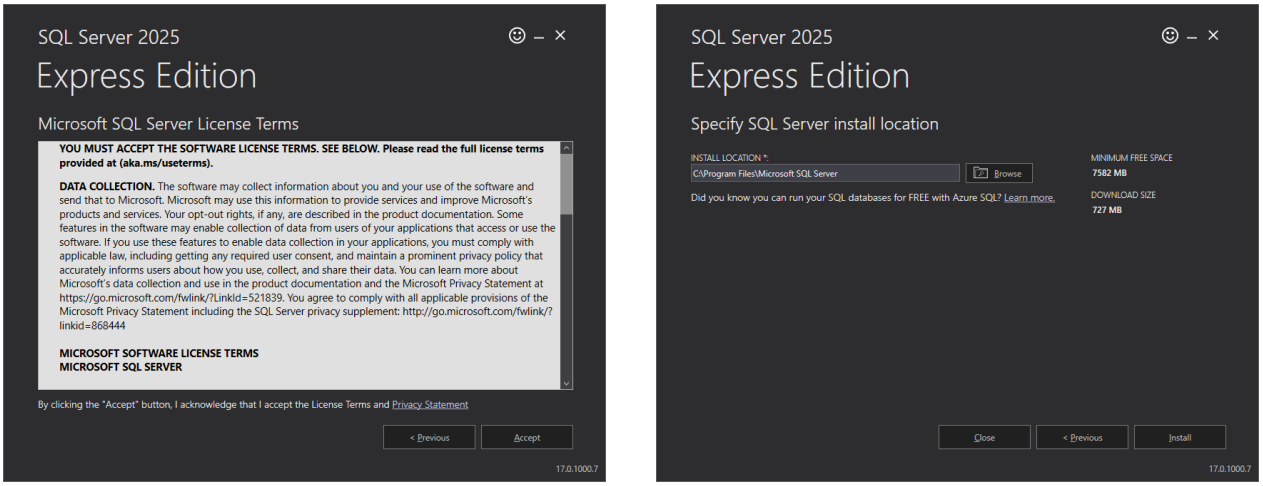


Figura 28: Paso M3, asistente de SQL Server 2025 Express: licencia y carpeta de instalación (se cerró antes de *Install*).

3.7.5 Paso M4. Instalar SSMS 22 y `sqlcmd` (camino A)

1. Instalar SQL Server Management Studio 22 desde `https://learn.microsoft.com/ssms/install/install`.
2. Instalar `sqlcmd` desde `https://learn.microsoft.com/sql/tools/sqlcmd/sqlcmd-utility`, si no quedó instalado con SQL Server.
3. Abrir SSMS. La primera vez ofrece iniciar sesión con una cuenta de Microsoft o de GitHub; no es necesario: se elige **Omitir y agregar cuentas más tarde**.
4. Conectarse con **Nombre del servidor** `localhost\SQLEXPRESS`, **Autenticación** de Windows y la opción **Confiar en el certificado del servidor** marcada. El mismo

resultado se obtiene abriendo SSMS desde la terminal:

```
& "C:\Program Files\Microsoft SQL Server Management Studio 22\Release\
    Common7\IDE\SSMS.exe" -S localhost\SQLEXPRESS -d persona_db -C
```

SSMS pregunta si se desea conectar al servidor indicado con autenticación de Windows y, al aceptar, el Explorador de objetos muestra la instancia con sus carpetas. La conexión también se comprueba con `sqlcmd`, como en el paso M3; si la terminal no reconoce `sqlcmd`, se cierra y se abre una nueva para que cargue el `PATH` actualizado por el instalador.

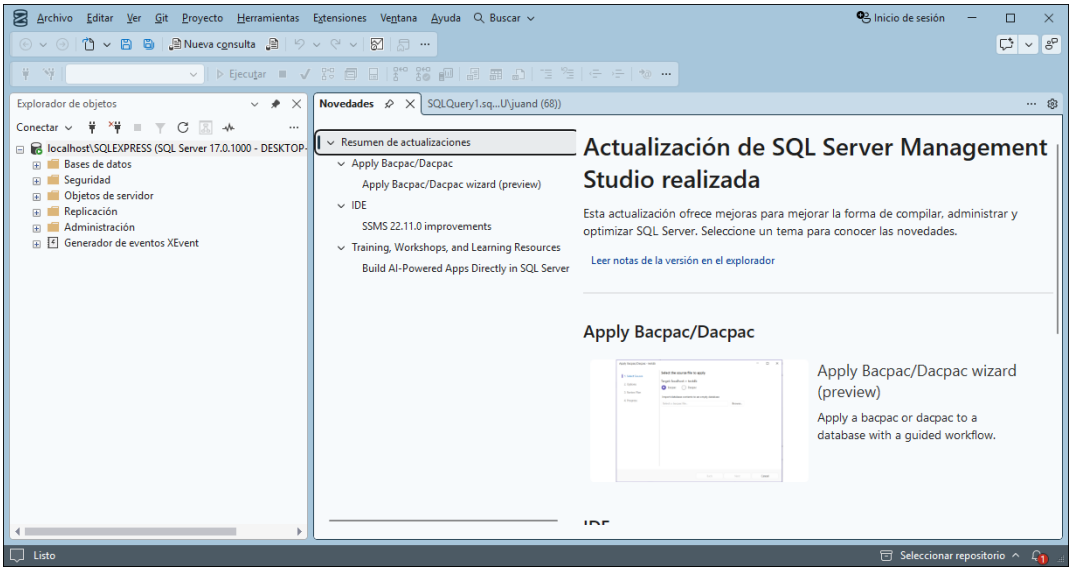


Figura 29: Paso M4: SSMS 22 conectado a `localhost\SQLEXPRESS`; el Explorador de objetos muestra la instancia.

Instalación por terminal (`winget`). El mismo paso se puede hacer desde PowerShell con el gestor de paquetes de Windows, que descarga el instalador oficial, comprueba su hash y lo ejecuta. Si la herramienta ya está instalada, `winget` lo detecta y cancela la reinstalación, como ocurrió en el equipo de verificación:

```
winget install --id Microsoft.SQLServerManagementStudio.22 -e --source
    winget --accept-package-agreements --accept-source-agreements
winget install --id Microsoft.Sqlcmd -e --source winget --accept-package
    -agreements --accept-source-agreements
sqlcmd --version
```

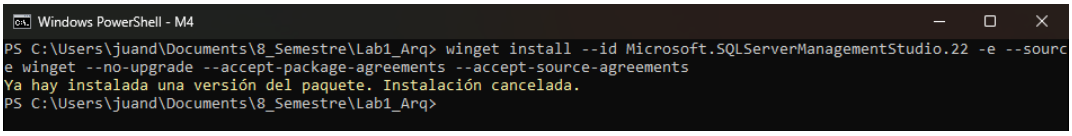


Figura 30: Paso M4: `winget install` de SSMS 22.

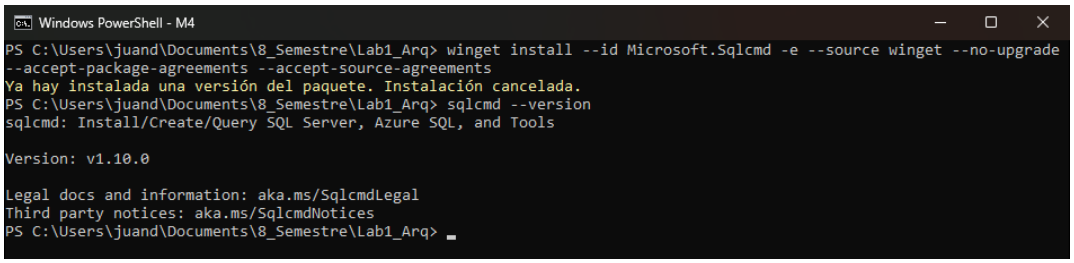


Figura 31: Paso M4: `winget install` de `sqlcmd` y su versión.

SSMS 22 se instala y se actualiza con el Visual Studio Installer. Con el botón **Modificar** se ven sus cargas de trabajo; para este laboratorio bastan los componentes principales, que se instalan siempre.

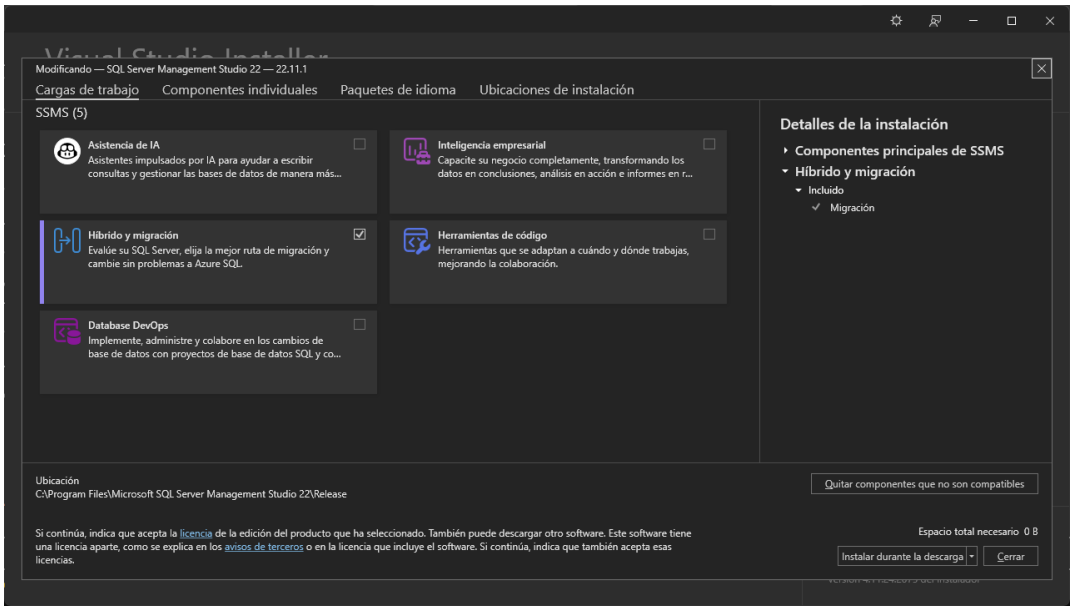


Figura 32: Paso M4: Visual Studio Installer, pantalla Modificar de SQL Server Management Studio 22 (se cerró sin cambios).

Asistentes gráficos. El instalador de SSMS 22 es un arranque del Visual Studio Installer: pide aceptar la licencia con *Continuar*, descarga y prepara el propio instalador y, si SSMS no está instalado, abre la selección de componentes con *Instalar*. En el equipo de verificación, con SSMS 22.11.1 instalado, ofreció la actualización a la 22.11.2, que se cerró sin aceptar; SSMS quedó en la misma versión. `sqlcmd` se instala con un paquete MSI: con la herramienta ya instalada, el asistente ofrece *Change*, *Repair* o *Remove*; se canceló y el propio asistente confirma que el sistema no se modificó.

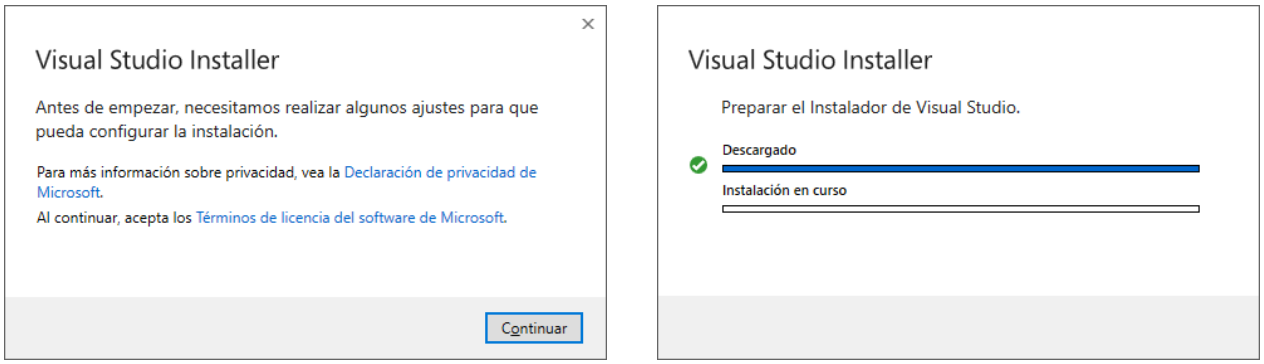


Figura 33: Paso M4, instalador de SSMS 22: licencia y preparación del Visual Studio Installer.

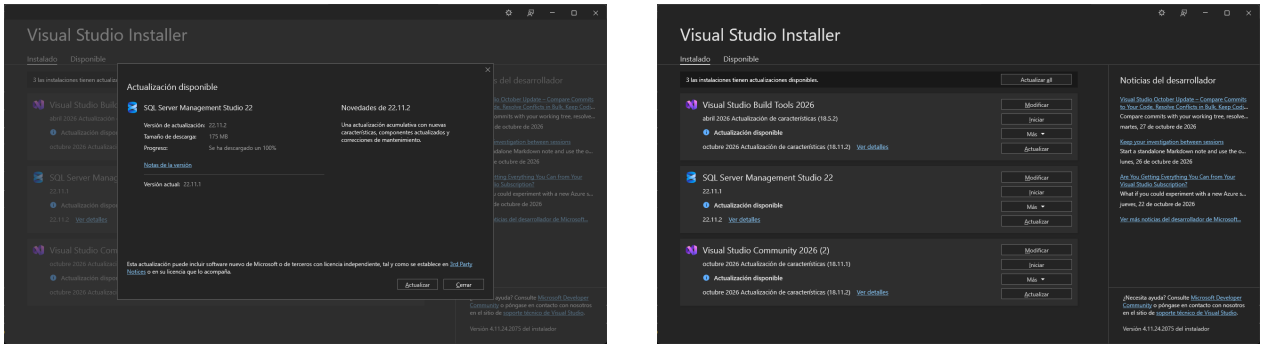


Figura 34: Paso M4, instalador de SSMS 22: aviso de actualización (cerrado sin aceptar) y productos instalados.

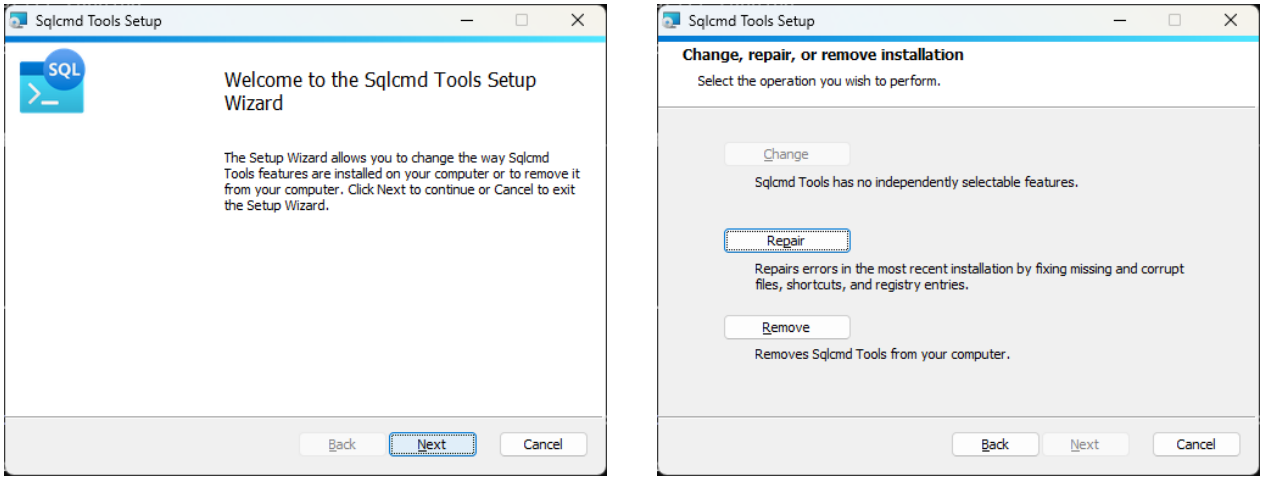


Figura 35: Paso M4, asistente MSI de `sqlcmd`: bienvenida y mantenimiento de una instalación existente.

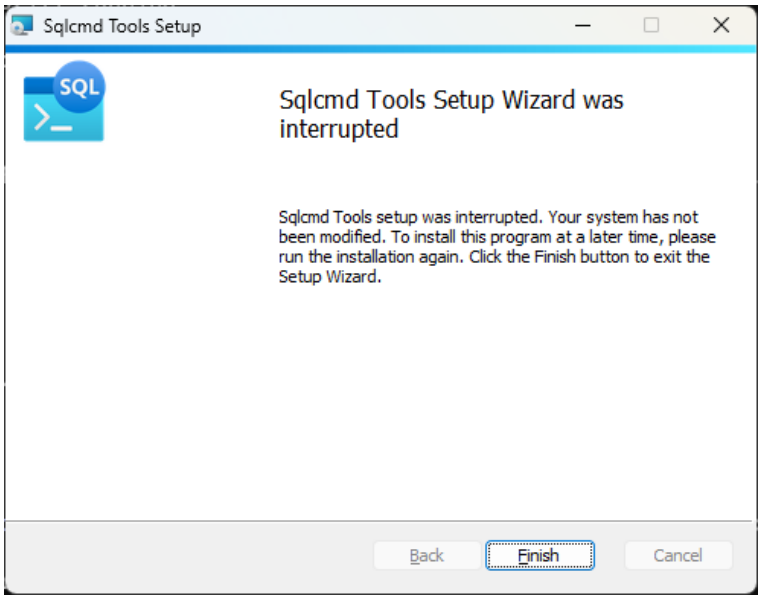


Figura 36: Paso M4, asistente de `sqlcmd` cancelado: el sistema no se modificó.

3.7.6 Paso M5. Instalar Visual Studio 2026 Community (opcional)

Solo es necesario para abrir y depurar la solución. Desde https://visualstudio.microsoft.com/ se instala con las cargas de trabajo **Desarrollo de ASP.NET y web** y **Almacenamiento y procesamiento de datos**. Los pasos de despliegue de este manual se hacen desde la terminal y no lo requieren. La herramienta `vswhere`, que se instala con Visual Studio, confirma qué instalación tiene las dos cargas de trabajo:

```
$vswhere = "${env:ProgramFiles(x86)}\Microsoft Visual Studio\Installer\
    vswhere.exe"
& $vswhere -products * -requires Microsoft.VisualStudio.Workload.NetWeb
    Microsoft.VisualStudio.Workload.Data -property displayName
# Visual Studio Community 2026
& $vswhere -products * -requires Microsoft.VisualStudio.Workload.NetWeb
    Microsoft.VisualStudio.Workload.Data -property
    catalog_productDisplayVersion
# 18.11.1
```

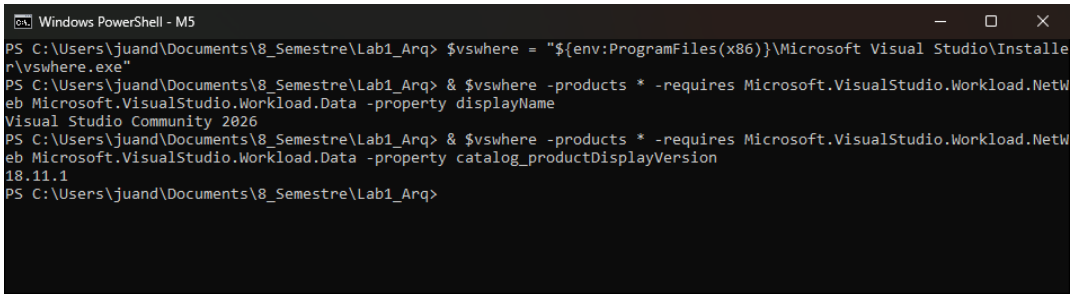


Figura 37: Paso M5: Visual Studio Community 2026 (18.11.1) con las cargas de ASP.NET y web y de datos.

Instalación por terminal (`winget`). El mismo paso se puede hacer desde PowerShell con el gestor de paquetes de Windows, que descarga el instalador oficial, comprueba su hash y lo ejecuta. Si la herramienta ya está instalada, `winget` lo detecta y cancela la reinstalación, como ocurrió en el equipo de verificación:

```
winget install --id Microsoft.VisualStudio.Community -e --source winget
    --accept-package-agreements --accept-source-agreements
```

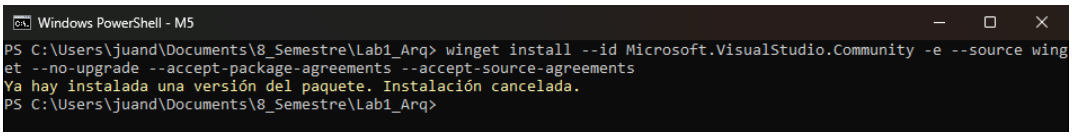


Figura 38: Paso M5: `winget install` de Visual Studio Community.

El Visual Studio Installer muestra los productos instalados. Con **Modificar** en Visual Studio Community 2026 se marcan las dos cargas de trabajo y se confirma con **Modificar**, o con **Instalar** en una instalación nueva:

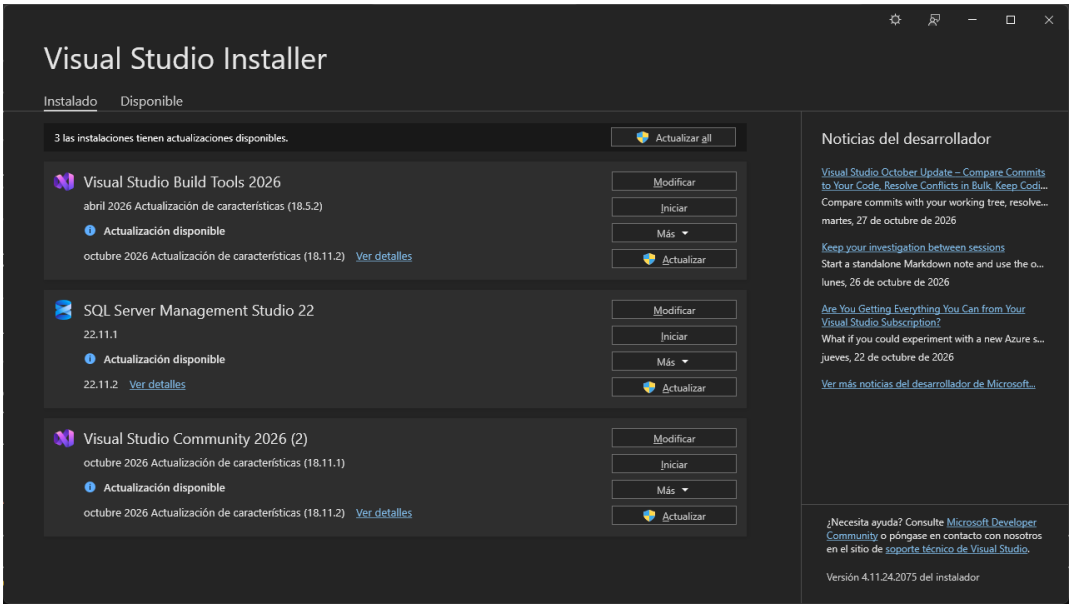


Figura 39: Paso M5: Visual Studio Installer con Build Tools, SSMS 22 y Visual Studio Community 2026.

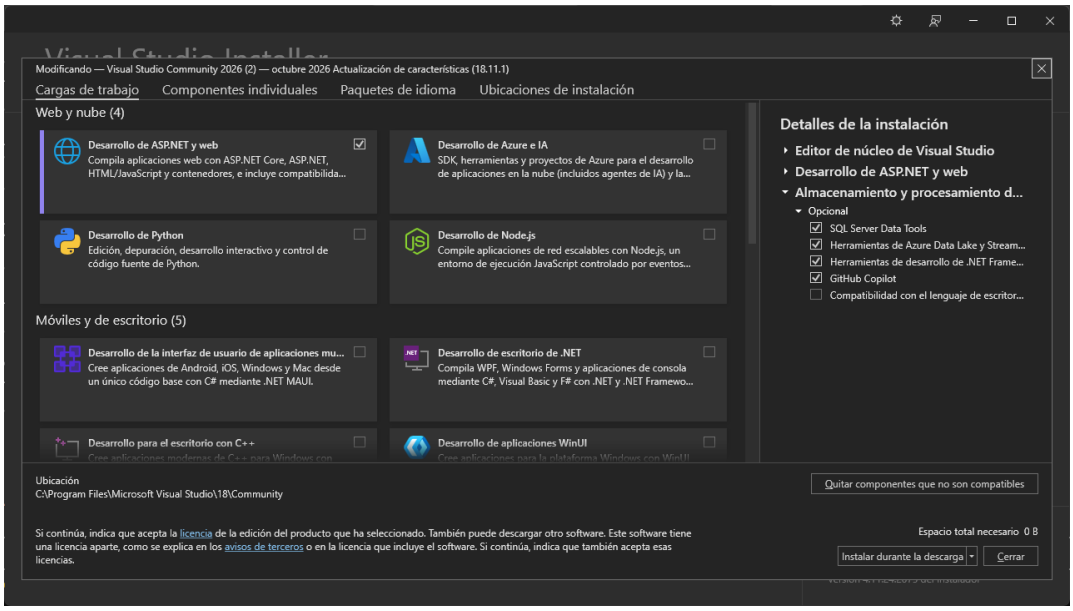


Figura 40: Paso M5: carga Desarrollo de ASP.NET y web marcada y, en el panel derecho, Almacenamiento y procesamiento de datos (se cerró sin cambios).

Asistente gráfico. El instalador de Visual Studio Community 2026 es el mismo arranque del Visual Studio Installer que el de SSMS: *Continuar*, preparación y, en un equipo sin Visual Studio, la selección de cargas de trabajo de la figura anterior. Con Visual Studio ya instalado ofreció la actualización de características de octubre de 2026 (18.11.2), que se cerró sin aceptar.

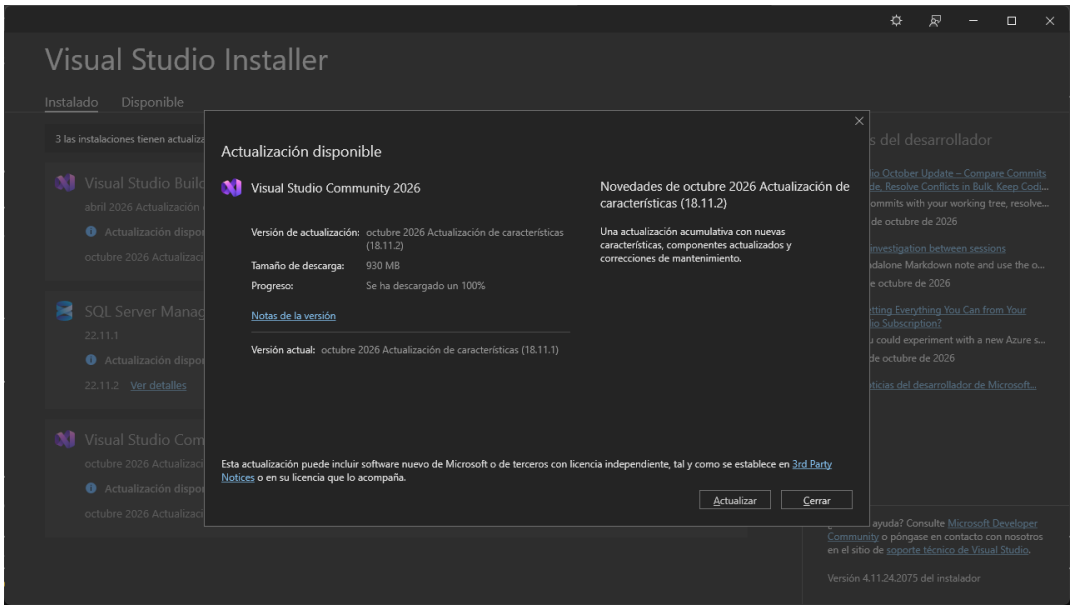


Figura 41: Paso M5, instalador de Visual Studio Community 2026: aviso de actualización (cerrado sin aceptar).

3.7.7 Paso M6. Instalar Docker Desktop (camino B)

1. Descargar Docker Desktop para Windows desde https://www.docker.com/products/docker-desktop/ e instalarlo con el motor **WSL 2** (opción por defecto). Si Windows lo pide, reiniciar.
2. Abrir Docker Desktop y esperar a que indique que el motor está en ejecución (*Engine running*).
3. Comprobar desde la terminal:

```
docker --version
# Docker version 29.8.1, build 4a63305
docker compose version
# Docker Compose version v5.5.1
docker info --format 'Motor: {{.ServerVersion}} ({{.OSType}}, {{.
    OperatingSystem}})'
# Motor: 29.8.1 (linux, Docker Desktop)
```

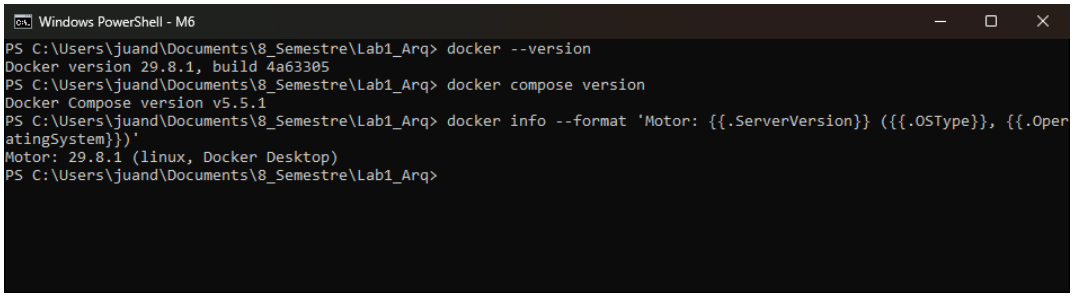


Figura 42: Paso M6: Docker Desktop instalado, con el motor Linux en ejecución.

Instalación por terminal (`winget`). El mismo paso se puede hacer desde PowerShell con el gestor de paquetes de Windows, que descarga el instalador oficial, comprueba su hash y lo ejecuta. Si la herramienta ya está instalada, `winget` lo detecta y cancela la reinstalación, como ocurrió en el equipo de verificación:

```
winget install --id Docker.DockerDesktop -e --source winget --accept-
    package-agreements --accept-source-agreements
```

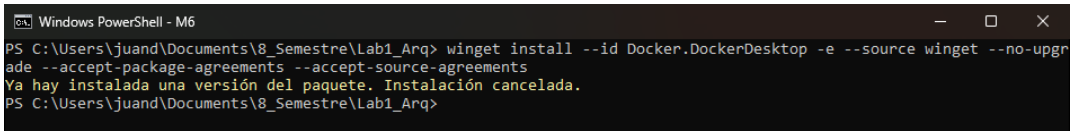


Figura 43: Paso M6: `winget install` de Docker Desktop.

Asistente gráfico. El instalador de Docker Desktop (unos 620 MB) pide permiso de administrador y, en un equipo limpio, muestra la página de configuración con la opción *Use*

WSL 2 instead of Hyper-V, que debe quedar marcada; *OK* instala y al terminar puede pedir cerrar la sesión de Windows. En el equipo de verificación se lanzó la versión 4.97.0 sobre la 4.93.0 instalada y se canceló: el instalador lo confirma y Docker Engine siguió en la 29.8.1, con los contenedores del proyecto en ejecución.

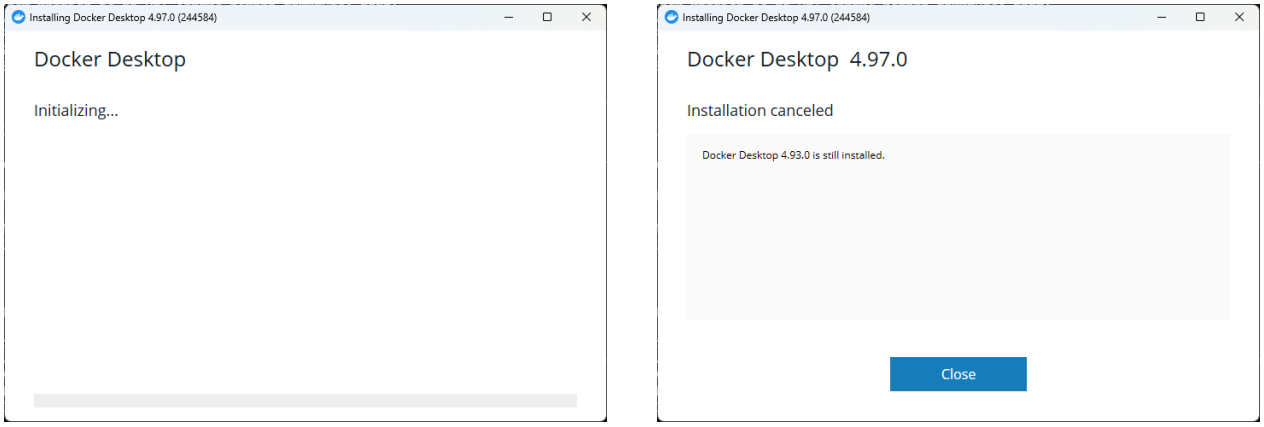


Figura 44: Paso M6, instalador de Docker Desktop: inicio y cancelación sin cambios.

3.7.8 Paso M7. Clonar el repositorio

```
git clone <URL-del-repositorio>.git
cd <carpeta-del-clon>
git checkout v1.0.0      # version entregada
```

La URL del repositorio público es https://github.com/USUARIO/personapi-dotnet, la misma que se indica al final del procedimiento. La raíz del clon contiene `personapi-dotnet.sln`, `Dockerfile`, `docker-compose.yml`, `.env.example` y las carpetas `scripts/`, `personapi-dotnet/` y `tests/`. Todos los comandos siguientes se ejecutan desde esa raíz.

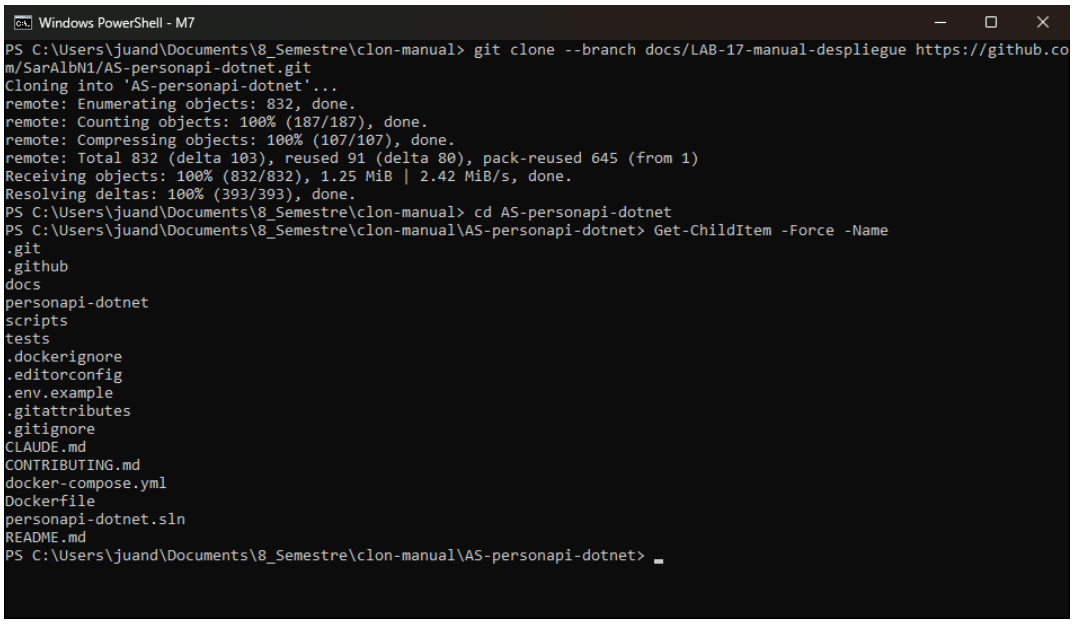


Figura 45: Paso M7: repositorio clonado y contenido de su raíz. La captura se tomó antes de publicar la etiqueta, clonando la rama de esta historia.

Para confirmar la rama y conocer la estructura del clon:

```
git branch --show-current
git log --oneline -5
git remote -v
Get-ChildItem -Name
Get-ChildItem scripts -Name
Get-ChildItem personapi-dotnet -Name -Exclude bin,obj
```

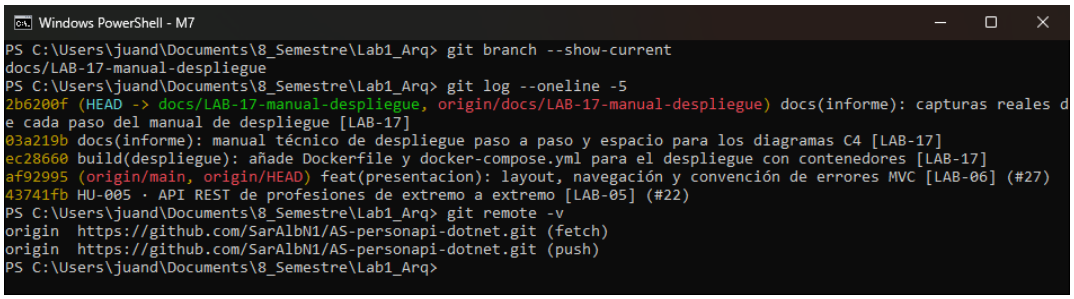


Figura 46: Paso M7: rama, últimos commits y remoto del clon.

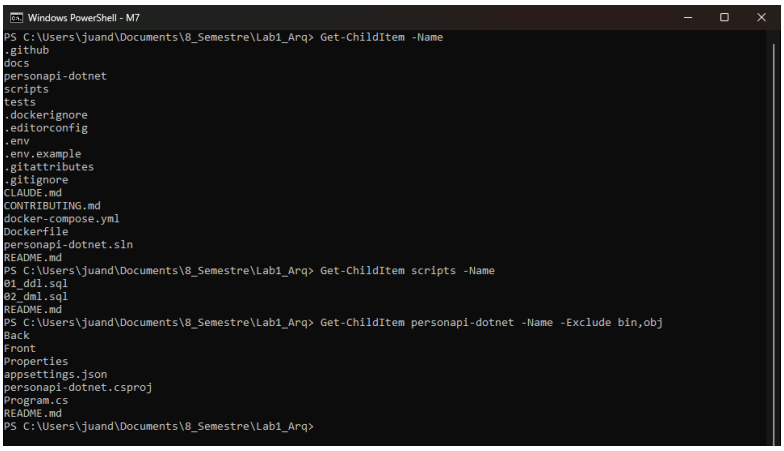


Figura 47: Paso M7: contenido de la raíz, de `scripts/` y del proyecto `personapi-dotnet/`.

3.7.9 Camino A · Paso M8. Crear `persona_db`

Se ejecutan los dos scripts en orden: primero el DDL, que crea la base, las tablas, las llaves y las restricciones, y después el DML, con tres filas de demostración por tabla. También pueden abrirse y ejecutarse en SSMS, en el mismo orden.

```
sqlcmd -S "localhost\SQLEXPRESS" -E -C -b -i scripts/01_ddl.sql
sqlcmd -S "localhost\SQLEXPRESS" -E -C -b -i scripts/02_dml.sql
sqlcmd -S "localhost\SQLEXPRESS" -E -C -d persona_db -Q "SELECT COUNT(*)
    FROM persona; SELECT COUNT(*) FROM profesion; SELECT COUNT(*) FROM
   telefono; SELECT COUNT(*) FROM estudios"
# 3, 3, 3 y 3 filas
```

Reejecución. Los dos scripts son idempotentes: si la base y las filas ya existen no las duplican ni las modifican, y no muestran error. Para volver al estado inicial exacto se elimina la base (`DROP DATABASE persona_db;`) y se ejecutan de nuevo los dos scripts.

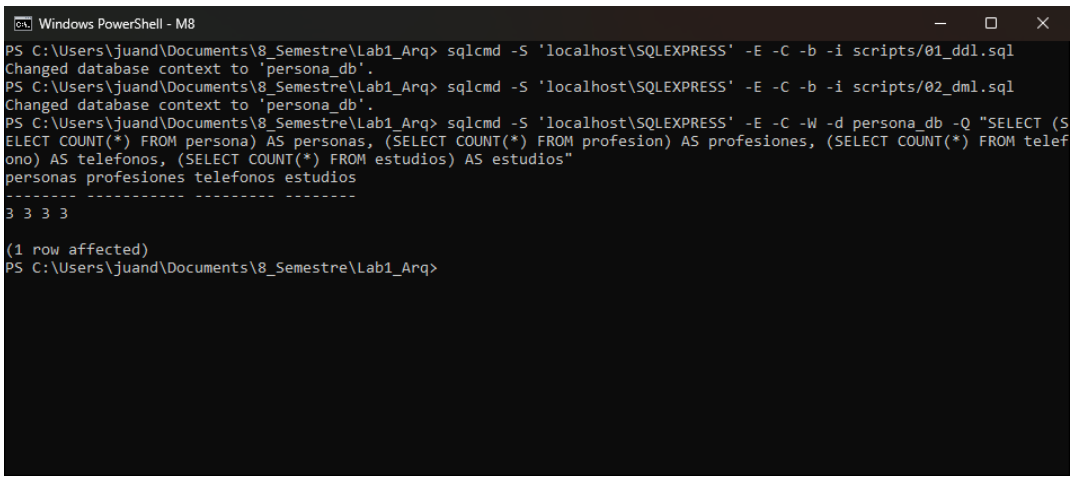


Figura 48: Paso M8: creación y carga de `persona_db`.

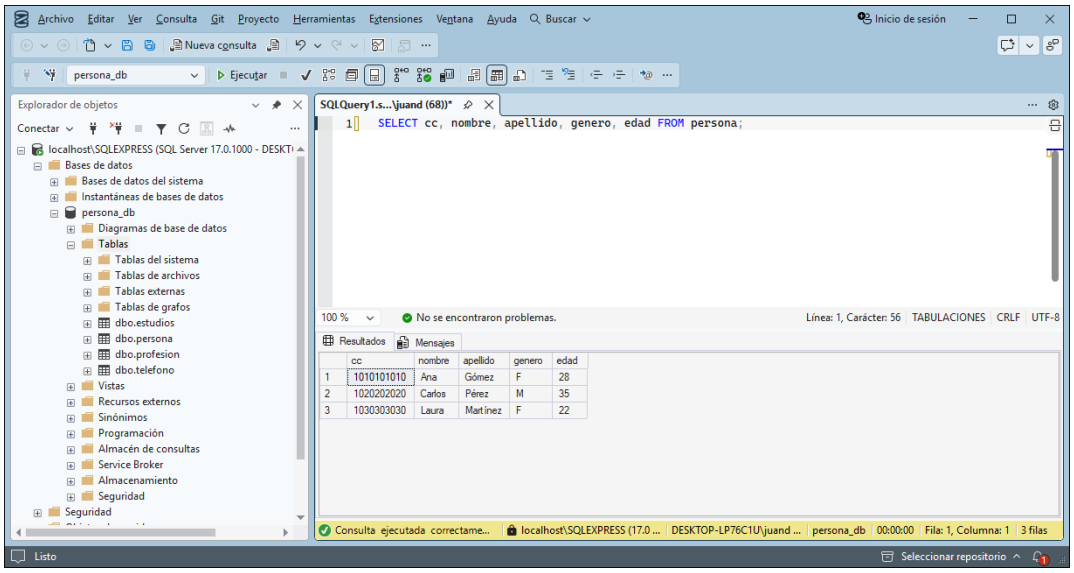


Figura 49: Paso M8: las cuatro tablas de `persona_db` en el Explorador de objetos de SSMS y las tres personas cargadas por el DML.

El contenido de cada tabla se revisa con una consulta por tabla, en SSMS o con `sqlcmd`:

```
sqlcmd -S localhost\SQLEXPRESS -E -C -d persona_db -Q "SELECT * FROM
    persona"
sqlcmd -S localhost\SQLEXPRESS -E -C -d persona_db -Q "SELECT * FROM
    profesion"
sqlcmd -S localhost\SQLEXPRESS -E -C -d persona_db -Q "SELECT * FROM
    telefono"
sqlcmd -S localhost\SQLEXPRESS -E -C -d persona_db -Q "SELECT * FROM
    estudios"
```

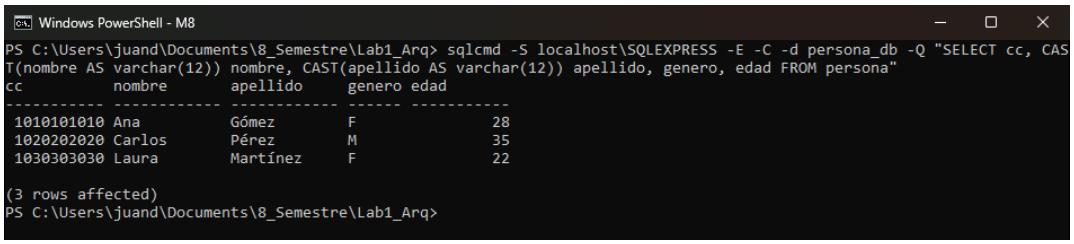


Figura 50: Paso M8: tabla `persona` con las tres personas de demostración.

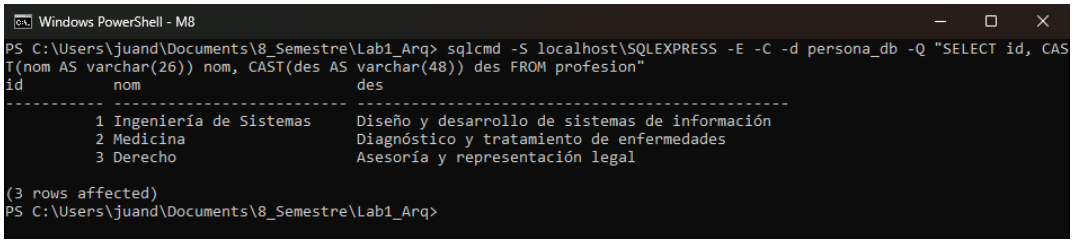


Figura 51: Paso M8: tabla `profesion`.

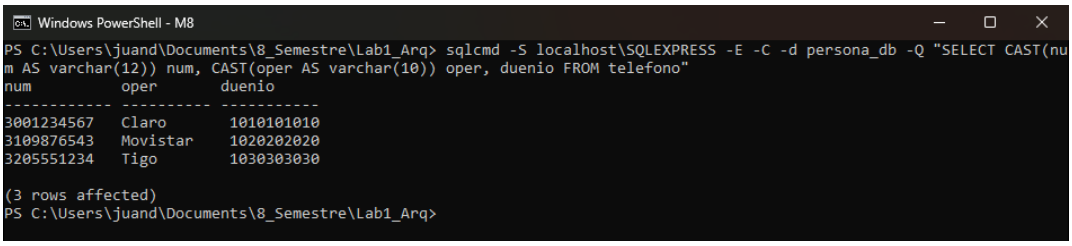


Figura 52: Paso M8: tabla `telefono`; `duenio` referencia a `persona.cc`.

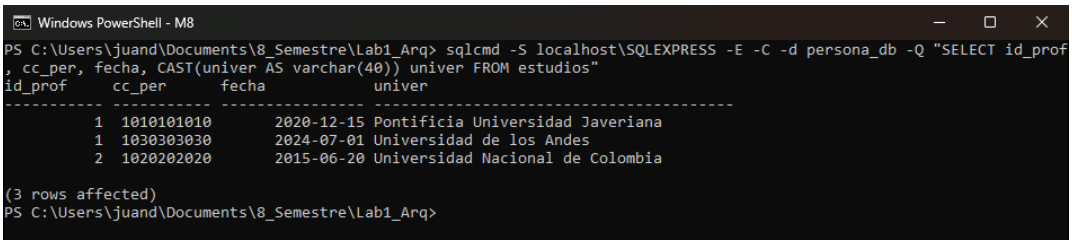


Figura 53: Paso M8: tabla `estudios`, con llave primaria compuesta (`id_prof`, `cc_per`).

El DML se puede ejecutar de nuevo sin duplicar filas:

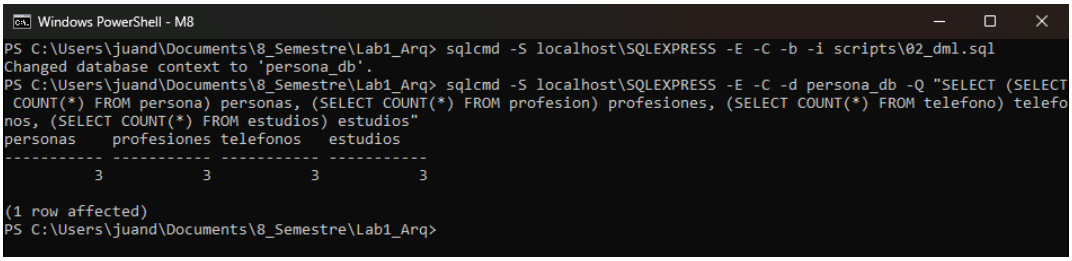


Figura 54: Paso M8: segunda ejecución de `02_dml.sql`; los conteos siguen en 3 por tabla.

3.7.10 Camino A · Paso M9. Configurar la cadena de conexión

La aplicación lee la cadena de `ConnectionStrings:PersonaDb`. En `appsettings.json` está vacía a propósito, para no versionar datos de conexión; el valor local se guarda con *User Secrets*, fuera del repositorio:

```
dotnet user-secrets set "ConnectionStrings:PersonaDb" "Server=localhost\
    SQLEXPRESS;Database=persona_db;Trusted_Connection=True;
    TrustServerCertificate=True" --project personapi-dotnet/personapi-
    dotnet.csproj
dotnet user-secrets list --project personapi-dotnet/personapi-dotnet.
    csproj
```

Esta cadena no contiene contraseña porque usa la autenticación integrada de Windows. Como alternativa equivalente, sin *User Secrets*, se define la variable de entorno en la terminal antes de ejecutar:

```
$env:ConnectionStrings__PersonaDb = "Server=localhost\SQLEXPRESS;
    Database=persona_db;Trusted_Connection=True;TrustServerCertificate=
    True"
```

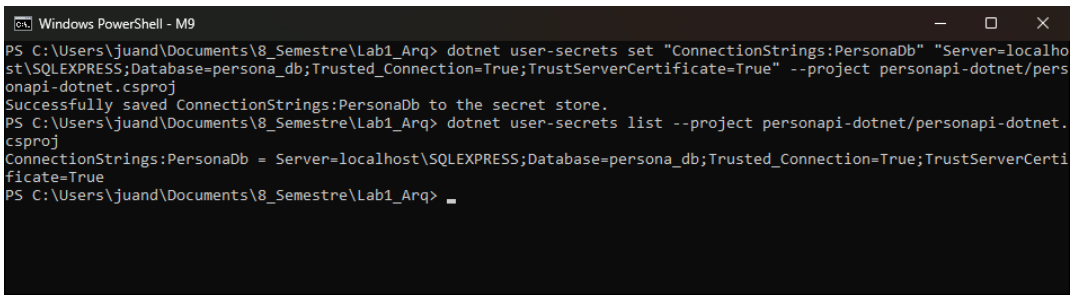


Figura 55: Paso M9: cadena de conexión configurada con User Secrets.

3.7.11 Camino A · Paso M10. Restaurar, compilar y probar

```
dotnet restore
dotnet build
# Compilación realizado correctamente
dotnet test
# Correctas! - Con error: 0, Superado: 56, Omitido: 0, Total: 56
```

Las pruebas de integración usan la misma `persona_db`: con la cadena definida se ejecutan contra la base y dejan los datos como los deja `02_dml.sql`; sin ella se omiten.

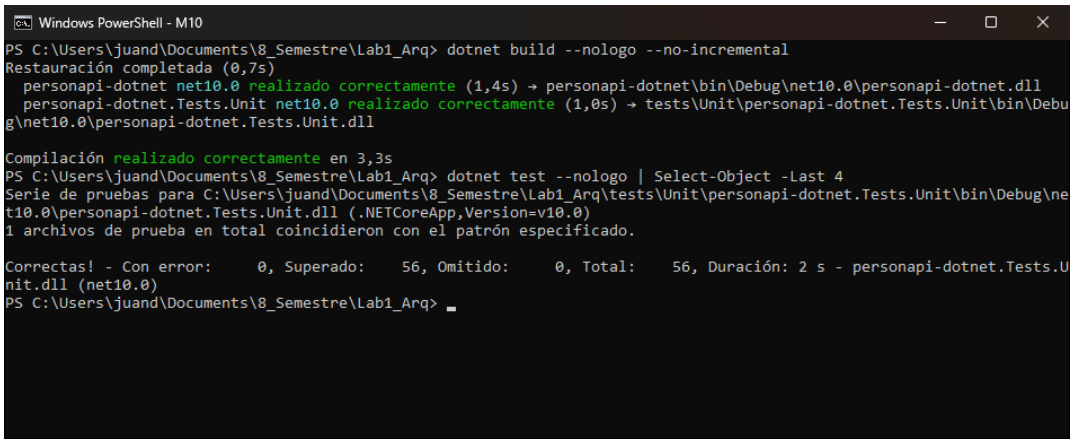


Figura 56: Paso M10: compilación y pruebas en verde.

Los tres comandos también se pueden ejecutar por separado, para ubicar mejor un error:

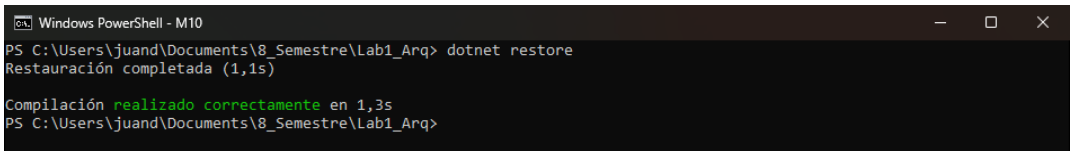


Figura 57: Paso M10: `dotnet restore`.

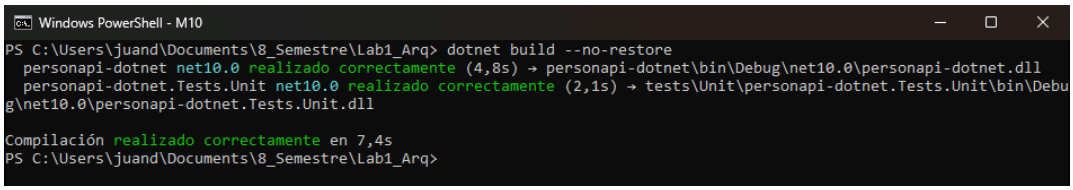


Figura 58: Paso M10: `dotnet build -no-restore` compila la aplicación y el proyecto de pruebas.

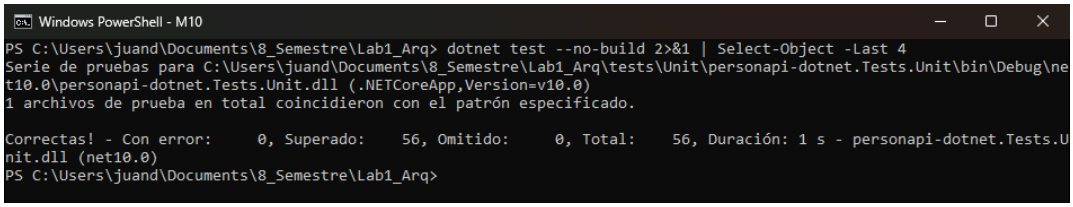


Figura 59: Paso M10: `dotnet test -no-build`: 56 pruebas superadas, ninguna con error.

3.7.12 Camino A · Paso M11. Ejecutar la aplicación

```
dotnet run --project personapi-dotnet/personapi-dotnet.csproj
# Now listening on: http://localhost:5093
# Hosting environment: Development
```

Con la aplicación en marcha:

- Vistas web: `http://localhost:5093`
- Swagger UI: `http://localhost:5093/swagger`

Se detiene con `Ctrl+C` en la terminal.

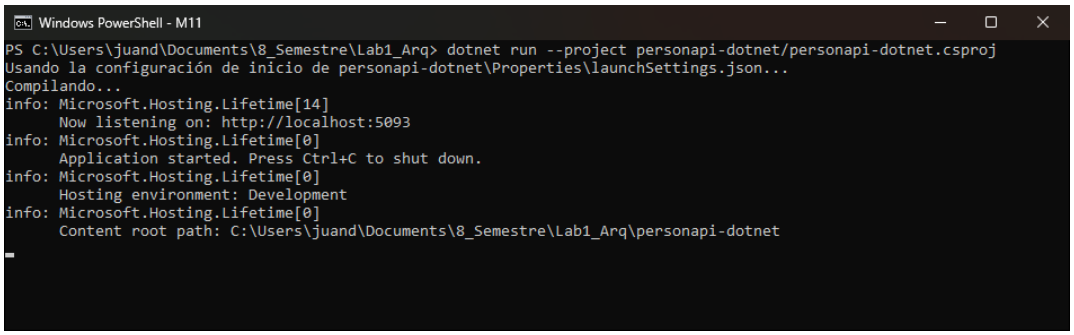


Figura 60: Paso M11: aplicación en ejecución local.

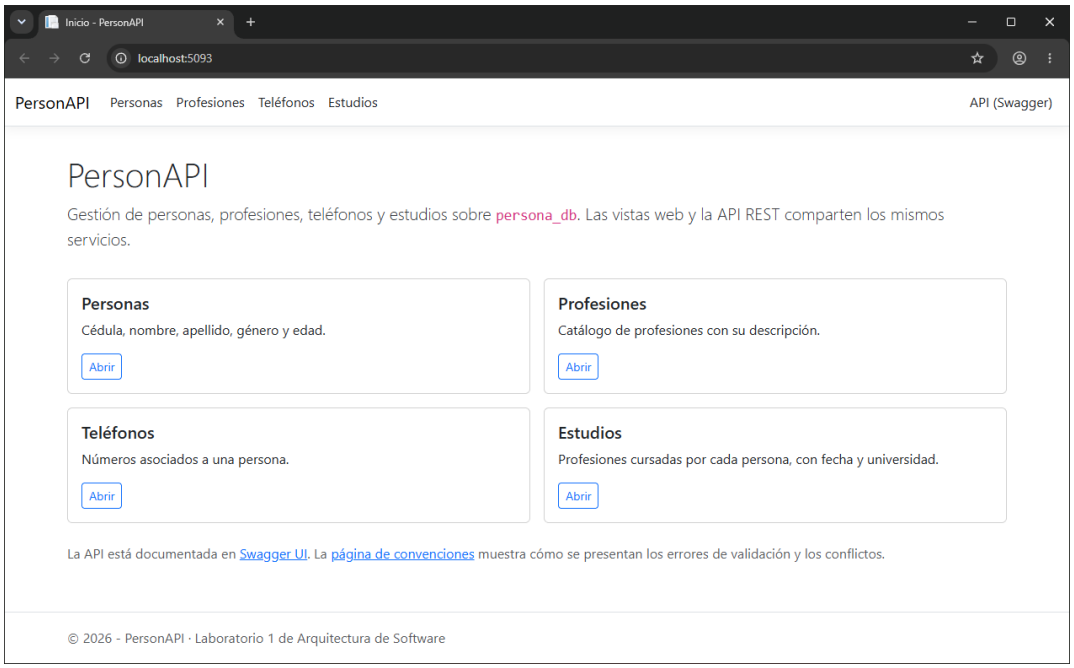


Figura 61: Paso M11: página de inicio servida localmente.

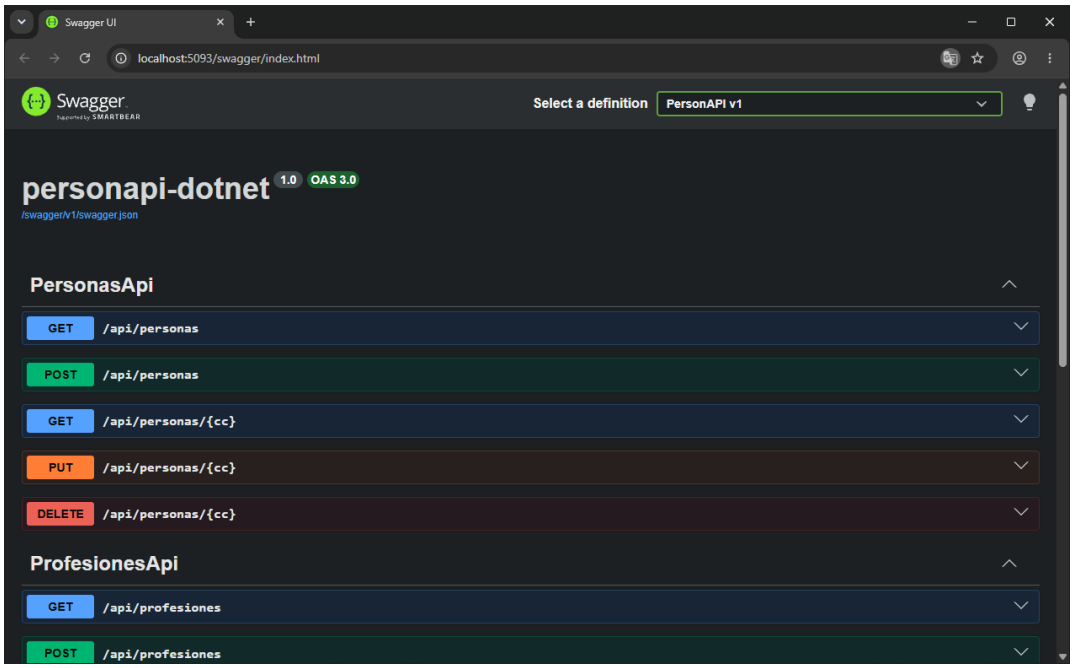


Figura 62: Paso M11: Swagger UI en ejecución local.

3.7.13 Camino B · Paso M12. Crear el archivo `.env`

Docker Compose lee la contraseña del usuario `sa` del contenedor de SQL Server de un archivo `.env` en la raíz. Se crea a partir de la plantilla y se edita para dar valor a `SA_PASSWORD`:

```
Copy-Item .env.example .env
notepad .env            # escribir SA_PASSWORD=<contrasena-local> y
    guardar
git status --short      # .env no aparece: Git lo ignora
```

La contraseña debe tener al menos 8 caracteres e incluir tres de estos cuatro grupos: mayúsculas, minúsculas, dígitos y símbolos; si no, SQL Server no arranca. Es una contraseña local de laboratorio y **no se captura ni se escribe en ningún documento**.

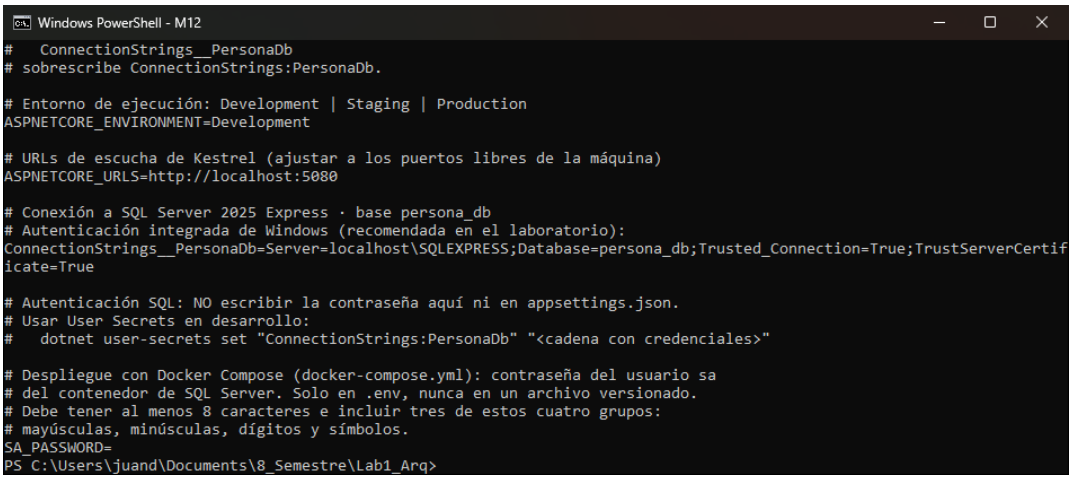


Figura 63: Paso M12: plantilla `.env.example`; `SA_PASSWORD` va vacía y se completa solo en `.env`, que no se captura.

3.7.14 Camino B · Paso M13. Construir y levantar los contenedores

```
docker compose up --build -d
```

El comando construye la imagen de la aplicación con el `Dockerfile` y arranca tres servicios en la red `personapi-net`:

`sqlserver` SQL Server 2025 Express. Publica el puerto 1433 y guarda los datos en el volumen `sqldata`. Compose espera a que su comprobación de salud (`SELECT 1`) responda.

`db-init` Ejecuta `scripts/01_ddl.sql` y `scripts/02_dml.sql` contra `sqlserver` y termina. Como los scripts son idempotentes, repetir `docker compose up` no duplica datos.

`personapi` La aplicación. Arranca solo cuando `db-init` terminó bien y recibe la cadena de conexión por la variable `ConnectionStrings__PersonaDb`, con el usuario `sa` y la contraseña de `.env`.

Salida esperada al final:

```
[+] up 6/6
 Image personapi-personapi        Built
 Network personapi_personapi-net  Created
 Volume personapi_sqldata         Created
 Container personapi-sqlserver-1  Healthy
 Container personapi-db-init-1    Exited
 Container personapi-personapi-1  Started
```

La primera vez, antes de estas líneas, Docker descarga las imágenes base y compila la aplicación dentro de la imagen `sdk:10.0`; eso tarda varios minutos. En las siguientes ejecuciones los pasos salen como `CACHED` y el arranque completo toma unos segundos.

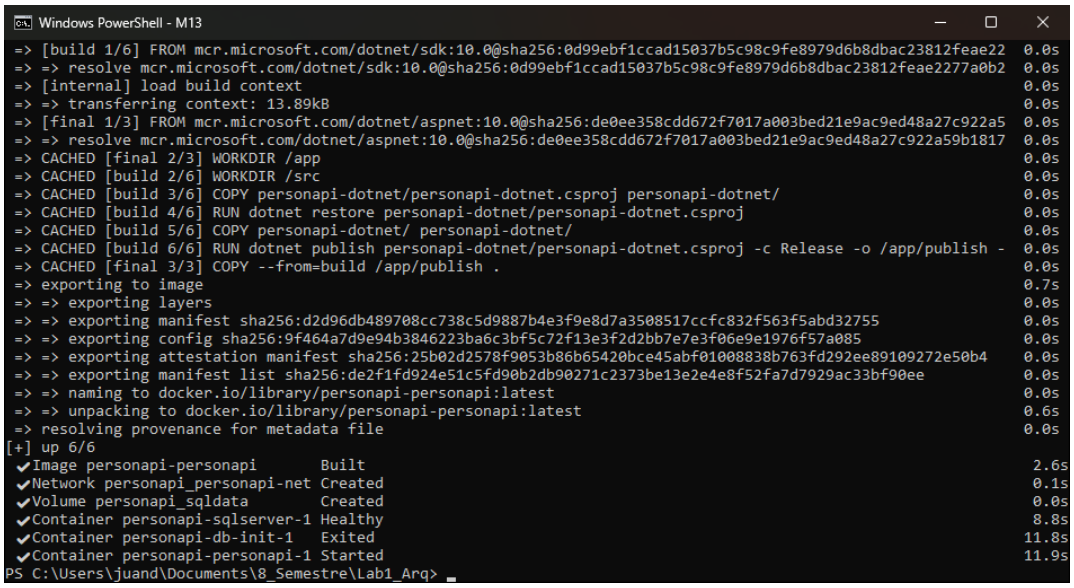


Figura 64: Paso M13: contenedores construidos y levantados.

Los dos archivos que usa este paso están en la raíz del repositorio:

```dockerfile
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY personapi-dotnet/personapi-dotnet.csproj personapi-dotnet/
RUN dotnet restore personapi-dotnet/personapi-dotnet.csproj
COPY personapi-dotnet/ personapi-dotnet/
RUN dotnet publish personapi-dotnet/personapi-dotnet.csproj -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
# La imagen aspnet:10.0 ya escucha en 8080 (ASPNETCORE_HTTP_PORTS); se declara el puerto.
EXPOSE 8080
ENTRYPOINT ["dotnet", "personapi-dotnet.dll"]
```

Listing 9: `Dockerfile`.

```yaml
name: personapi

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2025-latest
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: "${SA_PASSWORD:?Defina SA_PASSWORD en .env}"
      MSSQL_PID: "Express"
    ports:
      - "1433:1433"
    volumes:
      - sqldata:/var/opt/mssql
```

```
    healthcheck:
      test: ["CMD-SHELL", "/opt/mssql-tools18/bin/sqlcmd -S localhost -U
          sa -P \"$$MSSQL_SA_PASSWORD\" -C -Q 'SELECT 1' || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 20s
    networks:
      - personapi-net

  db-init:
    image: mcr.microsoft.com/mssql/server:2025-latest
    depends_on:
      sqlserver:
        condition: service_healthy
    volumes:
      - ./scripts:/scripts:ro
    environment:
      SA_PASSWORD: "${SA_PASSWORD:?Defina SA_PASSWORD en .env}"
    entrypoint:
      - bash
      - -c
      - |
        for f in /scripts/01_ddl.sql /scripts/02_dml.sql; do
          echo "Ejecutando $$f"
          /opt/mssql-tools18/bin/sqlcmd -S sqlserver -U sa -P "
              $$SA_PASSWORD" -C -b -i "$$f" || exit 1
        done
    networks:
      - personapi-net

  personapi:
    build: .
    depends_on:
      db-init:
        condition: service_completed_successfully
    environment:
      ConnectionStrings__PersonaDb: "Server=sqlserver,1433;Database=
          persona_db;User Id=sa;Password=${SA_PASSWORD:?Defina
          SA_PASSWORD en .env};TrustServerCertificate=True"
    ports:
      - "8080:8080"
    networks:
      - personapi-net

volumes:
  sqldata:

networks:
  personapi-net:
```

Listing 10: `docker-compose.yml`.

Dentro de un contenedor Linux no existe la autenticación integrada de Windows, por eso

este camino usa el usuario `sa`. La contraseña nunca está en el repositorio: Compose la toma de `.env` y la sustituye en `${SA_PASSWORD}` al arrancar.

3.7.15 Camino B · Paso M14. Comprobar los contenedores

```
docker compose ps -a
# personapi    running    0.0.0.0:8080->8080/tcp
# sqlserver    running (healthy)    0.0.0.0:1433->1433/tcp
# db-init      exited (0)
docker compose logs db-init
# Ejecutando /scripts/01_ddl.sql
# Ejecutando /scripts/02_dml.sql
docker compose logs personapi
# Now listening on: http://[::]:8080
```

`db-init` debe terminar con código `0`: es un servicio de un solo uso, no un fallo.

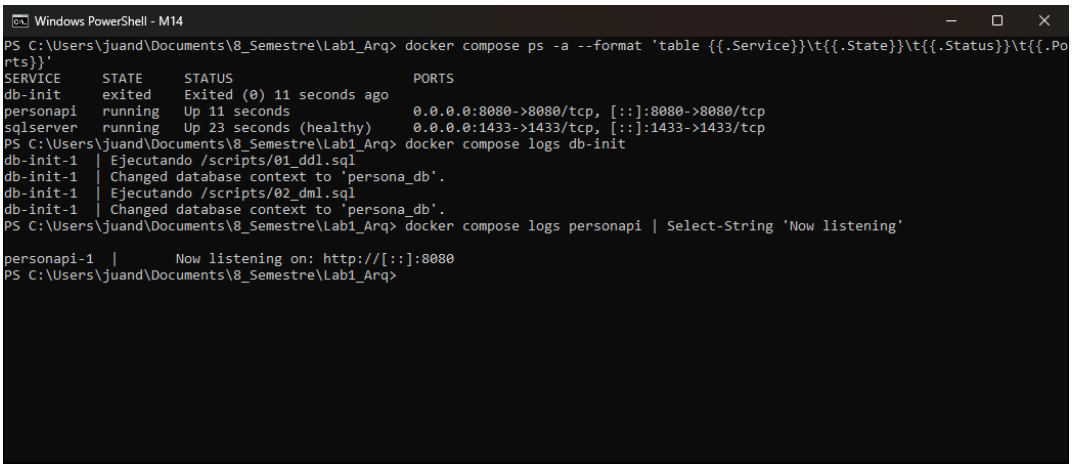


Figura 65: Paso M14: estado de los tres servicios.

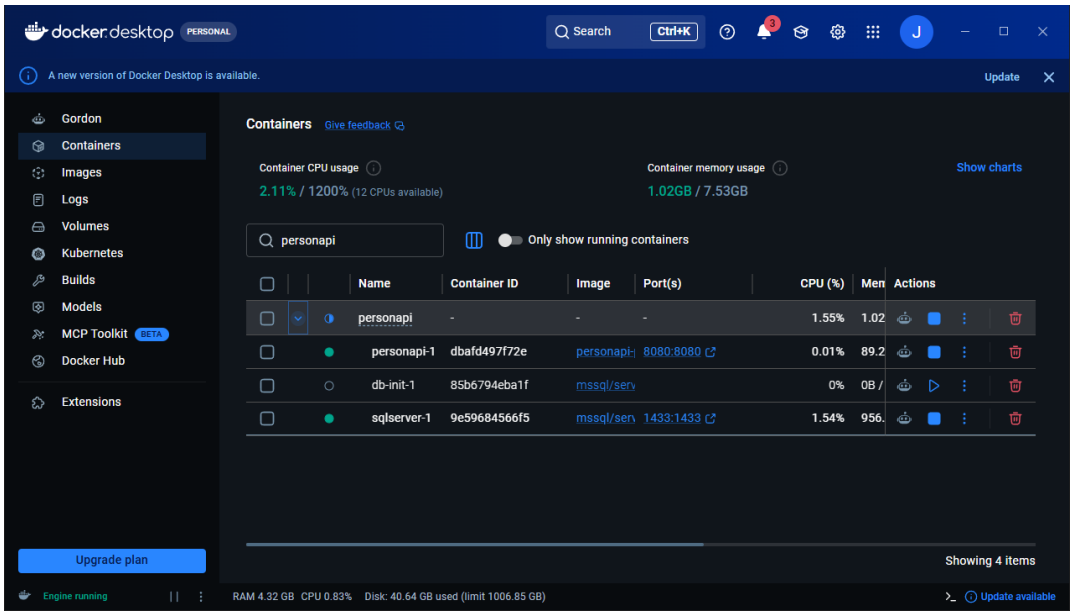


Figura 66: Paso M14: contenedores en Docker Desktop.

Los registros y los recursos creados se revisan desde la terminal:

```
docker compose logs db-init
docker compose logs personapi --tail 15
docker image ls personapi-personapi
docker network ls --filter name=personapi
docker volume ls --filter name=personapi
```

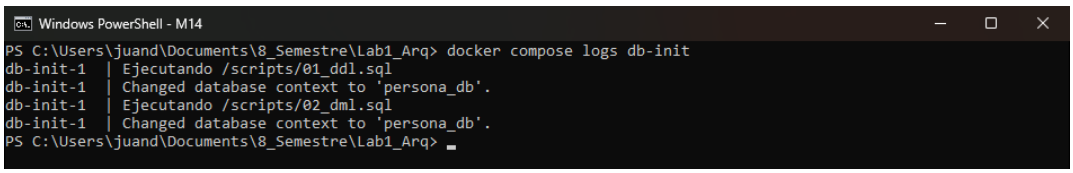


Figura 67: Paso M14: `db-init` ejecutó `01_ddl.sql` y luego `02_dml.sql`.

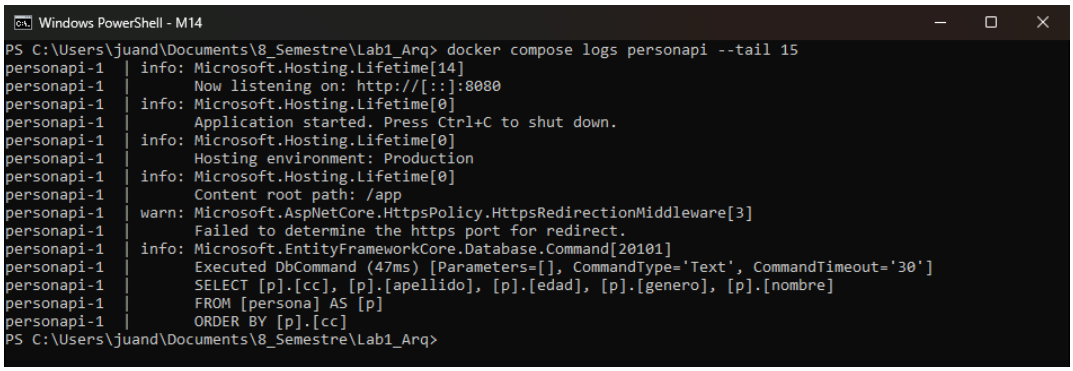


Figura 68: Paso M14: `personapi` escucha en el puerto 8080 del contenedor y consulta `persona_db`.

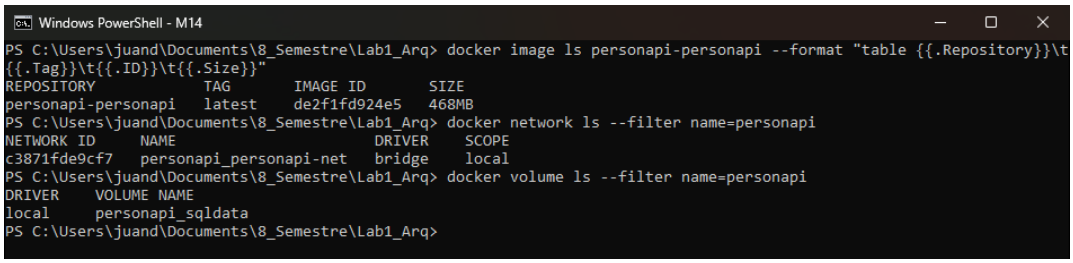


Figura 69: Paso M14: imagen `personapi-personapi`, red `personapi_personapi-net` y volumen `personapi_sqldata`.

Lo mismo se ve en Docker Desktop, filtrando por `personapi` en cada pestaña:

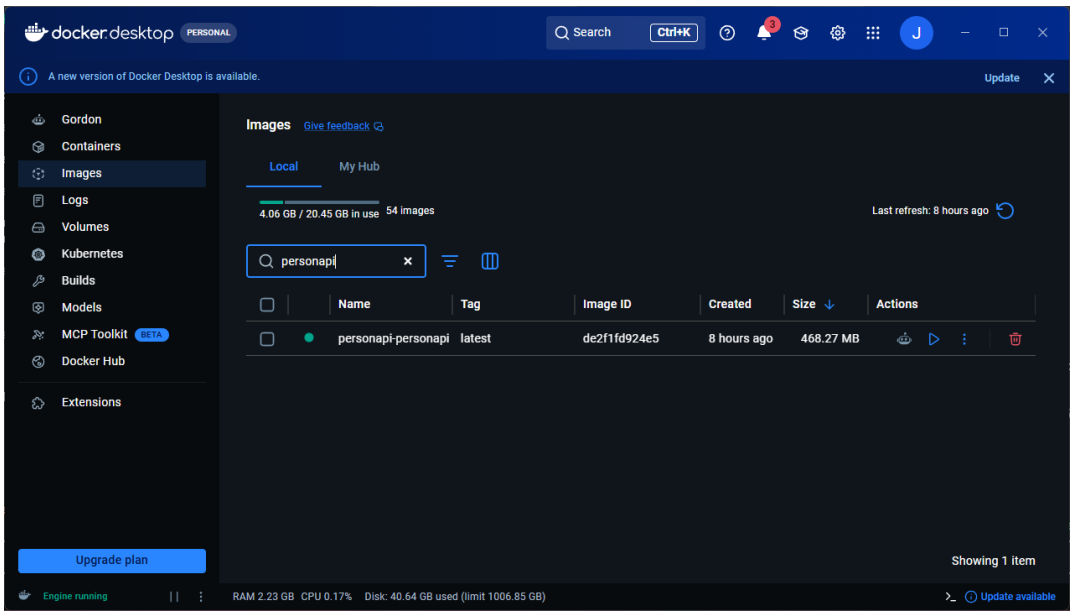


Figura 70: Paso M14: Docker Desktop, pestaña Images, con la imagen de la aplicación.

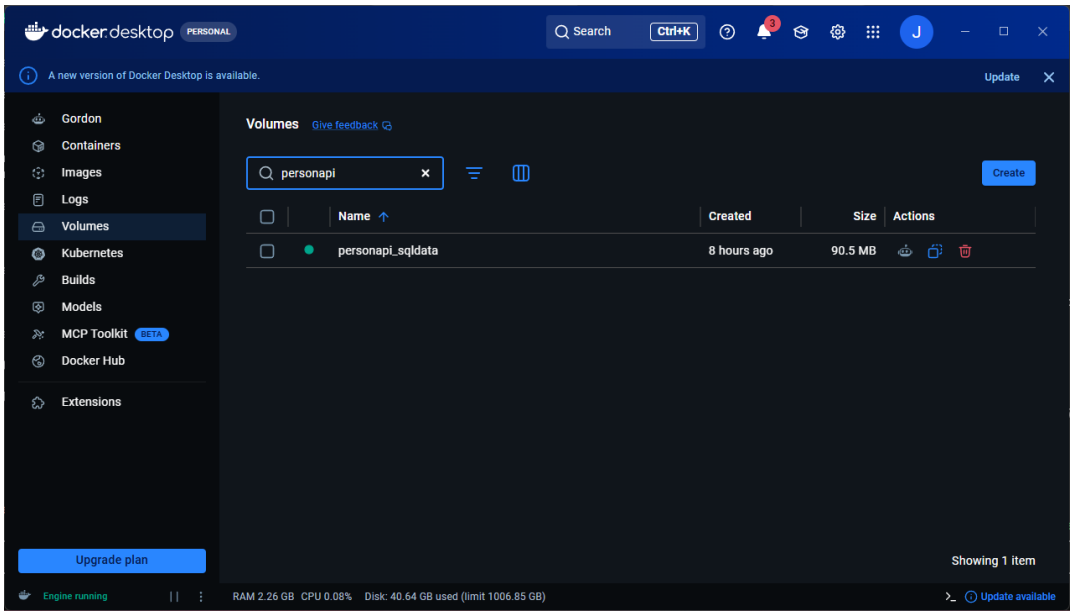


Figura 71: Paso M14: Docker Desktop, pestaña Volumes, con el volumen de la base.

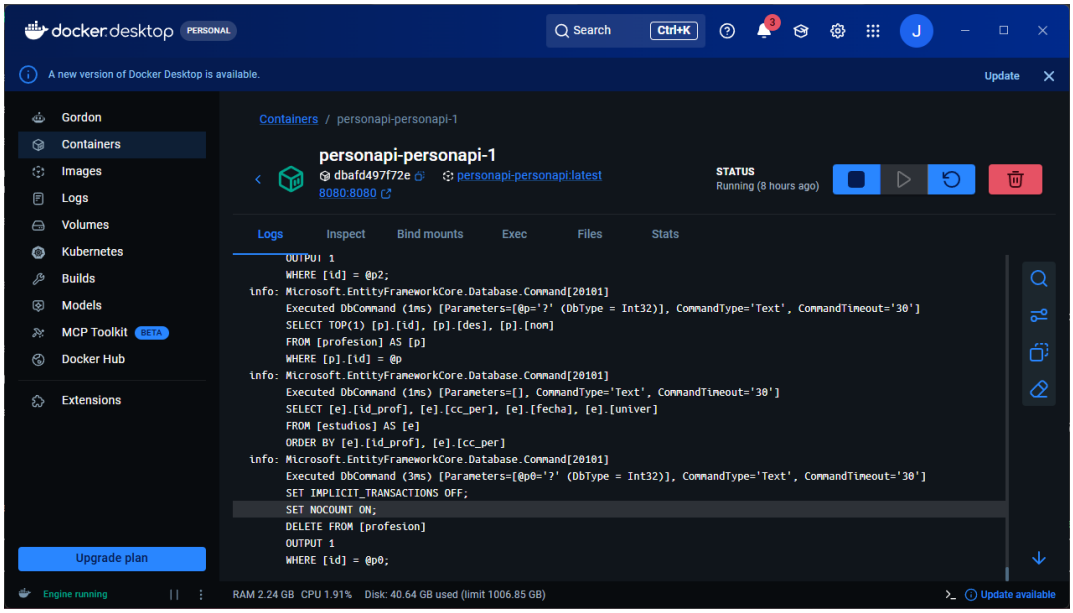


Figura 72: Paso M14: Docker Desktop, registros del contenedor `personapi-personapi-1`.

3.7.16 Camino B · Paso M15. Usar la aplicación

- Vistas web: `http://localhost:8080`
- Swagger UI: `http://localhost:8080/swagger`

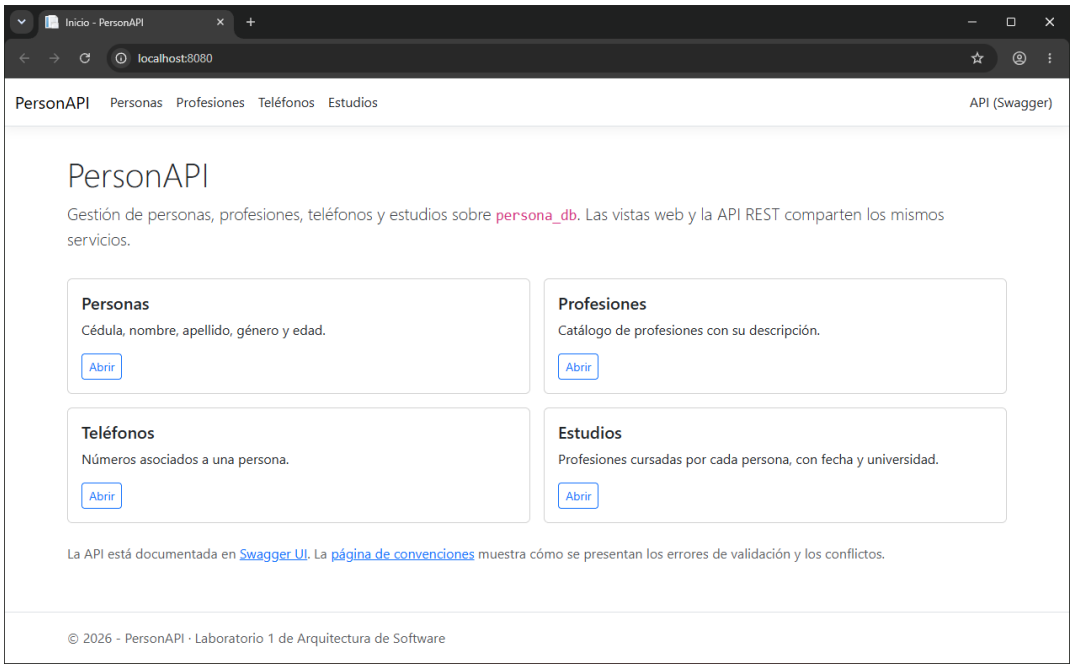


Figura 73: Paso M15: aplicación servida desde el contenedor.

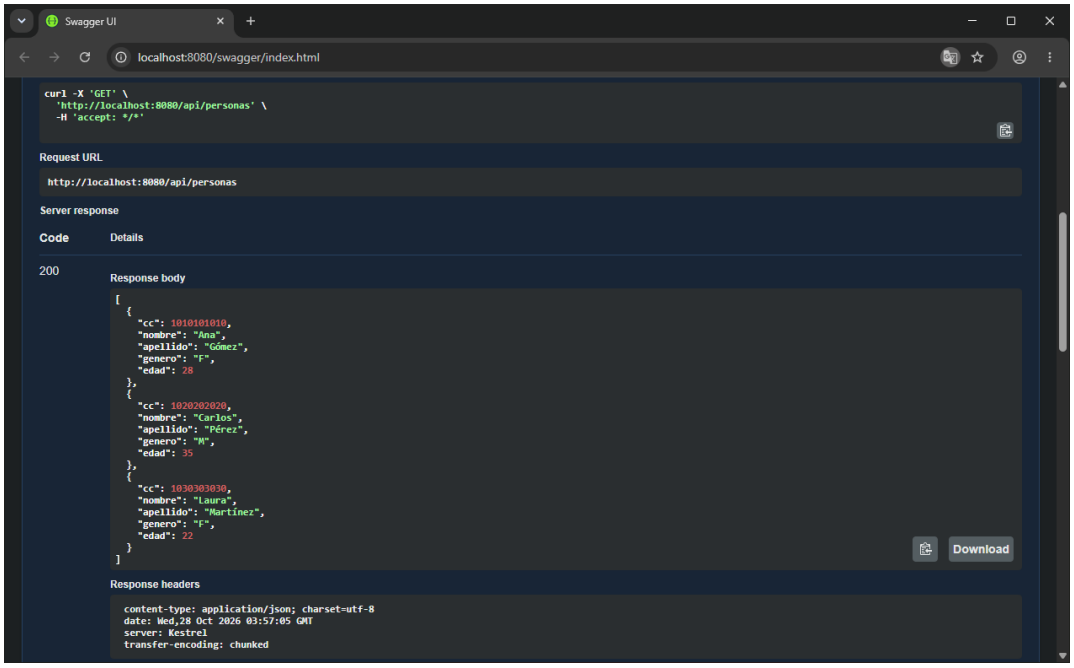


Figura 74: Paso M15: Swagger UI desde el contenedor.

Con Swagger UI se recorre un CRUD completo sin otra herramienta: en cada operación se pulsa **Try it out**, se llenan los parámetros o el cuerpo y se pulsa **Execute**. La secuencia siguiente crea la profesión 40, la modifica, la consulta, la elimina y comprueba que ya no existe, así que la base queda como estaba:

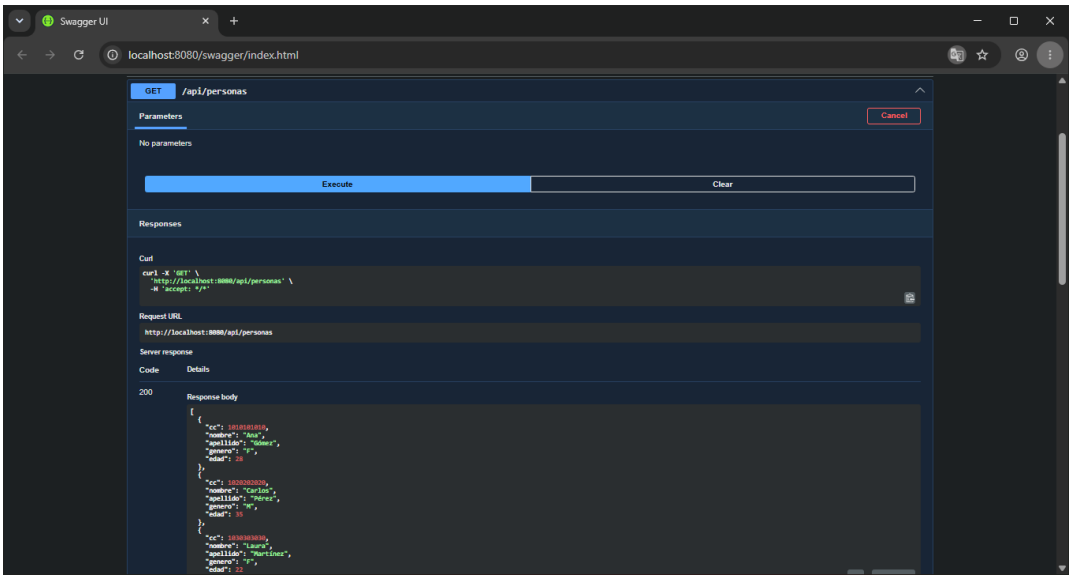


Figura 75: Paso M15: `GET /api/personas` con respuesta 200 y las tres personas.

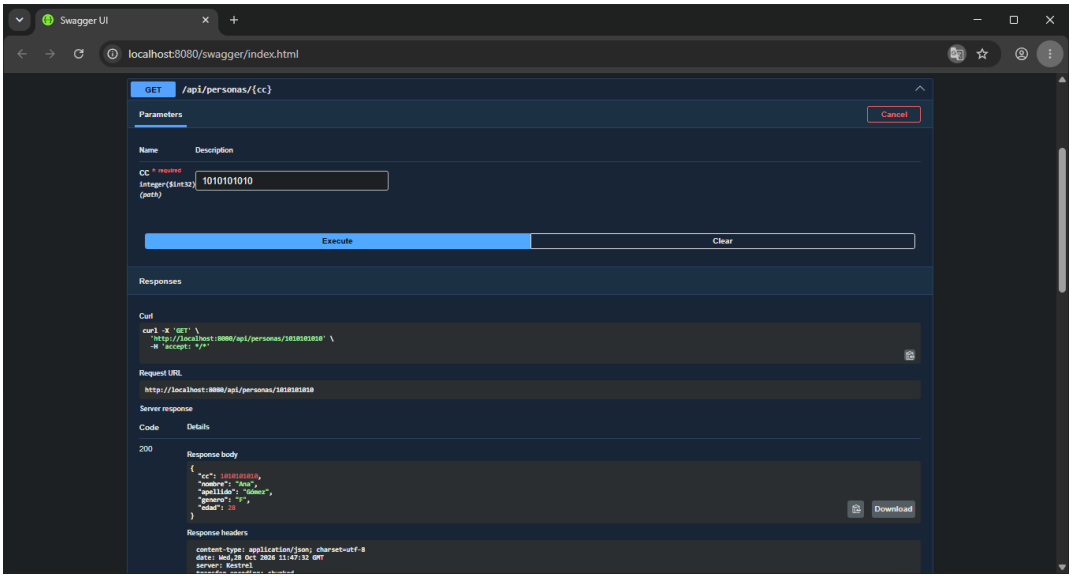


Figura 76: Paso M15: `GET /api/personas/1010101010` con respuesta 200.

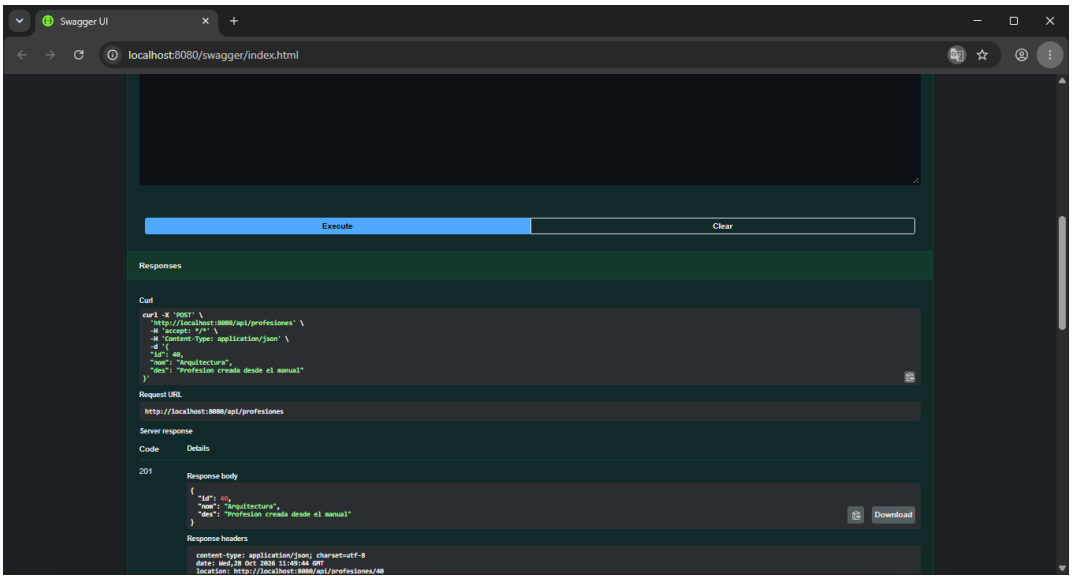


Figura 77: Paso M15: `POST /api/profesiones` con la profesión 40; respuesta 201.

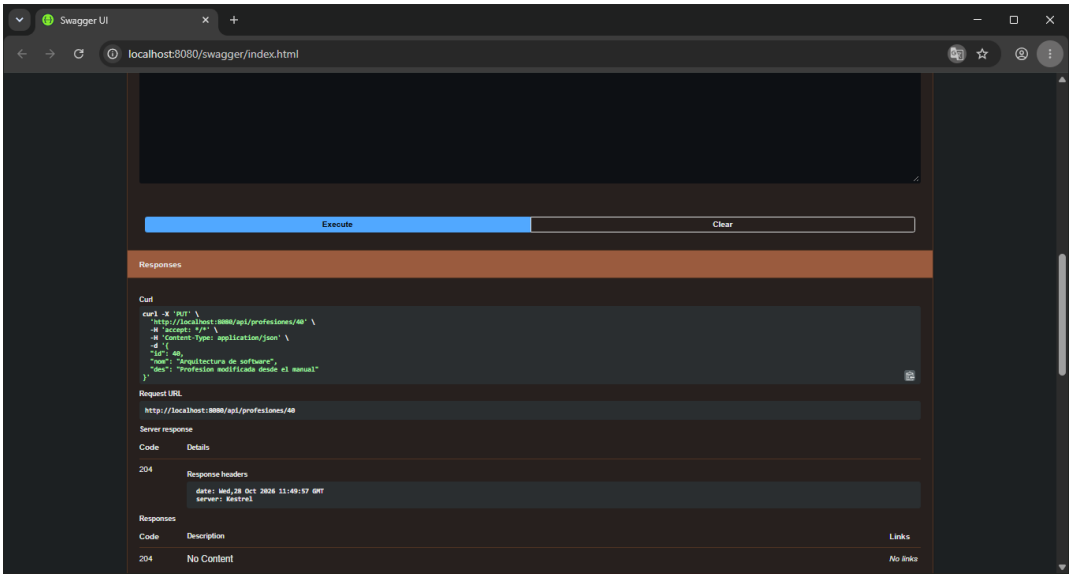


Figura 78: Paso M15: `PUT /api/profesiones/40`; respuesta 204.

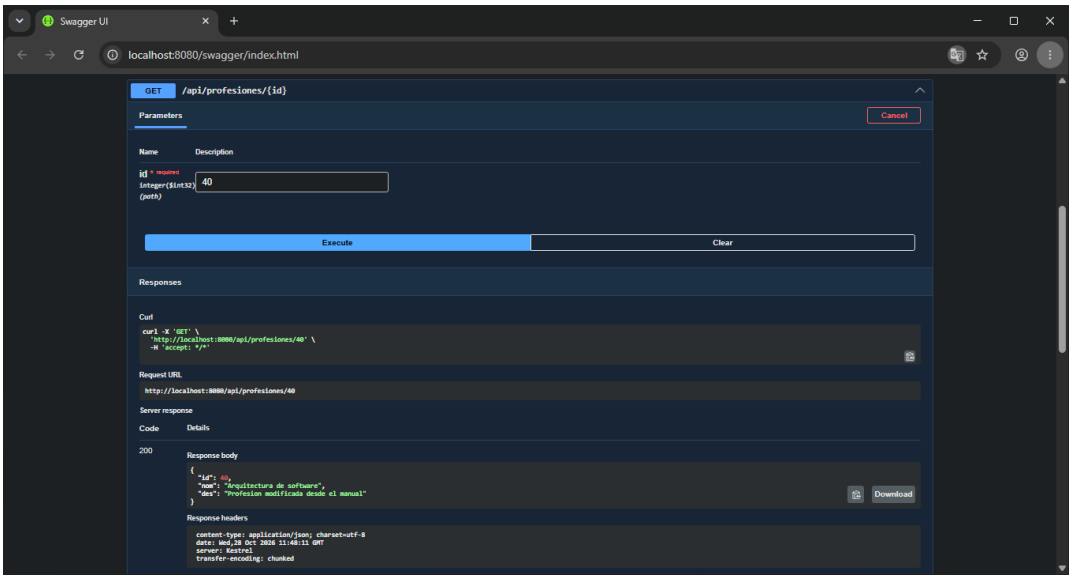


Figura 79: Paso M15: `GET /api/profesiones/40` devuelve la profesión modificada.

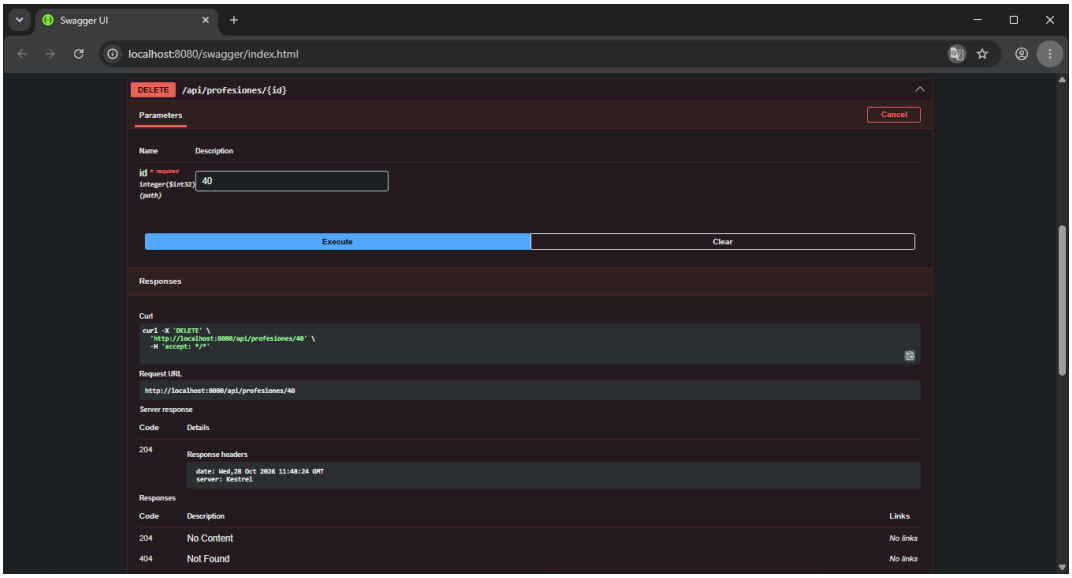


Figura 80: Paso M15: `DELETE /api/profesiones/40`; respuesta 204.

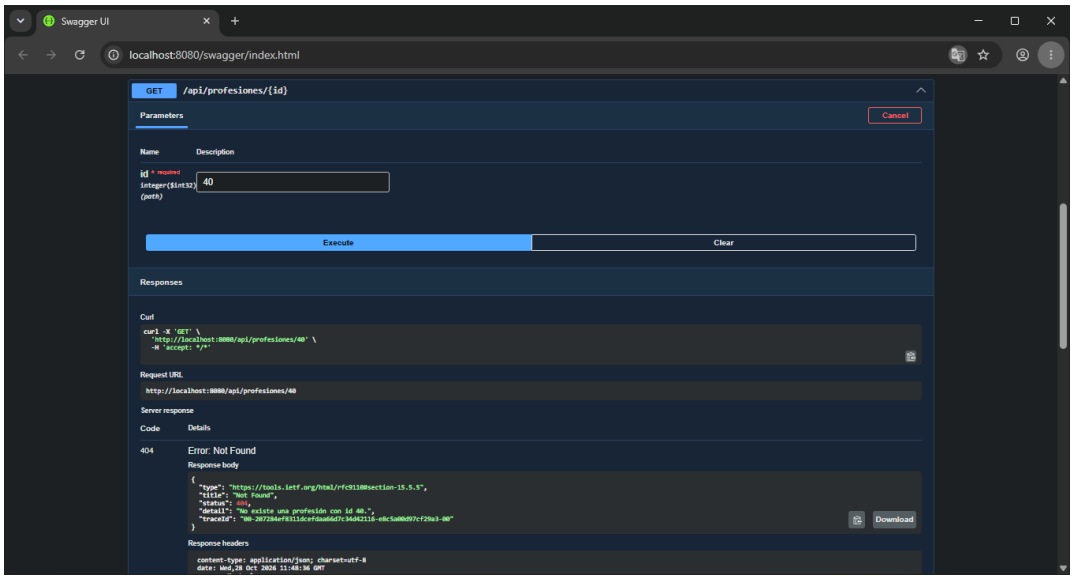


Figura 81: Paso M15: `GET /api/profesiones/40` después de eliminarla; respuesta 404 con `ProblemDetails`.

La interfaz MVC también responde en el 8080, por ejemplo la página de convenciones de errores:

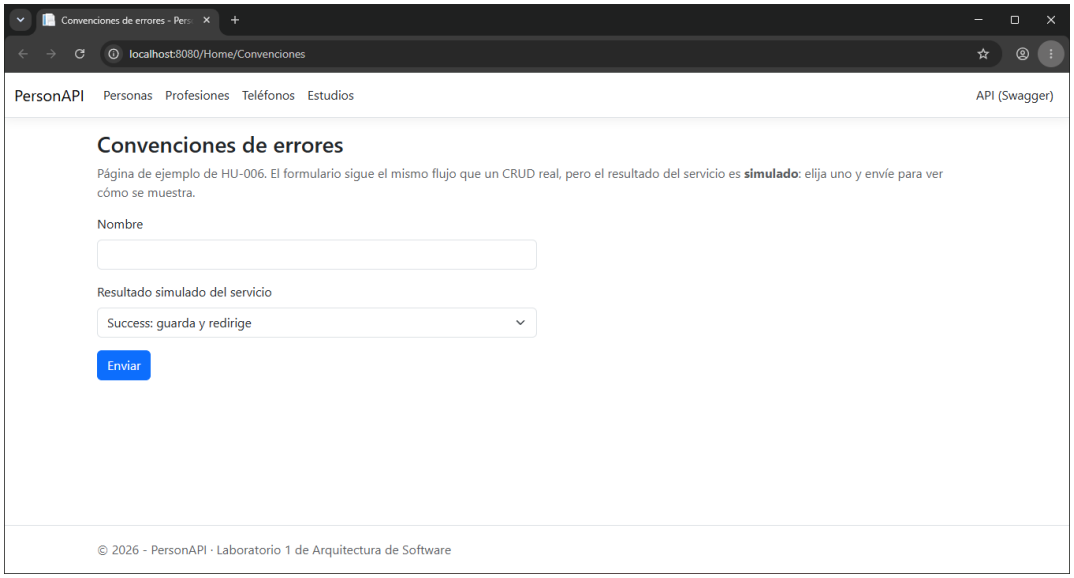


Figura 82: Paso M15: página Convenciones de errores en `http://localhost:8080/Home/Convenciones`.

3.7.17 Camino B · Paso M16. Detener y limpiar

```
docker compose down        # detiene y elimina los contenedores; conserva
    los datos
```

```
docker compose down -v     # ademas elimina el volumen sqldata: la base
    se pierde
```

Tras `down -v`, el siguiente `docker compose up -d` vuelve a crear la base desde los scripts.

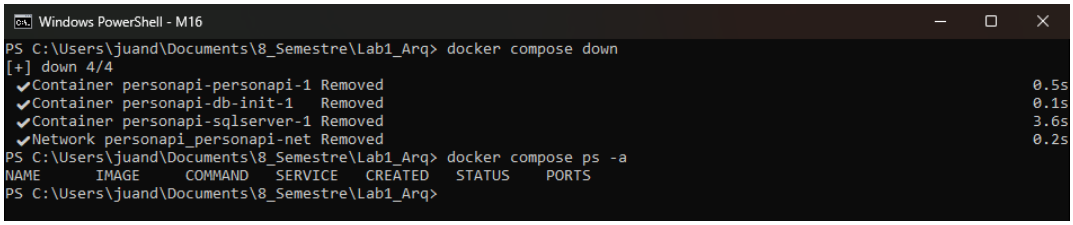


Figura 83: Paso M16: `docker compose down` elimina contenedores y red; el volumen se conserva.

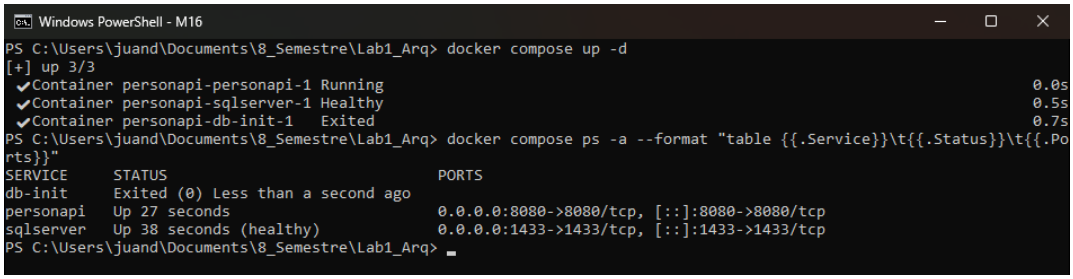


Figura 84: Paso M16: `docker compose up -d` vuelve a levantar la pila con los datos del volumen.

3.7.18 Verificación posterior al despliegue

Sirve para los dos caminos; solo cambia el puerto: `5093` en el camino A y `8080` en el B. Las comprobaciones se hacen desde el navegador, desde Swagger UI o con estos comandos de PowerShell:

```
$b = "http://localhost:8080"      # en el camino A: http://localhost:5093
(Invoke-WebRequest "$b/" -UseBasicParsing).StatusCode
                        # 200
(Invoke-WebRequest "$b/swagger/index.html" -UseBasicParsing).StatusCode
     # 200
Invoke-RestMethod "$b/api/personas" | ForEach-Object { $_ } | Format-
    Table cc, nombre, apellido
Invoke-WebRequest "$b/api/profesiones" -Method Post -ContentType "
    application/json" -Body '{"id":40,"nom":"Arquitectura","des":"Prueba
    de despliegue"}' -UseBasicParsing | Select-Object StatusCode    # 201
(Invoke-WebRequest "$b/api/profesiones/40" -Method Delete -
    UseBasicParsing).StatusCode        # 204
```

Tabla 6: Resultado esperado de la verificación posterior al despliegue.

Comprobación	Esperado	Qué demuestra
`GET /`	200	La aplicación MVC responde
`GET /swagger/index.html`	200	Swagger UI publicado
`GET /swagger/v1/swagger.json`	200, OpenAPI 3.0.4	Contrato OpenAPI 3
`GET /api/personas`	200 y las 3 personas del DML	Conexión real a `persona_db`
`POST /api/profesiones` (id 40)	201 con `Location`	Escritura en la base
`DELETE /api/profesiones/40`	204	La base queda como estaba

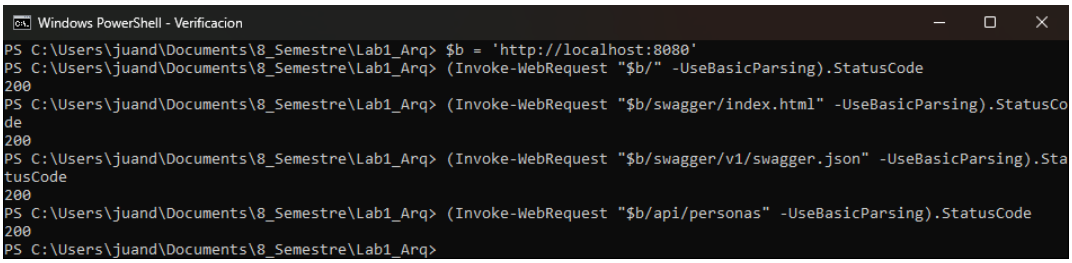


Figura 85: Verificación: códigos de estado de la página de inicio, Swagger UI, el documento OpenAPI y la API.

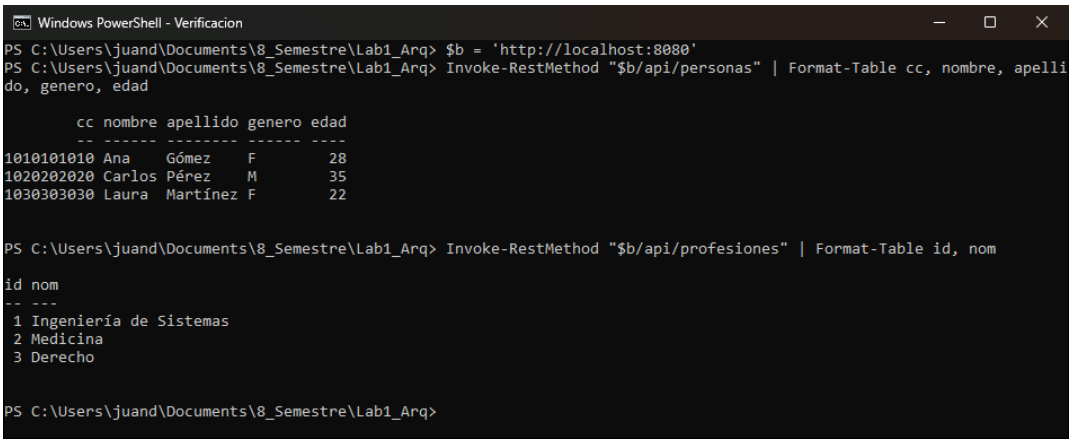


Figura 86: Verificación: datos de `/api/personas` y `/api/profesiones` leídos de `persona_db`.

3.7.19 Fallos frecuentes

Tabla 7: Fallos frecuentes del despliegue, su causa y su solución.

Síntoma	**Causa**	**Solución**
`/api/*` responde 500 y el log dice *No se ha inicializado la propiedad ConnectionString*	La cadena `ConnectionStrings:PersonaDb` no está definida (camino A)	Repetir el paso M9 en el mismo usuario y proyecto, o definir la variable de entorno en la misma terminal donde se ejecuta `dotnet run`.
`/api/*` responde 500 y el log dice *Cannot open database* con el nombre `persona_db`	La base no existe o la cadena apunta a otra base	Ejecutar el paso M8 y revisar el nombre `Database=persona_db` de la cadena.
`sqlcmd` o la aplicación no conectan: *A network-related or instance-specific error*	Conexión rechazada: el servicio `SQL Server (SQLEXPRESS)` está detenido o el nombre de la instancia es otro	Iniciar el servicio en `services.msc` y comprobar el nombre con SSMS (paso M4).
`docker compose up` falla con *Bind for 0.0.0.0:8080 failed: port is already allocated* (o 1433)	Puerto ocupado por otro programa o por otra instancia de los contenedores	Liberar el puerto (`netstat -ano | findstr :8080`) o cambiar el puerto publicado en `docker-compose.yml`, por ejemplo `8081:8080`.
`dotnet run` falla con *address already in use* en el 5093	Otra instancia de la aplicación sigue en ejecución	Cerrarla con `Ctrl+C` o terminar el proceso `personapi-dotnet`.
`docker compose up` se detiene con *Defina SA_PASSWORD en .env*	Falta el archivo `.env` o la variable está vacía	Repetir el paso M12.
`sqlserver` nunca llega a *healthy* y su log rechaza la contraseña	La contraseña no cumple la política de complejidad de SQL Server	Poner una contraseña más fuerte en `.env` y ejecutar `docker compose down -v` y luego `up -build -d`.
`docker` responde *cannot connect to the Docker daemon*	Docker Desktop no está abierto o su motor no arrancó	Abrir Docker Desktop y esperar a *Engine running* (paso M6).

Dos de estos fallos se reprodujeron a propósito en el equipo de verificación:

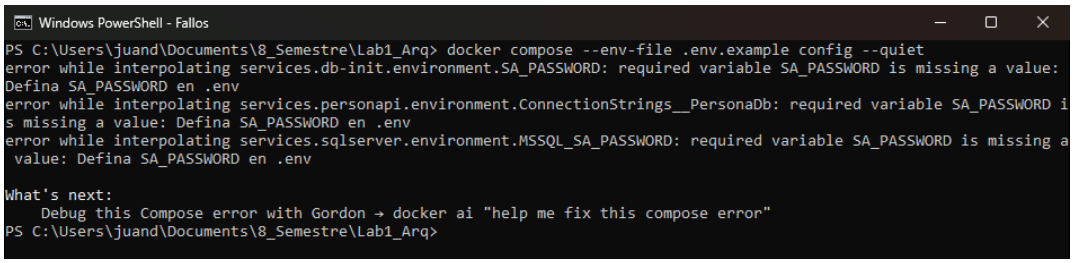


Figura 87: Fallo: `SA_PASSWORD` vacía; Compose se detiene con el mensaje *Defina SA_PASSWORD en .env*.

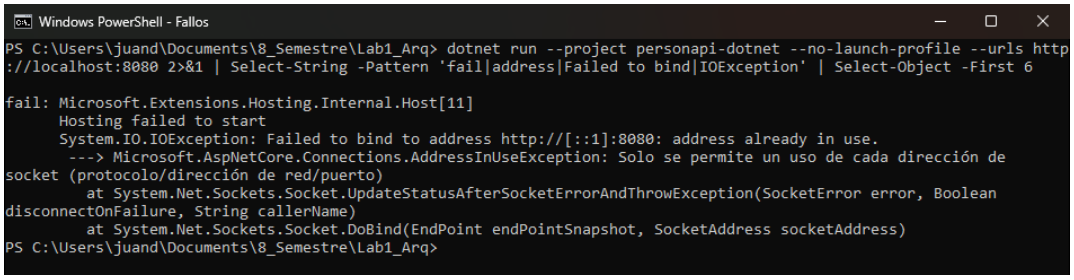


Figura 88: Fallo: puerto ocupado; `dotnet run` en el 8080, que ya usa el contenedor, termina con *address already in use*.

3.7.20 Registro de verificación

El 27 de octubre de 2026 se ejecutaron los dos caminos en un equipo con Windows 11 Pro, con las versiones de la Tabla 5:

- **Camino B desde cero.** Se eliminaron contenedores, red y volumen (`docker compose down -v`) y se ejecutó `docker compose up -build -d` sobre un clon nuevo del repositorio. `sqlserver` quedó *healthy*, `db-init` terminó con código 0 tras ejecutar los dos scripts y `personapi` escuchó en el 8080. Las comprobaciones de la Tabla 6 dieron los resultados esperados, y la consulta a la base del contenedor mostró 3 filas por tabla. Una segunda ejecución de `docker compose up -d` no duplicó datos.
- **Camino A.** Con la base local creada por los scripts, `dotnet run` escuchó en `http://localhost:5093`; `/` y `/swagger` respondieron 200, y `/api/personas` y `/api/profesiones` devolvieron los datos de `persona_db`. Sin cadena de conexión y con una base inexistente se reprodujeron los dos primeros fallos de la Tabla 7.

Capturas. Las figuras de este manual son capturas reales de ese mismo equipo, tomadas el 27 de octubre de 2026 al ejecutar cada paso: las ventanas de PowerShell, SSMS 22 y Docker Desktop, y Google Chrome sobre la aplicación en los puertos 5093 y 8080. El 28 de octubre de 2026 se añadieron las de instalación con `winget`, las del Visual Studio Installer, el CRUD por Swagger, los recursos de Docker y dos fallos reproducidos. En los pasos de instalación (M1

a M6), `winget install` se ejecutó de verdad sobre herramientas que ya estaban instaladas, así que las capturas muestran cómo detecta la versión existente y cancela la reinstalación; las pantallas de Modificar del Visual Studio Installer se cerraron sin cambios, y el asistente gráfico de Git se recorrió con los valores por defecto y se canceló antes de *Install*. Los asistentes de M2 a M6 (SDK de .NET 10.0.403, SQL Server 2025 Express, SSMS 22, `sqlcmd`, Visual Studio Community 2026 y Docker Desktop 4.97.0), descargados con `winget download`, también se recorrieron y se cerraron o cancelaron antes de instalar; las versiones de la tabla de requisitos no cambiaron. Ninguna captura muestra la contraseña de `sa`.

Pendiente: la ejecución de los pasos M1 a M6 en un equipo sin ninguna de las herramientas instaladas. En la fecha del registro, los recursos de teléfonos y estudios y las vistas CRUD estaban en desarrollo en otras historias; la verificación se limitó a lo integrado en ese momento.

3.8 Pruebas de funcionamiento

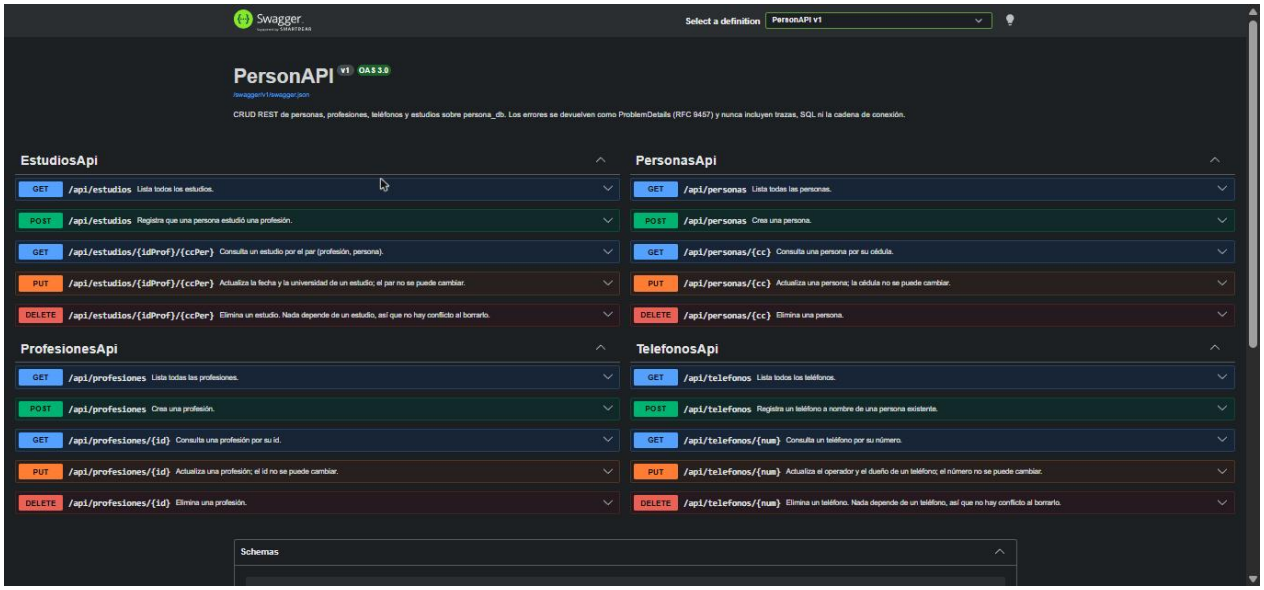


Figura 89: Swagger UI (`/swagger`) con las 20 operaciones REST agrupadas por recurso: estudios, personas, profesiones y teléfonos, cinco cada uno.

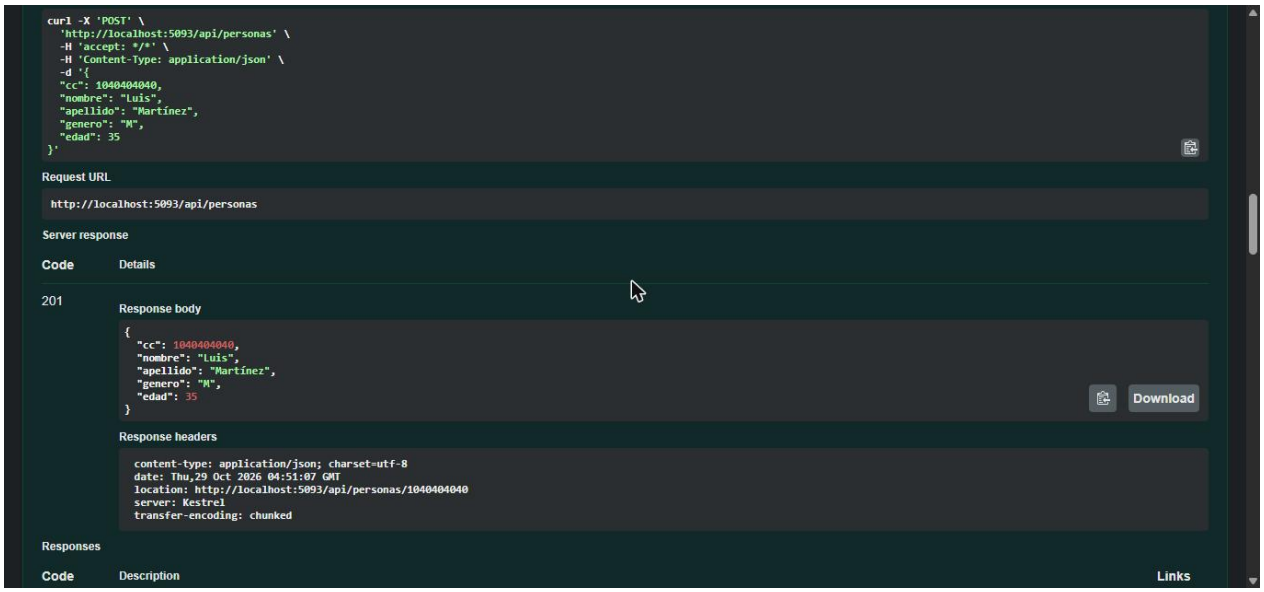


Figura 90: Ejecución de `POST /api/personas` desde Swagger UI: respuesta `201` con la persona creada y la cabecera `Location` hacia `/api/personas/1040404040`.



Figura 91: Vista `/Personas` (Index) con las tres personas que carga `scripts/02_dml.sql`.

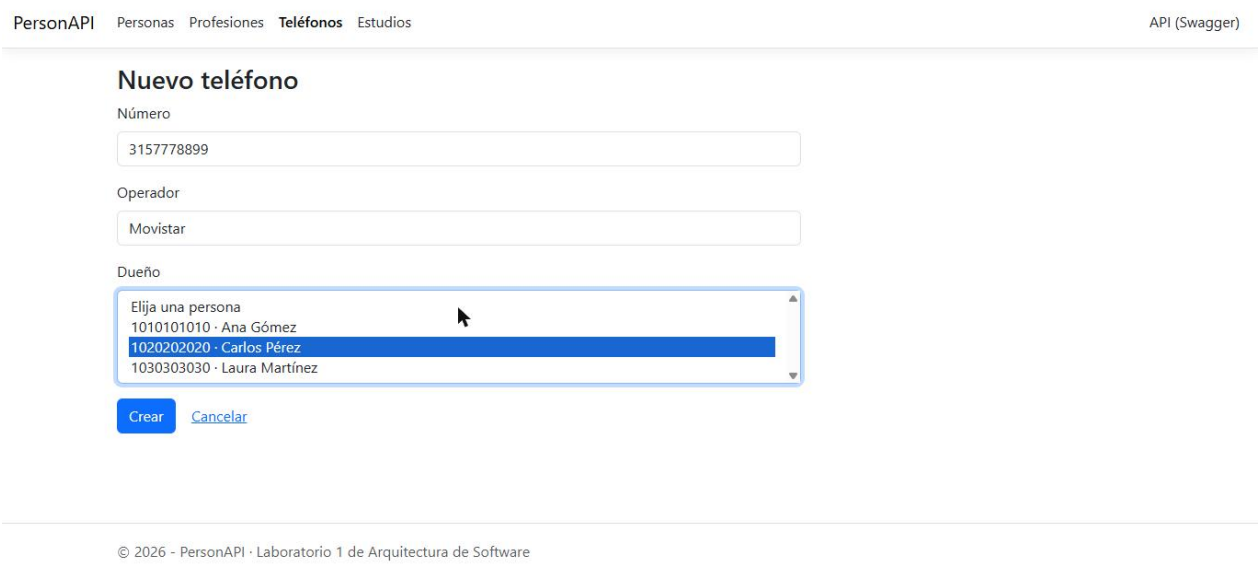


Figura 92: Formulario `/Telefonos/Create`: el selector de dueño ofrece las personas registradas (se muestra desplegado como lista para que se vean las opciones).

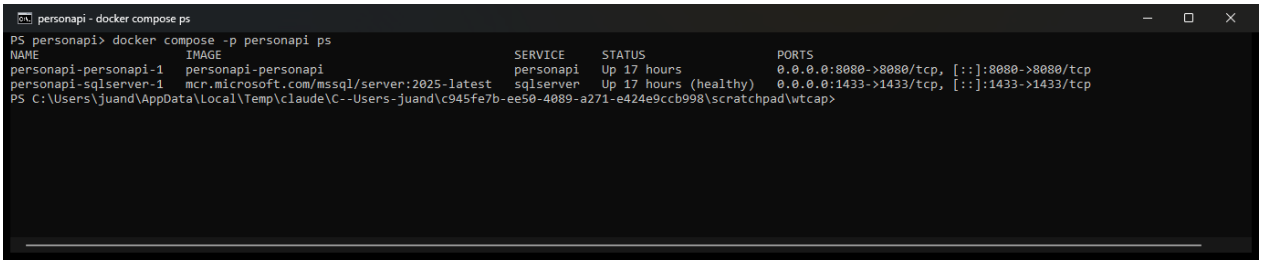


Figura 93: `docker compose ps`: `personapi` publica el puerto 8080 y `sqlserver` el 1433 en estado *healthy*; `db-init` ya terminó y no aparece.

Tabla 8: Matriz de pruebas funcionales.

#	Caso	Esperado	Resultado
1	Listar personas (API y vista)	200 / tabla	✓
2	Crear persona con cc nueva	201	✓
3	Crear persona con cc existente	409	✓
4	Crear persona con género distinto de M/F	400	✓
5	Actualizar persona existente	204	✓
6	Consultar persona inexistente	404	✓
7	Eliminar persona con teléfonos asociados	409	✓
8	Crear teléfono para dueño inexistente	400	✓
9	CRUD completo de profesiones	200/201/204	✓
10	CRUD de estudios con llave compuesta	200/201/204	✓
11	Despliegue completo con `docker compose up`	app en :8080	✓

3.9 Paso 9 y 10. Publicación y etiqueta

```
git add .
git commit -m "Laboratorio 1: monolito MVC + DAO con .NET 10 y SQL
    Server 2025"
git push origin main
git tag -a v1.0.0 -m "Entrega Laboratorio 1"
git push origin v1.0.0
```

Repositorio: https://github.com/USUARIO/personapi-dotnet – Tag: `v1.0.0`

4. Conclusiones y lecciones aprendidas

4.1 Conclusiones

- Un monolito bien estructurado en capas ofrece una separación de responsabilidades comparable a la de arquitecturas más complejas, con un costo operativo mucho menor: un solo artefacto, un solo despliegue y comunicación en proceso. Para un dominio pequeño como el de este laboratorio es la opción más adecuada.
- El patrón MVC permitió que una misma lógica de negocio sirviera dos interfaces distintas (vistas Razor y API REST) sin duplicar código: solo cambia el controlador y el formato de la respuesta.
- El patrón DAO aisló por completo a los controladores de Entity Framework. Gracias a la inyección de dependencias, cambiar la implementación de persistencia solo requeriría modificar una línea de registro en `Program.cs`.
- El enfoque *Database First* con `Scaffold-DbContext` redujo drásticamente el código repetitivo: a partir del esquema se obtuvieron entidades, relaciones y contexto listos para usar.
- Docker Compose hizo reproducible el entorno completo (base de datos, scripts y aplicación) con un solo comando, independientemente del sistema operativo de cada integrante.

4.2 Lecciones aprendidas

- **Migrar un esquema entre motores no es automático**: tipos como `ENUM` o `TEXT` de MySQL no existen o están obsoletos en SQL Server y deben reemplazarse por equivalentes (`CHECK`, `VARCHAR(MAX)`).
- **Las versiones importan**: .NET 10 exige Visual Studio 2026; la guía, basada en versiones anteriores, debió adaptarse. Elegir una versión LTS asegura soporte durante la vida del proyecto.

- **Las relaciones bidireccionales generan ciclos** al serializar a JSON (`Persona` → `Telefonos` → `Persona`); se resolvió con `ReferenceHandler.IgnoreCycles`. Una alternativa más limpia es exponer DTOs en lugar de entidades.
- **La autenticación de Windows no funciona en contenedores Linux**: la cadena de conexión debe externalizarse y sobrescribirse por variables de entorno según el ambiente.
- **Las llaves foráneas sin cascada deben manejarse en la aplicación**: en lugar de dejar que la excepción de SQL llegue al usuario, se valida previamente y se responde con un código HTTP significativo (`409 Conflict`).
- **Las llaves compuestas complican el enrutamiento**: la entidad `estudios` obliga a diseñar rutas con dos parámetros y vistas que identifican el registro por ambos valores.

4.3 Preguntas para el grupo

1. Entity Framework Core ya implementa internamente los patrones *Repository* (`DbSet`) y *Unit of Work* (`DbContext`). ¿Se justifica agregar una capa DAO propia encima de EF Core, o se trata de una abstracción redundante? ¿En qué escenarios aporta valor real (pruebas, cambio de motor, consultas complejas) y en cuáles solo añade código?
2. Si el sistema creciera (por ejemplo, millones de personas y equipos de desarrollo independientes para teléfonos y estudios), ¿qué señales indicarían que es momento de migrar este monolito a microservicios, y qué decisiones del diseño actual facilitarían o dificultarían esa transición?

Referencias

[1] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 4.ª ed. Boston: Addison-Wesley, 2021.

[2] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island: Manning, 2018. Patrón «Monolithic Architecture»: https://microservices.io/patterns/monolithic.html

[3] Microsoft Patterns & Practices, *Application Architecture Guide 2.0: Designing Applications on the .NET Platform*. Microsoft, 2009.

[4] T. Reenskaug, «Models – Views – Controllers», Xerox PARC, Technical Note, 1979. https://folk.universitetetioslo.no/trygver/themes/mvc/mvc-index.html

[5] Microsoft, «Overview of ASP.NET Core MVC», Microsoft Learn. https://learn.microsoft.com/aspnet/core/mvc/overview

[6] D. Alur, J. Crupi y D. Malks, *Core J2EE Patterns: Best Practices and Design Strategies*, 2.ª ed. Prentice Hall, 2003. Patrón «Data Access Object».

[7] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.

[8] Microsoft, «Directiva de soporte técnico de .NET y .NET Core». https://dotnet.microsoft.com/es-es/platform/support/policy/dotnet-core

[9] Microsoft, «Entity Framework Core» y «Scaffolding (Reverse Engineering)», Microsoft Learn. https://learn.microsoft.com/ef/core/managing-schemas/scaffolding/

[10] Microsoft, «What's new in SQL Server 2025», Microsoft Learn. https://learn.microsoft.com/sql/sql-server/what-s-new-in-sql-server-2025

[11] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, tesis doctoral, University of California, Irvine, 2000.

[12] OpenAPI Initiative, «OpenAPI Specification v3.1.0». https://spec.openapis.org/oas/v3.1.0

[13] Microsoft, «Get started with Swashbuckle and ASP.NET Core», Microsoft Learn. https://learn.microsoft.com/aspnet/core/tutorials/getting-started-with-swashbuckle

[14] Docker Inc., «Docker overview» y «Docker Compose overview». https://docs.docker.com/get-started/docker-overview/

[15] Microsoft, «Quickstart: Run SQL Server Linux container images with Docker». https://learn.microsoft.com/sql/linux/quickstart-install-connect-docker

[16] P. Deitel y H. Deitel, *Programación en la web*. Pearson Educación, 2008.

[17] hdeleon.net, «Video tutorial ASP.NET Core MVC con Entity Framework». https://www.youtube.com/watch?v=6nT-RjMEG0o

[18] hdeleon.net, «Video tutorial Web API con .NET». https://www.youtube.com/watch?v=28LjewDjaz4

[19] V. Guamán, «API con ASP.NET MVC 6 y SQL Server mediante Entity Framework Core 6 – Code First», partes 1 y 2, DEV Community. https://dev.to/veronicaguamann

[20] C# Corner, «Building ASP.NET Web API in .NET Core with Entity Framework». https://www.c-sharpcorner.com/article/building-asp-net-web-api-in-net-core-with-entity-framework/

[21] Material del curso Arquitectura de Software: «8.1 Introducción a la Plataforma .NET» y «8.2 Laboratorio .NET».